Comply iV (CiV)

System Specification

How the TCPA audit system works, top to bottom: backend, portal, flows, logic and functions

23 October 2026 · covers Modules 1–16, the user system and the fixes from the full browser test. Written from the code in the AUDIT (portal, docs) and Audit-backend repositories.

Contents

1. The system at a glance 4

What CiV is 4
Who uses it 4
How the parts fit together 5
The main flow, end to end 5
Principles that apply everywhere 6
Module map 7

2. Foundation and infrastructure (Module 1 and shared parts) 8

Configuration (config/settings.py, config/urls.py, config/celery.py) 8
Deploy and hosting (Dockerfile, deploy/, .github/workflows/deploy.yml, .env.example) 11
Tenancy lock (apps/tenancy) — Module 1 step 5 13
Client accounts and login (apps/accounts) 15
CiV staff accounts (apps/staff) 18
Admin panel (apps/admin_panel) 19
Client companies (apps/clients) 19
Onboarding (apps/onboarding) 21
Access log (apps/access_log) 22
Phone codes / keyring (apps/keyring) — Module 1 step 2 22
Fingerprint record / ledger (apps/ledger) — Module 1 step 3 23
Daily seal / anchoring (apps/anchoring) — Module 1 step 4 24
Background workers (apps/jobs) — Module 1 step 1 24
Contract & billing (apps/billing) 25
Client documents (apps/documents) 25
Frontend bridge (cross-cutting, frontend/src) 26

3. Rulebook and the law watcher (Modules 2 and 13) 27

1. The Rulebook file — backend/rulebook/v3.0.yaml 27
2. Rulebook app — apps/rulebook (Module 2) 30
3. Regulatory app — law changes, rule updates, lawyer reviews (Module 2 step 5–6) 34
4. AI law watcher — Module 13 (steps 1–4), in apps/regulatory 38
5. Module 13 step 5 — Rulebook admin (covered in section 2) 41
Tests present (names only) 41

4. Data intake: connectors, sync, normalizer, imports, reconciliation, uploads (Modules 3–5) 42

1. Connections (apps/connections) — Module 3 setup side 42
2. Live sync engine (apps/sync) — Module 3 47
3. Normalizer — Contacts (apps/contacts) — Module 4 52
4. Import batches (apps/imports) — Module 5 step 1 (+ Module 8 step 3) 55
5. Records completeness / reconciliation (apps/reconcile) — Module 5 step 3 (M37) 57
6. Upload centre (apps/uploads) — files for sources with no API 59
7. Portal screens (frontend) 62
8. Rulebook keys used by intake (rulebook/v3.0.yaml) 63
9. Privacy rules applied across intake (from code) 64

5. Consent, certificates, page capture, lead provenance and proof requests (Modules 6–8) 65

1. Certificates (apps/certificates) — finding and claiming TrustedForm / Jornaya certificates 65

2. Consent engine (apps/consent) — Module 6....68
3. Page capture (apps/captures) — Module 7 steps 3, 4, 6....76
4. Lead provenance (apps/provenance — ladder + evidence) — Module 8 steps 1–3, 5....78
5. Proof requests (apps/provenance/requests.py) — Module 8 step 4....80
Cross-area event map (who triggers whom)....82

6. Opt-outs, AI conversation review and contact checks (Module 9)....84

1. The STOP list (apps/optouts) — detection, deadline, re-added numbers....84
2. Did the tools mark the stop? (apps/optouts/marks.py, Module 9 step 3)....86
3. M08 / M09 and the Revocation page numbers (apps/optouts/measures.py, step 5)....88
4. Opt-outs card API (apps/optouts/api.py)....89
5. Alerts on opt-out problems (apps/alerts/raise_.py)....89
6. AI conversation review (apps/conversations, Module 9 steps 1, 2, 4; Module 16 consent flag)....90
7. "Who You Contact" page backend (apps/metrics/who.py)....93
8. Contact Conduct: calling hours, frequency, robo, abandon (apps/metrics/conduct.py)....95
9. "Can I call now?" and the number's calling window (apps/contacts)....96
10. Numbers page (evidence file) — opt-out parts (apps/contacts/api.py:number_file)....96
11. Scoring uses (for cross-reference)....97
12. Frontend wiring summary....97

7. Metrics, scoring, findings, alerts, reports and litigation (Modules 10–11)....99

Metrics (apps/metrics + metric producers in other apps)....99
Scoring (apps/scoring) — Module 10....109
Findings (apps/findings) — Module 11 steps 1, 2, 4....113
Alerts (apps/alerts) — Module 11 step 3....115
Reports (apps/reports) — Module 11 step 5....117
Litigation (apps/litigation) — Module 10 step 4b....119
Portal screens → endpoints....121
Tests present....121

8. Vendors, evidence vault, insurer views and document AI (Modules 12, 14, 15, 16)....122

Vendor scorecard (Module 12) — apps/vendors....122
Evidence vault (Module 14) — apps/evidence (+ vault stats in apps/anchoring)....127
Insurer views (Module 15) — apps/buyers....130
Document AI (Module 16) — apps/docai (+ spoken consent flag in apps/conversations)....136

9. The portal (frontend)....140

1. Tech stack....140
2. Folder layout....141
3. How data reaches a screen....142
4. Shell, layout and global UI....147
5. Views and roles....149
6. Page table....151
7. Other cross-cutting UI behaviour....164
8. Sample identities in the mock (for reference)....164
9. Tests (frontend/tests, run with npm test = vitest run)....165

1. The system at a glance

What CiV is

Comply iV (CiV) is an independent audit system for companies that call and text people for marketing: call centers, sellers, lead buyers, some making 10,000+ calls a day. It connects read-only to the tools a company already uses (dialers, texting platforms, CRMs, consent-certificate services, lead tools), reads every call and text, and checks each one against the Telephone Consumer Protection Act (TCPA) and state rules written down in CiV's **Rulebook**.

From that it produces, every day and every month:

- **Proof per contact**: was there consent, was an opt-out honored, was it inside calling hours, where did the number come from.
- **Metrics and scores**: about 45 measured metrics, an Audit Score for the company, and an Exposure Indicator (how likely and how costly a lawsuit is).
- **Findings and alerts**: problems in plain words, delivered inside the portal only (no email).
- **Evidence**: a sealed, fingerprinted record that can be exported for a lawyer or a court.
- **Insurer views**: a monthly risk attestation the company can share with its insurer or a buyer of the business.

CiV states facts and measurements. It does not give legal advice; how to respond is for the client and its lawyers.

Who uses it

Who	Where	What they do
Client Admin	Client portal	Connects tools, uploads files, manages users, shares with insurers, signs statements, sees everything for the company
Client Member	Client portal	Sees every page except Settings and Users & Access (Admin only); cannot change people, settings or evidence files
Client lawyer	Client portal (Member or Admin with the lawyer flag)	Only the lawyer reviews Rulebook drafts (agree, disagree, suggest) and suggests law changes; with Admins: signs statements, decides findings, uploads, exports evidence, sets legal holds
CiV staff	Admin panel (/civ-admin/)	Onboards clients (with the client Admin's invitation), approves Rulebook changes, reviews proof files, manages insurers, sends password links, runs the system; separate login from clients; every staff member has full access for now
Insurer or acquirer	Insurer portal (same portal, underwriter view)	Sees only companies that share with it: Portfolio, Exposure, Data Integrity, Risk Attestation, Litigation,

Who	Where	What they do
		Underwriting Export

How the parts fit together

```
Client's tools (dialers, texting, CRM, certificates, lead tools)
       |  read-only APIs, polled every 2 minutes (no inbound webhooks)
       v
Django backend (Audit-backend repo)  <-->  PostgreSQL (row-level security per company)
  - Celery workers + Redis: sync, nightly jobs, monthly jobs
  - REST API under /api/  (DRF)
  - Admin panel for CiV staff under /civ-admin/
       |  JSON over HTTPS (session cookie + CSRF)
       v
Next.js portal (AUDIT repo, frontend/)
  - /api/* route forwards to Django; partly real pages are laid over sample data
  - Client view, and underwriter (insurer) view
       |
       v
People: client users, client lawyers, insurers.  Outbound only: insurer rating engines (signed
JSON).
```

- **Backend**: Django 6 + Django REST Framework, PostgreSQL, Redis + Celery (workers and a beat schedule), reportlab for PDFs, Playwright for page captures, cryptography for locked keys.
- **Frontend**: Next.js + TypeScript, TanStack Query, Tailwind CSS; tests with Vitest and MSW.
- **Hosting**: Docker on a Hostinger VPS behind Traefik (shared with n8n).
- **Two repositories**: AUDIT holds the docs and the portal; Audit-backend holds the backend.

The main flow, end to end

1. **Onboarding.** CiV staff create the client with its Admin's name and email; the Admin gets an invitation link, chooses a password, and invites their own team (with a lawyer tick box).
2. **Connecting.** The Admin (or CiV with a connect link) saves a read-only key for each tool. The key is encrypted at once and never shown again.
3. **Fetching.** Every 2 minutes the sync workers ask each tool for new calls, texts, contacts and leads. Hourly catch-ups and a nightly full sweep fill gaps. Phone numbers are turned into **phone codes** (keyed per client) the moment they arrive; the number itself is never stored.
4. **Normalizing.** Every record becomes one standard **contact event** (who, when, channel, direction, which tool, source grade), with duplicates across tools merged and each number's ownership period ("epoch") tracked.
5. **Fingerprinting.** Each record gets a SHA-256 fingerprint in a ledger; every night the day's fingerprints are sealed and time-stamped, so later changes can be detected.

6. **Checking.** Each contact goes through the engines: consent (is there valid written consent for this seller?), certificates (claimed automatically), page capture (what the consent page really said), lead provenance (where the number came from), opt-out and revocation (did the person say stop; was it honored within 10 business days), the AI conversation review (texts and call recordings), calling hours and frequency, internal and national do-not-call lists.
7. **Uploads.** What a tool cannot give by API comes in through the Upload centre: internal DNC list, DNC check records, reassigned-number queries, complaints, policies and training, vendor lead files and vendor contracts.
8. **Reconciling.** Counts are checked against independent totals (bills, the tool's own totals, plausibility) to see whether CiV was given all records.
9. **Measuring.** Each night the metrics are computed; each month a snapshot is saved: metric scores, factors F1–F6, the defect term, the Exposure Indicator, evidence coverage.
10. **Reporting.** Findings and alerts are refreshed; monthly reports are made on the 6th; the evidence vault can export a sealed bundle for any number.
11. **Sharing.** Each night a monthly risk attestation is saved with its fingerprint. If the client shares with an insurer, the insurer sees it, and can have it sent to its own rating engine.
12. **Keeping the rules current.** The law watcher reads regulatory sources every morning, drafts Rulebook changes, clients' lawyers review them, and CiV staff approve them in the admin panel.

Principles that apply everywhere

- **Phone numbers are never stored.** Only phone codes (keyed per client) and masked labels.
- **API only** for tool data, plus the Upload centre for sources that have no API.
- **No inbound hooks.** The public door is closed; CiV polls. Reopening needs the owner's OK and three security fixes.
- **Same setup for every client**, strictest defaults. No per-client engine settings.
- **Unverified counts as worst case.** A field CiV cannot verify takes its pessimistic value; a company's statement can lower a score, never raise it.
- **Not measured is not the worst case on screens.** A metric CiV has not built or cannot measure says "Not measured"; scoring then treats it as described in the scoring chapter.
- **Every company's data is locked in the database itself** (row-level security), not only in the code.
- **Files are private and kept 4 years**, then deleted with their fingerprints kept; legal holds stop deletion.
- **Alerts are in the portal**, and each company's **Admins** can also get them by email (they choose which types in Settings → Notifications).
- **Nobody types a password for someone else.** People join by an invitation link and choose their own password; CiV staff send password links; everyone changes their own in My account. There is no "Forgot password".

- **Seeing vs doing.** Every client user sees every page except Settings and Users & Access (Admin only); what they can change depends on their role (Admin, Member, lawyer). Each person has their own "View as" switch.
- **The AI is plugged into fixed slots** (conversation review, speech-to-text, document reading). Until a slot is switched on, nothing is sent anywhere. Every AI must pass an accuracy exam before its answers count.

Module map

No.	Module	What it adds
1	Data foundation	Workers, phone codes, fingerprint ledger, daily seal, company lock in the database
2	Rulebook	Every rule as data (YAML), versions, lookups by date and state
3	Connectors + live sync	Read-only connections to tools, polling every 2 minutes
4	Normalizer	One standard contact event from every tool
5	Imports + reconciliation	Batches, completeness against independent totals (Gate 1)
6	Consent engine	Consent result per contact with reason codes
7	Certificates + page capture	Automatic certificate claiming, screenshots of consent pages
8	Lead provenance + proof requests	Where each number came from; asking the client for proof (Gate 2)
9	Revocation + AI review	Opt-outs, deadlines, AI reading texts and calls (Gate 3)
10	Metrics + scoring	Metrics, Audit Score, Exposure Indicator, monthly snapshots
11	Findings, alerts, reports	Problems in plain words, alerts, PDF reports
12	Vendor scorecard	Lead vendors scored monthly; contracts and disputes
13	Rulebook admin + law watcher	Regulatory changes, lawyer review, CiV approval
14	Evidence vault	Evidence per number, re-fetch, sealed bundles, legal holds (Gate 4)
15	Insurer views	Sharing, statements, attestation snapshots, insurer pages, rating engine (Gate 5)
16	Document AI	Readers for contracts, consent pages, policy and training files; spoken consent flag

The chapters that follow describe each part from the code: purpose, data and its source, tables, flow, logic, functions, API endpoints, scheduled jobs, admin panel and portal pages.

2. Foundation and infrastructure (Module 1 and shared parts)

Read from the code on 22 October 2026. Backend root: `/home/shakib/Audit/backend`. Paths below are relative to it unless they start with `frontend/`.

Configuration (config/settings.py, config/urls.py, config/celery.py)

- **Purpose:** one settings file; every secret and per-machine value comes from `.env` (loaded with python-dotenv).
- **Data it receives:** environment variables only.

Key settings (real values)

- `SECRET_KEY` = `DJANGO_SECRET_KEY` (required, crash if missing). `DEBUG` = `DJANGO_DEBUG` (default false).
- `ALLOWED_HOSTS` = `DJANGO_ALLOWED_HOSTS` (default `localhost,127.0.0.1`); `CSRF_TRUSTED_ORIGINS` default `http://localhost:3000`.
- `AUTH_USER_MODEL = 'staff.StaffUser'` (Django's user = CiV staff, NOT client users). `LOGIN_URL=/civ-admin/login/`, `LOGIN_REDIRECT_URL=/civ-admin/`.
- **Database:** PostgreSQL. Two logins:
  - Owner (`DB_USER`/`DB_PASSWORD`) — used when `DB_AS_OWNER=1` or when no `DB_APP_USER` is set.
  - Limited app user (`DB_APP_USER`/`DB_APP_PASSWORD`) — used day to day when set, so row-level security applies.
- **Passwords:** Argon2 for new hashes; BCryptSHA256 accepted and upgraded on next sign-in. Validators: user-attribute similarity, **min length 12**, common-password, numeric-only.
- **Cookies/sessions:** `SESSION_COOKIE_HTTPONLY=True`, `SAMESITE='Lax'`, `SECURE` and `CSRF_COOKIE_SECURE` = not DEBUG, `SESSION_COOKIE_AGE = 12 hours`, `PASSWORD_RESET_TIMEOUT = 1 hour`. Session engine = Django default (database, table `django_session`).
- **REST framework:** JSON renderer only; auth classes `apps.accounts.authentication.ClientSessionAuthentication` then `apps.buyers.authentication.BuyerSessionAuthentication` (Module 15); default permission `AllowAny` (each view sets its own); `UNAUTHENTICATED_USER=None`.
- **Email (Gmail, 2026-10-23):** with `GMAIL_USER` + `GMAIL_APP_PASSWORD` in `.env` (a Google app password), `MAILERS` uses Gmail SMTP (smtp.gmail.com:587, TLS); without them emails print to the terminal. `DEFAULT_FROM_EMAIL` = `Comply iV <GMAIL_USER>` (else `CiV <no-reply@complyiv.com>`). Every email uses one design (`apps/accounts/mailer.py`, `templates/emails/base.html`: heading, lines, one button, plain-text copy). `PORTAL_URL` is the base for portal links, `ADMIN_URL` for staff welcome links.
- **Admin theme:** `UNFOLD = unfold_config(PORTAL_URL)` (see Admin panel).
- **Celery:** `REDIS_URL` → broker; without it broker `memory://` and `CELERY_TASK_ALWAYS_EAGER=True` (jobs run at once, in-process). `EAGER_PROPAGATES=True`, `IGNORE_RESULT=True`, `ACKS_LATE=True` (crashed

worker's job re-runs), `PREFETCH_MULTIPLIER=1`, timezone UTC. `CELERY_TASK_ROUTES`: `apps.captures.tasks.capture_pages` → queue `capture`.

- **Hooks:** `SYNC_HOOKS_ENABLED` (default false). Only when true is `/hooks/<connection_id>` routed (owner decision 2026-10-16: polling only).
- **OAuth app registrations:** `SALESFORCE_CLIENT_ID/SECRET`, `HUBSPOT_CLIENT_ID/SECRET` (only to renew a client's token).
- `SYNC_BATCH_SIZE` default 500 records per turn.
- **Anchoring:** `ANCHOR_TSA_URLS` default `https://freetsa.org/tsr,http://ts.ssl.com`; `ANCHOR_OTS_CALENDARS` default `https://a.pool.opentimestamps.org,https://b.pool.opentimestamps.org`.
- Locale `en-us`, `TIME_ZONE='UTC'`, `USE_TZ=True`, `FORMAT_MODULE_PATH=['config.formats']` (portal-style dates).
- **Files:** `MEDIA_ROOT` env (default `BASE_DIR/media`), private; `DATA_UPLOAD_MAX_MEMORY_SIZE = 25 MB`. Static via WhiteNoise in production (`CompressedManifestStaticFilesStorage`), plain storage in DEBUG.
- **Proxy/HTTPS:** `SECURE_PROXY_SSL_HEADER=('HTTP_X_FORWARDED_PROTO','https')` (Traefik ends TLS). Non-DEBUG: `SECURE_HSTS_SECONDS` = `DJANGO_HSTS_SECONDS` default 3600; `SECURE_CONTENT_TYPE_NOSNIFF=True`.
- `CHROMIUM_PATH` (empty = Playwright's own browser). `CONVERSATION_REVIEW_MODEL` (empty = no AI chosen; nothing sent anywhere; owner decision 2026-10-21).
- `CIV_SECRETS_KEYS` (read in `apps/connections/crypto.py`, not settings.py): comma list of Fernet keys; first one locks, any one unlocks (MultiFernet). Missing → `ImproperlyConfigured`.

Installed apps (order as in settings)

`apps.admin_panel` (before unfold so its dashboard wins), `unfold`, django admin/auth/contenttypes/sessions/messages/staticfiles, `rest_framework`, then: staff, access_log, clients, accounts, connections, onboarding, billing, documents, jobs, keyring, ledger, anchoring, tenancy, rulebook, regulatory, sync, certificates, contacts, imports, optouts, reconcile, consent, captures, provenance, uploads, metrics, scoring, litigation, findings, alerts, reports, vendors, evidence, buyers, docai (Module 16), conversations.

Middleware order

SecurityMiddleware → (WhiteNoise inserted at position 1 when not DEBUG) → Sessions → Common → CSRF → Authentication → **`apps.tenancy.middleware.TenantScopeMiddleware`** → Messages → XFrameOptions.

CELERY_BEAT_SCHEDULE — every scheduled task (all times UTC)

Beat name	Task	When
heartbeat	apps.jobs.tasks.heartbeat	every 60 s
seal-nightly	apps.anchoring.tasks.seal_nightly	daily 00:30

Beat name	Task	When
stamp-missing	apps.anchoring.tasks.stamp_missing	hourly at :15
sync-poll	apps.sync.tasks.poll_due	every 120 s
sync-catchup	apps.sync.tasks.catchup_all	hourly at :40
sync-full-sweep	apps.sync.tasks.full_sweep_due	daily 03:10
sync-health	apps.sync.tasks.check_health	every 600 s (10 min)
claim-certificates	apps.certificates.tasks.claim_certificates	every 60 s (72-hour claim window)
consent-needs	apps.consent.tasks.work_out_needs	every 60 s
consent-recheck	apps.consent.tasks.keep_results_up_to_date	every 60 s
provenance	apps.provenance.tasks.rebuild_provenance	every 30 min (`minute='*/30'`)
purge-old-proof-files	apps.provenance.tasks.purge_old_proof_files	daily 04:50
purge-old-uploads	apps.uploads.tasks.purge_old_uploads	daily 04:55
conversation-review	apps.conversations.tasks.review_conversations	every 60 s (off until an AI model is chosen)
conversation-exam	apps.conversations.tasks.run_exams	daily 02:10
webrecon-litigation	apps.litigation.tasks.read_webrecon	Mondays 05:30
alerts	apps.alerts.tasks.raise_alerts	every 300 s (5 min)
monthly-reports	apps.reports.tasks.monthly_reports	6th of each month 06:00
read-documents	apps.docai.tasks.read_documents	every 15 min (Module 16)
doc-exams	apps.docai.tasks.run_doc_exams	daily 02:20
rating-sends	apps.buyers.tasks.monthly_rating_sends	6th of each month 06:30 (Module 15)
purge-old-contracts	apps.vendors.tasks.purge_old_contracts	daily 04:50
law-watcher	apps.regulatory.tasks.watch_laws	daily 06:15 (Module 13)
weekly-refetch	apps.evidence.tasks.weekly_refetch	Sundays 03:40 (`day_of_week=0`) (Module 14)
metric-snapshots	apps.scoring.tasks.snapshot_metrics	daily 04:00 (Module 10)
optout-mark-check	apps.optouts.tasks.check_marks	every 300 s (urgent within 15 min of an opt-out)
capture-pages	apps.captures.tasks.capture_pages	hourly at :20 (each page ~once/day; routed to `capture` queue)
purge-old-captures	apps.captures.tasks.purge_old_captures	daily 04:45

Beat name	Task	When
consent-nightly	apps.consent.tasks.recheck_time_passed	daily 03:00 (expiring certificates)
proof-dates	apps.consent.tasks.refresh_proof_dates	every 300 s
import-batches	apps.imports.tasks.rebuild_batches	every 900 s (15 min)
records-completeness	apps.reconcile.tasks.check_completeness	daily 02:30 (this month and last)

32 entries. Note purge-old-proof-files and purge-old-contracts share 04:50.

config/urls.py

- `''` → `root_redirect` (302 to `/civ-admin/`).
- `civ-admin/` → Django admin (Unfold).
- `api/health` → `config/health.py:health` (GET, anyone; runs `SELECT 1`, returns `{"status":"ok"}`).
- `api/` includes, in order: accounts, rulebook, regulatory, sync, contacts, optouts, consent, captures, provenance, access_log, litigation, scoring, findings, alerts, reports, vendors, evidence, buyers, anchoring, clients, reconcile, uploads, metrics, onboarding.
- `hooks/<int:connection_id>` → `apps.sync.views.hook` only if `SYNC_HOOKS_ENABLED`.
- Note: `apps.connections`, `apps.imports`, `apps.docai`, `apps.conversations`, `apps.jobs`, `apps.billing`, `apps.documents`, `apps.ledger`, `apps.keyring` have no `api/` include here (they are reached through other apps' views or background jobs).

config/celery.py

Celery app `'civ'`, config from Django settings namespace `CELERY`, `autodiscover_tasks()` (each app's `tasks.py`).

Deploy and hosting (Dockerfile, deploy/, .github/workflows/deploy.yml, .env.example)

- **Purpose:** run CiV in Docker on a server that already runs Traefik (README: the Hostinger VPS that also runs n8n; n8n untouched).

Dockerfile (backend image)

1. `python:3.14-slim` + `uv` 0.12 copied from `ghcr.io/astral-sh/uv:0.12`.
2. `uv sync --frozen --no-dev --no-install-project` (packages layer), then copy code.
3. `collectstatic` at build with a throwaway `DJANGO_SECRET_KEY=build-only`.
4. Non-root user `civ` (uid 10001), `/app/media` owned by it. Exposes 8000. Entrypoint `deploy/entrypoint.sh`.

deploy/entrypoint.sh (backend start)

1. `DB_AS_OWNER=1 python manage.py migrate --noinput`
2. `DB_AS_OWNER=1 python manage.py create_app_role` (create/refresh limited DB user)
3. `python manage.py load_rulebook` (rulebook/*.yaml; unchanged files skipped)
4. `exec gunicorn config.wsgi --bind 0.0.0.0:8000 --workers ${GUNICORN_WORKERS:-3} --timeout 60 --access-logfile -`

deploy/Dockerfile.capture

Same base + apt `chromium`, `fonts-liberation`, `fonts-noto-color-emoji`; `CHROMIUM_PATH=/usr/bin/chromium`; non-root `civ`; entrypoint `celery -A config worker --queues capture --loglevel info --concurrency 1`.

deploy/docker-compose.yml (project name civ)

Service	Image / build	Networks	Notes
db	postgres:17-alpine, POSTGRES_DB/USER `civ`, password `${DB_PASSWORD}` required	internal only	volume `pgdata`; healthcheck `pg_isready` every 10 s, 5 retries
backend	build `..`	internal + traefik	env_file `.env.production`; env DB_HOST=db, DB_NAME=civ, DB_USER=civ (owner), REDIS_URL=redis://redis:6379/0; volume `media:/app/media`; Traefik router `civ-admin`: `Host(${ADMIN_DOMAIN}) && (Path('/') or PathPrefix('/civ-admin') or PathPrefix('/static'))`, TLS with `${TRAEFIK_CERTRESOLVER}`, port 8000. The client API is NOT exposed on the admin domain.
redis	redis:7-alpine, `--save "" --appendonly no` (nothing on disk)	internal	healthcheck `redis-cli ping`
worker	build `..`, entrypoint `celery -A config worker --loglevel info --concurrency 2`	internal	same env as backend
capture	build `deploy/Dockerfile.capture`	internal	`media` volume (screenshots), `shm_size: 512mb`
beat	build `..`, `celery -A config beat --schedule /tmp/celerybeat-schedule`	internal	depends on redis
portal	build `../../AUDIT/frontend`	internal + traefik	env `BACKEND_URL=http://backend:8000`; router `civ-portal`: `Host(${APP_DOMAIN})`, TLS, port 3000

Variables used: `DB_PASSWORD`, `ADMIN_DOMAIN`, `APP_DOMAIN`, `TRAEFIK_NETWORK` (external network name), `TRAEFIK_ENTRYPOINT`, `TRAEFIK_CERTRESOLVER`. Volumes `pgdata`, `media`.

deploy/update.sh (installed as /opt/civ/deploy.sh)

1. `flock -w 900` on `/opt/civ/.deploy.lock` (one update at a time).

2. `git pull --ff-only` both `/opt/civ/AUDIT` and `/opt/civ/Audit-backend`.
3. `docker compose --env-file .env.production up -d --build --remove-orphans`; `docker image prune -f`.
4. Health loop up to 30 tries × 4 s (~2 min): backend `http://127.0.0.1:8000/api/health` and portal `http://127.0.0.1:3000/sign-in`. Success → "DEPLOY OK"; else prints last 40 log lines, "DEPLOY FAILED", exit 1.

.github/workflows/deploy.yml ("Test and deploy")

- Trigger: push to `main`, or manual. Concurrency group `deploy`, no cancel.
- **test job:** Postgres 17 service; creates role `civ` as `NOSUPERUSER NOBYPASSRLS CREATEDB` (so row-level-security tests are real); throwaway `CIV_SECRETS_KEYS` from `openssl rand 32`; `uv sync --frozen`; `manage.py check`; `pytest -q`.
- **deploy job** (needs test): writes `DEPLOY_SSH_KEY`, `DEPLOY_KNOWN_HOSTS`; `ssh root@$DEPLOY_HOST` — server-side key is restricted to `command="/opt/civ/deploy.sh",restrict` (README). `[skip ci]` in a commit skips deploy (README).

.env.example (local)

`DJANGO_SECRET_KEY`, `DJANGO_DEBUG=true`, `DJANGO_ALLOWED_HOSTS`, `DJANGO_CSRF_TRUSTED_ORIGINS`, `DB_NAME=civ`, `DB_USER/PASSWORD/HOST/PORT`, `PORTAL_URL`, `CIV_SECRETS_KEYS` (with the Fernet generate command), `REDIS_URL` (empty locally).

Tenancy lock (apps/tenancy) — Module 1 step 5

- **Purpose:** the database itself keeps clients apart. Shared tables + PostgreSQL row-level security (RLS) (decision Module 1 = A).
- **Data:** session-level Postgres settings `civ.scope`, `civ.client_id`, `civ.buyer_id`.

Scopes (apps/tenancy/scope.py)

- `_scope(scope, client_id)`: reads current `civ.scope`/`civ.client_id`, sets new ones with `set_config(..., false)` (session-level, not transaction-local), and in `finally` **restores the previous values** so a scope never leaks into the next request/job.
- `tenant(client)` → scope `'client'`, client_id = pk. One client's rows only (jobs, portal requests).
- `all_clients()` → scope `'all'`. Every client's rows (CiV nightly jobs, admin panel).
- `buyer(org)` → sets `civ.buyer_id`=org pk, scope `'buyer'`, client_id `''`. Client tables show nothing; buyer-locked tables show its own rows (Module 15). Clears buyer_id afterwards.
- No scope (`''`) → locked tables show **nothing**, even `WHERE client_id = X` queries.

Policies (apps/tenancy/sql.py)

- `POLICY` = `current_setting('civ.scope', true) = 'all' OR client_id = NULLIF(current_setting('civ.client_id', true), '')::bigint`.

- `lock_table(table)` → migration `RunSQL`: `ENABLE ROW LEVEL SECURITY`, `FORCE ROW LEVEL SECURITY` (owner obeys too), `CREATE POLICY tenant_only ... USING (POLICY) WITH CHECK (POLICY)` (reads and writes). Reversible.
- `BUYER_POLICY` = POLICY `OR (scope='buyer' AND org_id = civ.buyer_id)`; `lock_buyer_table(table)` → policy `tenant_or_buyer`.
- Special policy in `apps/buyers/migrations/0004_attestation_locks.py`: `attestation_snapshot` policy `shared_with_buyer` — buyer sees a client's rows only while a `buyer_link` exists for (client, org) with `revoked_at IS NULL` and `ends_on` null or ≥ today.

Tables locked (found in migrations)

- Module 1 (`tenancy/0001_lock_client_data`): `client_key`, `fingerprint`, `purge_log`, `merkle_anchor`.
- Others (other modules): `contact_event`, `phone_epoch`, `crm_phone_holder`, `certificate_claim`, `consent_proof`, `consent_need`, `consent_result`, `consent_result_change`, `consent_recheck`, `consent_page`, `page_capture`, `opt_out_event`, `optout_readded`, `optout_mark_check`, `sync_cursor`, `sync_gap`, `provenance`, `proof_request`, `uploaded_file`, `internal_list_entry`, `dnc_check`, `rnd_query`, `complaint`, `training_record`, `reconciliation`, `period_integrity`, `snapshot_run`, `metric_snapshot`, `finding`, `alert`, `client_report`, `vendor_month`, `vendor_contract`, `vendor_lead`, `refetch_check`, `legal_hold`, `conversation_review`, `doc_reading`, `lead_source_check`, `rule_update_review`, `reg_change_read`, `client_suggestion`, `buyer_invite`, `insured_statement`; buyer-policy: `buyer_link`, `rating_send`; special: `attestation_snapshot`.
- **Not locked on purpose** (CiV business tables read across clients by admin; listed in `tests/tenancy/test_lock.py:BUSINESS_TABLES`): `access_log`, `client_document`, `client_invite`, `client_status_change`, `client_user`, `connection`, `contract`, `onboarding_tick`.
- Guard test `test_every_table_with_client_data_is_locked`: every public table with a `client_id` column must have RLS + FORCE unless in BUSINESS_TABLES.

Middleware flow (apps/tenancy/middleware.py: TenantScopeMiddleware)

1. Path starts `/civ-admin/` and a staff user is authenticated → whole request inside `all_clients()`.
2. Path starts `/api/` and `accounts.session.current_user(request)` returns a client user → inside `tenant(user.client_id)`.
3. Path starts `/api/` and `buyers.session.current_buyer(request)` returns a buyer → inside `buyer(person.org_id)`.
4. Anything else → no scope (locked tables empty). Background jobs set their own scope (`ledger_job`, `Tokenizer`, `seal` do; tested by `test_jobs_set_their_own_scope`).

Limited DB user (management command create_app_role)

- Reads `DB_APP_USER`/`DB_APP_PASSWORD`; if missing prints "nothing to do".
- `CREATE` or `ALTER ROLE ... LOGIN PASSWORD ... NOSUPERUSER NOBYPASSRLS NOCREATEDB NOCREATEROLE`; GRANT CONNECT on DB, USAGE on schema public, SELECT/INSERT/UPDATE/DELETE on

all tables, USAGE/SELECT on all sequences; `ALTER DEFAULT PRIVILEGES FOR ROLE <owner>` so future tables get the same. Cannot create/alter tables.

- Run on every backend start as owner (entrypoint).

Client accounts and login (apps/accounts)

- **Purpose:** the portal's people (client users) and invites; the client login API. Kept apart from CiV staff (`staff_user`) and from buyers (`buyer_user`).
- **Data:** email/password from the portal (via Next.js proxy `/api/*` → Django), invite tokens, reset links.

Tables

- `client_user` (`ClientUser`, AbstractBaseUser): `client` FK→client (PROTECT), `email` unique, `name` (150), `role` `admin|member` (default member), `is_counsel` ("is our lawyer", default false), `is_active`, `created_at`, plus `password`, `last_login`. `USERNAME_FIELD='email'`. Ordering client name, name.
- `client_invite` (`Invite`): `client` FK (CASCADE), `email`, `name`, `role`, `is_counsel`, `invited_by` FK→ClientUser (SET_NULL), `token_hash` (SHA-256 hex, unique; token itself only in the email), `sent_at`, `expires_at`, `used_at`.

Roles and flags

- **Admin**: may invite (POST /api/users), change role/lawyer flag (PATCH), turn off users / cancel invites (DELETE).
- **Member**: sees every page except Settings and Users & Access, which are Admin only (2026-10-23; the server refuses `/api/access-log`, `/api/settings`, `/api/buyer-links`; `GET /api/users` gives non-Admins names only, to assign a finding).
- **is_counsel** (lawyer flag): only lawyers review rule drafts and suggest law changes; lawyers and Admins sign statements, decide findings, upload and export evidence. There is **no lawyer-only mode** (removed 2026-10-22, migration `clients/0007`): every user sees alerts and findings.
- **view** (`client_user.view`, owner/underwriter): each client user's own "View as" switch (`POST /api/session/view-as`); Underwriter shows only their own company with real data, as their insurer sees it. Insurers are always Underwriter and cannot switch.
- Rule: a company must keep **at least one active Admin** (`LAST_ADMIN = 'A company needs at least one active Admin.'`). A user cannot turn off themselves.
- (Underwriter is a per-person view switch, not a role.)

Sessions (apps/accounts/session.py)

- Own session keys `civ_client_user_id`, `civ_client_auth_hash` (so a staff login in the same browser never counts as a client).
- `sign_in`: `cycle_key()` (anti session-fixation), store pk + `get_session_auth_hash()`, update `last_login`.
- `sign_out`: pop keys, `cycle_key()`.

- `current_user`: user must be `is_active` and `client.is_active`; session hash compared with `constant_time_compare` → a password change ends old sessions.
- Cookie life 12 h (settings).
- `ClientSessionAuthentication` (DRF): uses `current_user`; enforces CSRF on every authenticated request; `authenticate_header='Session'` so unauthenticated gets 401 not 403.

Sign-in flow (POST /api/session/sign-in, views/sign_in.py)

1. Validate email (trimmed, lowercased) + password (max 256, not trimmed). CSRF protected.
2. If `is_locked(email)` → log `sign_in_locked`, **429** "Too many tries. Wait 15 minutes, then try again."
3. Look up ClientUser by email. If none, try buyer sign-in (Module 15 `BuyerUser` + active org); success → buyer session, log `sign_in` (actor_kind anonymous, label "email (org)").
4. If still no user, run `make_password(password)` to equalise timing (no email enumeration).
5. OK requires password match + `user.is_active` + `client.is_active`. Fail → log `sign_in_failed` (object=email, client_id if known) → 400 "That email and password do not match an account."
6. Success → `sign_in`, log `sign_in`, return `session_data`.

Lockout (apps/accounts/lockout.py)

- `MAX_FAILURES = 5`, `WINDOW = 15 minutes`. Locked when ≥5 `sign_in_failed` access-log rows for that email in the last 15 min. Effectively a rolling 15-minute lock. Counter is the access log itself (no separate table). Applies per email (not per IP).

Session data returned (serializers/session.py: session_data)

`signedIn`, `userId`, `name`, `initials` (first letters of first two words), `email`, `role`, `isCounsel`, `view`, `orgKind` (`client`), `canSwitchView`, `clientId`, `clientName`, `vertical` (= client.industry), `plan` (display name), `updatedAt` (max `SyncCursor.last_event_at` for the client — "last time CiV read something new"; omitted if none), `periods` (from `apps.scoring.snapshots.periods`). Not signed in → `{signedIn:false}`. For a buyer: `buyers.session.session_data` (view `underwriter`, `sampleData:false`).

Passwords (views/password_reset.py, change_password.py, tokens.py, emails.py)

There is **no "Forgot password"** (owner, 2026-10-22/23): the sign-in page has no link and `POST /api/session/reset` no longer exists. New passwords come from:

- a **password link CiV staff send** (client page → Users → *Send password link*; Insurers page → a person → *Send password link*, link starts `b-` for buyers; Staff → *Send password link* / welcome link), and
- **My account → Change password** (`POST /api/session/password {current, password}`, every client user and insurer): wrong current passwords count toward the sign-in lock; the new one must differ from the current one; other devices are signed out, this one stays in; logged `password_changed`.

1. Link: `PORTAL_URL/sign-in?reset=<uid>.<token>` (buyers `reset=b-<uid>.<token>`).
2. Token = `ClientResetTokenGenerator` (Django PasswordResetTokenGenerator with own salt `civ.accounts.ClientResetTokenGenerator`, so staff tokens can't be reused); valid `PASSWORD_RESET_TIMEOUT` = 1 hour; becomes invalid after password change (Django behaviour).

3. POST `/api/session/reset/confirm` {link:"uid.token", password}: validate token, password validators (12+ etc.), `set_password` (ends old sessions), log `password_reset_done` → `{done:true}`; bad → 400 "This link has expired or was already used. Ask for a new one."

Invites (views/invites.py, views/users.py)

1. Admin POST `/api/users` {email, name?, role (default member), isCounsel (default false)}. Refused if the email already has a ClientUser or BuyerUser ("That email already has an account.").
2. A **waiting** invite for the same email is never replaced silently: the request is refused with "... already has an invite (sent <date>). Use Resend, or Cancel it and invite again." An **expired** one is replaced. New `Invite` with `token_hash`, `expires_at = now + 7 days` (shared helper `apps/accounts/invites.py`, also used by the staff panel).
3. Email `PORTAL_URL/sign-in?invite=<token>`: "invited ... as an Admin / a Member [and the company's lawyer]", one use, 7 days. Log `invite_sent`. Returns invite row (201).
4. GET `/api/invites/check?token=` → `{email, name, clientName}` if invite unused, unexpired, client active; else tries a buyer invite (Module 15); else 400 "This invite has expired or was already used. Ask your Admin for a new one."
5. POST `/api/invites/accept` {token, password}: creates ClientUser with invite's role and lawyer flag, marks `used_at`, signs in, logs `invite_accepted`. If the token is not a client invite → `accept_buyer` (creates BuyerUser, signs in as buyer).

Users page API (views/users.py)

- GET `/api/users`: for the Admin, active users (`id 'u-<pk>'`, name, email, role, isCounsel, status `active`, lastSeen) + every unused invite (`id 'inv-<pk>'`, status `invited` or `expired`, `sentAt`); for others, names only.
- POST `/api/users/inv-<pk>/resend` (Admin): a new 7-day link; the old one stops working.
- PATCH `/api/users/u-<pk>` (Admin): {role?, isCounsel?}; blocks demoting the last Admin; log `user_changed` "email role=... lawyer=...".
- DELETE `/api/users/inv-<pk>` (Admin): cancels an unused invite, log `invite_cancelled`. DELETE `/api/users/u-<pk>`: sets `is_active=False` (kept for the log), not self, not last Admin; log `user_turned_off`. 204.
- POST `/api/session/sign-out` (signed in): logs `sign_out`; for buyers just clears buyer keys.
- GET `/api/session` (anyone): `ensure_csrf_cookie` — hands the browser its CSRF cookie.

Permissions (apps/accounts/permissions.py)

- `IsClientUser`: `request.user` is a `ClientUser`. Used by nearly every client API view.
- `IsClientAdmin`: IsClientUser and `role == admin` (message "Only an Admin can do this.").
- Buyers (`BuyerUser`) never pass `IsClientUser`, so they reach no client endpoint.

Admin panel parts

- `ClientUserAdmin`: hidden from menu (`has_module_permission=False`); a person's own page opened from the company's Users tab. Fields client, email, name, role, is_counsel, is_active, last_login, created_at. New users get an unusable password. Action "Send a link to set their password" (logs `password_link_sent`).
- `InviteAdmin`: hidden from menu; read-only; no add; token_hash excluded.
- `ClientUserInline` (Users tab on client page): edit people (no adding here; people are invited); `can_delete=False` (use Is active); per-row button "Send password link" (POST to `/civ-admin/clients/client/<client_id>/users/<user_id>/password-link/`, perm `accounts.change_clientuser`); logs `user_changed`.
- `NewInviteInline` ("Invite someone" tab): name, email, role, "is our lawyer" → invitation link (`invite_by_staff`, logged `invite_sent`).
- `InviteInline` (Invites tab): every invite with Sent / Joined / Expired and **Resend invitation** (`/civ-admin/clients/client/<id>/invites/<invite_id>/resend/`); one row per invite, no repeated heading.
- **New client:** the Add client company form requires "Client Admin's name" and "Client Admin's email"; saving sends that person an Admin invitation.

Frontend pages fed

`frontend/src/app/sign-in/page.tsx` (sign-in, `?reset=`, `?invite=`), `(portal)/users` (Users & Access, Admin only), `(portal)/account` (My account, everyone). Next.js `src/server/backend.ts` forwards `session`, `users`, `invites`, `access-log` to Django when `BACKEND_URL` is set; `mergeSession.ts` joins Django's session with mock-only fields (sample months); Django decides each person's view (`view`, `canSwitchView`).

CiV staff accounts (apps/staff)

- **Purpose:** CiV's own team; `AUTH_USER_MODEL`. Opens the admin panel only, never the client API.
- **Table:** `staff_user` (`StaffUser`, AbstractBaseUser + PermissionsMixin): `email` unique (login), `name`, `is_active`, `created_at`, `is_superuser`, groups, user_permissions, password, last_login. `is_staff` is a property = `is_active` (every active staff account may open the panel; what it can do comes from permissions).
- **Manager:** `create_user` / `create_superuser` (email lowercased, required). The first one via `manage.py createsuperuser`.
- **Adding staff (2026-10-22/23):** Staff → Add: name and email only; the person is made a full-access superuser (per-person permissions come later) and gets a **welcome link** (`staff_welcome` table, 7 days, one use, hash kept) to `/civ-admin/welcome/?token=…` where they choose their own password. "Send password link" (button on the person's page, and a list action) sends a new welcome link; Django's "set this user's password" page is switched off (403), and no password code is shown.
- **Sign-in lock:** the panel's login (`apps/staff/login.py`, `admin.site.login_form`) locks an email after 5 wrong passwords in 15 minutes, like the portal; wrong tries are logged `staff_sign_in_failed`.

- **Admin:** `StaffUserAdmin` (fieldsets: email/name/password link; Access: is_active, is_superuser, user_permissions; History). There is no Permission groups page (removed 2026-10-22).
- **Permissions actually checked in code:** `clients.change_client` (onboarding toggles), `accounts.change_clientuser` (password link), `connections.change_connection` (test/connect link), `documents.view_clientdocument` (open document), custom `regulatory.approve_ruleupdate` ("Can approve rule updates (changes the Rulebook)"), `regulatory.add_regchange`. Plus standard Django model admin permissions.

Admin panel (apps/admin_panel)

- **Purpose:** look, menu and dashboard of `/civ-admin/` (Django admin + Unfold), styled like the portal.
- **theme.py (`unfold_config`)**: SITE_TITLE "CiV admin", SITE_HEADER "Comply iV"; `SITE_URL` = PORTAL_URL; teal palette (600 `#0e8c7f` = portal --brand) and portal neutrals; custom `civ.css`/`civ.js`; environment tag "Local" (warning) when DEBUG, "Live" (danger) otherwise.
- **Sidebar menu:** Overview → Dashboard · Clients → Client companies, Proof requests · Insurers → Insurers and acquirers · Rules → Rulebook, Rulebook check, Regulatory changes, New law items, Client suggestions · AI → AI answers, AI exam results, Extra exam questions · CiV team → CiV staff · Records → Access log, Client reports, Failed-job cleanups (purge_log), Missed records (sync_gap).
- **base.py `CivModelAdmin`:** base for every admin section; `eyebrow` label + `page_sub` line via templates `admin_panel/page_head_list.html` and `page_head_form.html`.
- **civ.js:** first visit sets light theme and 248 px sidebar (localStorage `adminTheme`, `sidebarWidth`); moves tabs under the page heading where `[data-civ-tabs-after]` exists; whole list row clickable (Ctrl/Cmd opens new tab).
- **userlinks.html:** top bar: environment label, actions, light/dark toggle button.
- **dashboard.py (`DASHBOARD_CALLBACK`)** tiles:

1. Active clients (+ onboarding · trial · paused counts).
2. Client users (active) → client list sorted by users.
3. Open invites (unused, unexpired) → clients with waiting invites (`?attention=invites`).
4. Connections working (+ waiting, failed or stale) → clients with failed or stale connections (`?attention=connections`).
5. Renewals · 30 days (`renewals_within(30)`) → clients renewing within 30 days (`?attention=renewals`).
6. Rulebook: "Rules needing a decision" = pending count from `apps.rulebook.health.check()` ("Rules that change results and are not decided yet · N rules in all"), or "No Rulebook loaded".
7. New law items · high priority (state new, level High; foot: all new) → filtered law items.
8. Sign-ins · 24 h (+ failed, + "locked now" = emails with ≥5 failures in 15 min). Lists: 8 newest active clients (name, plan, users, start, access level); last 10 access-log rows (Who in plain words: "CiV (automatic)" for jobs, "Not signed in" before sign-in).

Client companies (apps/clients)

- **Purpose:** the client company record; stage (lifecycle); company admin page that hosts all tabs.
- **Tables:**
  - `client` (`Client`): `name` (unique), `industry`, `brand_names` (one per line; Module 7 seller check / docai), `plan` `starter|growth|enterprise` (default growth), `start_date`, `access_level` 0–4 (not editable; set by connections), `status` ("stage") `trial|onboarding|active|paused|ended` (default onboarding), `status_changed_at`, `is_active` (follows status; Ended = off), `created_at`.
  - `client_status_change` (`StatusChange`): client, old_status, new_status, reason (300), changed_at, changed_by FK→staff_user.
- **Logic:**
  - `Client.can_sync` = status in Trial/Onboarding/Active → Paused and Ended stop live sync (checked in `apps/sync/engine.py` and `sync/views.py`).
  - `status.prepare()`: `is_active = status != ended`; stamps `status_changed_at` when stage changes. `record_change()` writes `StatusChange` + access log `client_status_changed` ("Old → New (reason)").
  - Ended → `is_active=False` → sign-in, invites, resets, sessions refused (all check `client.is_active`).
  - Access level ladder (`apps/connections/access_level.py`): each level needs the one below: 1 = dialer or SMS connected; 2 = + certificates; 3 = + lead feed; 4 = + CRM. CONNECTED or STALE both count. Refreshed by a connections signal on save and by `sync/health.py`.
  - Stage pill colours: trial blue, onboarding amber, active green, paused gray, ended red.
- **Admin (`ClientAdmin`)**: list columns Name, Stage, Industry, Plan, Onboarding ("x of 7"), Data access ("Level n of 4"), Users, Connections ("n of m connected"), Renews (amber inside notice period). Filters Needs attention (waiting invites, failed connections, renewing within 30 days), status, plan, access_level. Search name, industry, users' email/name. Form fields (a new client also asks for its Admin's name and email): name, industry, plan, start_date, status, "Reason for stage change", status_since, counsel_directed, access level, created_at. Tabs (inlines): Connections, Contract & billing, Documents, Users, Invites, Consent pages (read-only, GET only). New company: only Connections, Contract, Documents tabs.
  - Summary tiles: Data access, Connections, Users, Open invites, Certificates (claimed · waiting · no key · lost), Records completeness (link), Provenance tile, AI tile. Onboarding card, stage history (last 20), Activity (last 10 access-log entries) + "Full access log" link.
  - Extra admin URLs under `/civ-admin/clients/client/<client_id>/`: `connections/<id>/test/`, `connections/<id>/send-link/`, `documents/<id>/open/`, `onboarding/<step>/`, `consent-pages/<capture_id>/photo/`, `completeness/`, `users/<user_id>/password-link/`.
- **API endpoints (apps/clients/api.py, all `IsClientUser`):**
  - GET `/api/settings`: `company` {name, vertical, states (distinct `ContactEvent.recipient_state`), engaged (start_date)}, `agreements` (Service agreement, Annex C, DPA, Signed statement: signed/pending + date from latest `ClientDocument` of that kind with `signed_on`), `usage` (from onboarding/readiness).

  - GET `/api/revocation`: `optouts.measures.revocation_page(client)` + `authorisation.signedOn` = latest signed Annex C date or "—".
  - GET `/api/billed-numbers`: CSV `billed-numbers.csv`, columns "number (masked)", "first contact", "contacts this month" (masked labels only, never the number).
- **Frontend pages fed:** `(portal)/settings` (merge path `settings`), `(portal)/revocation` (merge path), billed-numbers download.

Onboarding (apps/onboarding)

- **Purpose:** 7-step onboarding checklist on the client's admin page; portal Setup & Readiness and Plan & usage.
- **Table:** `onboarding_tick`: client, step (40), done_at, done_by FK→staff; unique (client, step). Only manual ticks are stored; auto steps computed live.
- **Steps (steps.py, in order):**

1. `agreement` Agreement signed — manual; also done when a `ClientDocument` kind `agreement` is uploaded.
2. `annex_c` Annex C signed — manual; also done by `annex_c` document.
3. `campaigns` Campaigns marked — manual.
4. `dialer` Dialer connected — auto (a DIALER connection status CONNECTED).
5. `texting` Texting tool connected — auto (SMS connection CONNECTED).
6. `client_admin` Client Admin added — auto (an active Admin ClientUser).
7. `first_sync` First sync done — manual ("Ticked by hand until live sync exists").

- `progress.checklist(client)` → steps with done/by_document/done_at/done_by, `done`, `total`=7, `percent`, `complete`.
- Toggle view (`views.toggle_step`, POST, perm `clients.change_client`, manual codes only): creates or deletes the tick; logs `onboarding_step_done` / `onboarding_step_undone`.
- **API:** GET `/api/readiness` (IsClientUser) → `readiness(client)`:
  - Window: last 30 days of outbound, primary `ContactEvent`s. `numbersPerMonth` = distinct `phone_token`s. `recommendedPlan` by `PLANS` = Starter ≤10,000, Growth ≤50,000, Scale ≤250,000, else Enterprise.
  - Checks (name, how, state ready/partial/missing, detail, fix): Dialer / SMS / CRM connection (ready when CONNECTED or STALE and sync state live or catching_up; dialer goes partial if any calls lack `dial_mode`, "counted as autodialed"); Consent proof (sample of up to **100** recent numbers, non-self-asserted, not withdrawn proofs); Certificate retention (lost claims, or claimed certs expiring within 365 days); Consent pages (photographed OK vs known); Lead records (untraced provenance in 30 days); Internal do-not-contact list (uploaded?); History depth (ready if earliest record ≥ **4 years** back — time to sue); Documents (DNC check records and reassigned-number query logs uploaded).

  - `canStart` = dialer or SMS check not missing. `preview` = strict consent results on the sample (VERIFIED/WEAK/CONFLICTING/NO_PROOF). `setup` rows: caller IDs, pages, vendors, campaigns ("all treated as marketing", owner decision), states.
  - `usage(client)` (used by /api/settings): this month + 3 before, distinct numbers per month, plan limit, `overLimitMonths` (consecutive from latest), `nextPlan`, `monthlyPrice` = contract price ÷ period months (monthly 1, quarterly 3, yearly 12).
  - `billed_numbers(client)`: this month's distinct tokens with masked label, first contact date, count.
- **Frontend pages fed:** `(portal)/setup` (merge path `readiness`), Settings → Plan & usage.

Access log (apps/access_log)

- **Purpose:** who did what, when — append-only audit trail for every module.
- **Table:** `access_log` (`AccessEntry`): `at` (auto, indexed), `actor_kind` `client_user|staff|system|buyer|anonymous`, `actor_id`, `actor_label`, `client_id` (bigint, indexed, no FK), `action` (indexed, 64), `object` (254), `ip_address`.
- **Immutability:** model `save()` raises if pk exists; `delete()` raises. DB trigger `access_log_no_update_or_delete` (function `access_log_block_changes`) raises on any UPDATE/DELETE, even bulk (migration 0002).
- **recorder.record(action, request=, actor_kind=system, actor_id, actor_label, client_id, object)**: the only writer; IP = `REMOTE_ADDR`.
- **labels.py:** plain words + pill colour per action (≈60 actions: sign_in, sign_in_failed, sign_in_locked, password/invite/user/document/contract/stage/onboarding/connection/reg-change/opt-out/certificate/source/rule/proof/upload/attestation/client_key events ...). Unknown action → capitalised words, gray.
- **API:** GET `/api/access-log` (IsClientUser): the company's last **300** entries, newest first: at, actor (person name; "CiV team" for staff; "CiV" for system; else label or "Not signed in"), role (`civ` for non-client), action label, object. Note: any client user (not only Admin) can read it.
- **Admin:** "Access log" list, read-only for everyone (no add/change/delete); columns When, Who (actor label or the email tried), What (pill), Object, Company, IP; filters actor_kind, action; date hierarchy; `?client_id__exact=` allowed.
- **Also used as:** the lockout counter (accounts), dashboard sign-in tile.
- **Frontend:** Users & Access → Access log (BACKEND_PATHS `access-log`).

Phone codes / keyring (apps/keyring) — Module 1 step 2

- **Purpose:** CiV never saves a phone number. Inside a job a number becomes a `phone_token` = HMAC-SHA256(E.164 number, client's secret key). Same number + same client → same code; two clients → different codes.
- **Table:** `client_key` (RLS-locked): client FK (PROTECT), `kid` (e.g. `k1`; saved next to every code as `phone_token_kid`), `locked_key` (Fernet-encrypted JSON `{"k": base64 32 random bytes}`, empty once

destroyed), created_at, destroyed_at. Constraints: unique (client, kid); at most one active key per client (`client_key_one_active`, where destroyed_at is null).

- **phones.py `normalize(raw, region='US')`**: `phonenumbers.parse`; accepts "possible" numbers (right length), not only "valid" ones, so unusual ranges are not dropped; returns E.164 (`'(916) 555-4471'` → `'+19165554471'`) or None.
- **tokens.py:**
  - `active_key(client)`: inside `tenant(client)`; returns active key; makes `k1` on first need (32 random bytes, locked with `CIV_SECRETS_KEYS`), race-safe via IntegrityError retry; logs `client_key_created`. If a key existed and was destroyed → raises `KeyDestroyed` (never makes a new one).
  - `Tokenizer(client)`: opens the key once (in memory only); `.token(raw)` → normalized → hex HMAC; None if not a phone number.
  - `destroy_key(client, actor…)`: contract end; wipes `locked_key`, sets `destroyed_at`, logs `client_key_destroyed`. All saved codes become unlinkable forever.
- **Used by:** `sync/engine.py`, `contacts/normalize.py`, `contacts/api.py`, `anchoring/position.py`.

Fingerprint record / ledger (apps/ledger) — Module 1 step 3

- **Purpose:** prove what CiV read without keeping it: one row per record read with SHA-256 of the exact received bytes.
- **Tables (both RLS-locked):**
  - `fingerprint`: client FK, `job_id` (UUID, indexed), `source_system` (e.g. twilio, convoso), `source_record_id`, `content_kind` (call, text, lead, certificate, crm_record, page_capture, billing, upload_row), `source_grade` A–D (A captured by CiV itself; B independent third party; C client's systems by API; D exported/uploaded by client; S and N never fingerprinted), `sha256`, `fetched_at` (indexed), `anchor` FK→merkle_anchor, `leaf_index`, `merkle_path` (JSON). Index (client, source_system, source_record_id).
  - `purge_log`: client, job_id, job_name, failed_at, rows_removed, reason (error **type** only — message could quote a record).
- **Flow (`record.ledger_job(client, name)`):**

1. New `LedgerJob` with UUID; enter `tenant(client)`.
2. `job.record(source_system, source_record_id, raw: bytes, content_kind, source_grade)` — raw must be `bytes` (TypeError otherwise, "not a re-built copy"); creates Fingerprint with SHA-256 and `fetched_at=now`; counts `written`.
3. On any exception: delete all fingerprints with that job_id, write one `PurgeLog` row (rows_removed=written, reason=exception class name), re-raise.

- **Admin:** "Failed-job cleanups" (PurgeLog) read-only list: failed_at, company, job_name, rows_removed, reason.
- **Frontend:** Evidence Vault counts (via anchoring API), hash check.

Daily seal / anchoring (apps/anchoring) — Module 1 step 4

- **Purpose:** every night, seal each client's fingerprints per finished UTC day into one Merkle root, stamped by outside services, so any later change is detectable.
- **Table:** `merkle_anchor` (RLS-locked): client, `day` (UTC date), `leaf_count`, `root` (hex), created_at, `tsa_url`, `tsa_response` (binary, kept whole), `tsa_time`, `ots_calendar`, `ots_proof` (binary, pending until a Bitcoin block confirms). Index (client, day).
- **Merkle (merkle.py):** leaf = SHA-256(0x00 || "client_id|source_system|source_record_id|content_kind|source_grade|sha256|fetched_at(UTC ISO, microseconds)") — seals the details, not just the hash. Node = SHA-256(0x01 || left || right). Odd node moves up unchanged. Each leaf's path = list of [side 'L'/'R', sibling hex]. `fold()` walks a leaf to the root.
- **Flow:**

1. `seal_nightly` (00:30 UTC) → `seal_finished_days()`: under `all_clients()`, finds (client, day) pairs with unsealed fingerprints for days **before today**; today's are sealed tomorrow.
2. `seal_day(client, day)` under `tenant(client)`: fingerprints ordered by id → build root → in one transaction create `MerkleAnchor` and bulk-update each fingerprint's anchor/leaf_index/merkle_path (batch 2000).
3. `stamp(anchor)`: RFC 3161 — for each `ANCHOR_TSA_URLS` in order, build SHA-256 request with nonce, POST (`application/timestamp-query`, 20 s timeout), accept if status 0/1 and message imprint = SHA-256(root); save url, response, gen_time. OpenTimestamps — POST root bytes to `<calendar>/digest`; save calendar + pending proof. A service down doesn't stop the seal.
4. `stamp_missing` (hourly :15): retries anchors missing TSA or OTS.

- **verify(fp)** → `matches` / `changed` (row incl. read time altered after sealing) / `not_sealed`.
- **API (all IsClientUser):**
  - GET `/api/vault`: `artifacts` (fingerprint count, formatted), `anchors` (count), `lastAnchor` (day or "—"), `engagedSince`, `coverage` (last 12 months: `before` start, `covered` past month with records, `partial` otherwise).
  - GET `/api/position?phone=&date=YYYY-MM-DD`: (anchoring/position.py) number normalized + tokenized (never stored); returns masked phone, date, `insideCoverage` (date ≥ start_date), rows as of end of that day: Phone epoch, Consent proof, Consent page as of date (capture, Tier 1), Lead source, Contact log (tier from grade A1…D4), Opt-out history, Internal list, Consent status (strict result + Rulebook version). 400 "Enter a date." / "Enter a full phone number."
  - GET `/api/hash-check?hash=`: found?, artifact "system · record id", capturedAt, anchoredOn (seal day), tier.
- **Frontend:** `(portal)/vault` (merge path `vault`; `position`, `hash-check` in BACKEND_PATHS).

Background workers (apps/jobs) — Module 1 step 1

- **Purpose:** Celery + Redis infrastructure; each app keeps its jobs in `tasks.py`; timetable in settings.

- **Table:** `job_heartbeat`: `name` unique, `seen_at`.
- **Task:** `apps.jobs.tasks.heartbeat` every 60 s: `update_or_create(name='workers', seen_at=now)` — proves clock (beat) and workers alive.
- **Local:** no `REDIS_URL` → eager (in-process). Real local workers: `celery -A config worker -l info` + `celery -A config beat -l info`.
- **Server:** `redis`, `worker` (concurrency 2), `capture` (concurrency 1, queue capture), `beat` containers.

Contract & billing (apps/billing)

- **Purpose:** what a client pays and when the contract renews; one per client.
- **Table:** `contract`: client OneToOne (CASCADE), `price` (decimal 10,2, nullable), `currency` (default USD), `billing_period` monthly|quarterly|yearly, `payment_method` invoice|card|ach|other, `contract_start`, `renewal_date` ("contract end / renewal date"), `auto_renew` (default True), `notice_days` (default 30), billing contact name/email/phone, notes, updated_at.
- **Logic:** `price_text` e.g. "$1,500 / month"; `days_to_renewal`; `renews_soon` = days ≤ notice_days (incl. past due); `renewals_within(days)` = renewal between today and today+days for active clients (dashboard uses 30).
- **Admin:** "Contract & billing" tab (StackedInline, max 1, no delete) with Price, Dates, Billing contact, Notes; `save_contract` logs `contract_added`/`contract_changed` with changed field **names** only. Client header shows price and "renews/ends <date>" + "within notice period" pill.
- **Used by:** `/api/settings` usage `monthlyPrice`.

Client documents (apps/documents)

- **Purpose:** signed paperwork between CiV and a client (never the client's customer records).
- **Table:** `client_document`: client FK, `kind` agreement | annex_c (test opt-out permission) | statement (Insured states) | dpa | other, `title`, `file` (stored at `client-docs/<client_id>/<uuid hex><ext>`), `link` (URL alternative), `signed_on`, `original_name`, `sha256`, `size`, `uploaded_by` FK→staff, `uploaded_at`.
- **Rules:** allowed extensions pdf, png, jpg, jpeg, docx; max **20 MB**; a file or a link required ("Add a file or a link.").
- **Flow (admin Documents tab, `save_documents`):** deleted docs → file removed from disk + log `document_removed`; new/changed file → SHA-256 (`fingerprint.sha256_of`), size, original name, uploaded_by; log `document_added`/`document_changed`. Uploading agreement/annex_c ticks the onboarding step.
- **Open (`views.open_document`):** staff only, perm `documents.view_clientdocument`; logs `document_opened`; `FileResponse` inline for PDF/PNG/JPEG else attachment; headers `X-Content-Type-Options: nosniff`, `Cache-Control: private, no-store`. Never a public URL (MEDIA_ROOT not served); the file box shows no "currently: <link>" (it pointed at a public address that does not exist), only the row's Open button. Sizes read "193 bytes", "1.3 KB", "3.4 MB".
- **Used by:** `/api/settings` agreements, `/api/revocation` Annex C date, onboarding.

Frontend bridge (cross-cutting, frontend/src)

- `src/app/api/[...path]/route.ts` serves every `/api/*` for the portal. With `BACKEND_URL` set:
  - `search`: Django numbers/vendors + mock pages/domains/metrics (max 10).
  - MERGE_PATHS (GET only): `vault, settings, revocation, insureds, metrics, who-contact, readiness, leads, vendors, litigation, score, domains, actions, reports` → Django fields laid over sample (`overlay`: lists merged by `code`/`id`/`name`). A response with header `X-CiV-Viewer: buyer` passes through unchanged (only litigation overlays sample).
  - BACKEND_PATHS (fully Django): `session, users, invites, rulebook, reg-changes, rule-updates, sources, data-health, contacts, evidence, can-call, optouts, consent, certificates, consent-pages, provenance, proof-requests, access-log, uploads, conduct, billed-numbers, position, hash-check, alerts, vendor-contracts, vendor-disputes, evidence-file, evidence-bundle, legal-holds, buyer-links, statements, rating-engine`. WRITE_PATHS: `who-contact, actions, reports`. `view-as` stays on mock.
  - Everything else → mock sample data.
- `forward()` passes headers cookie, content-type, x-csrftoken, origin, user-agent; body as bytes; returns cookies.
- In Docker the browser never talks to Django directly; portal uses `http://backend:8000`.

3. Rulebook and the law watcher (Modules 2 and 13)

Read from the code on 22 October 2026: `/home/shakib/Audit/backend` (apps `rulebook`, `regulatory`, file `rulebook/v3.0.yaml`, `config/urls.py`, `config/settings.py`, `deploy/entrypoint.sh`, `README.md`) and `/home/shakib/Audit/frontend/src` (`screens/rulebook`, `screens/regulatory`, `api/queries.ts`).

Both apps are mounted under `/api/` (`config/urls.py`: `path('api/', include('apps.rulebook.urls'))`, `path('api/', include('apps.regulatory.urls'))`).

1. The Rulebook file — backend/rulebook/v3.0.yaml

- **Purpose:** the single list of every legal setting and parameter the engine uses (calling hours, opt-out deadline, consent types, score bands, Lead Inspector thresholds, ...). It is code: changed by editing the file and pushing. Header comment: "Keys never say 'scrub' or 'screen'."
- **Top-level fields:** `version: '3.0'`, `status: provisional`, `effective_from: '2026-10-07'`, `source: docs/specs/02-rulebook-parameters.md v3.0 (2026-10-07)`, `rules: [...]`.
- **Rule fields** (all keys found in the file): `key`, `section`, `meaning`, `owner` (Counsel / Product / Engineering), `status` (set / proposed / tbd), `counsel_item` (code in the counsel brief, e.g. A1, B9, or null), `affects_status` (bool: can change a consent / checkpoint / lead result), `counsel_approved` (bool), `shown` (plain words for the portal), `value` (the value in force; absent for TBD), `provisional` (TBD only: test value used in draft runs), `variants` (`{strict, lenient}` for undecided keys), `jurisdictions` (list of state / forum override rows, each with `jurisdiction` and optional `status`, `value`, `provisional`, `variants`). Optional per-rule `effective_from`, `changed_by`, `approved_by`, `source_url` are read by the loader but not used in v3.0.
- **Status meaning:** `set` = decided; `proposed` = runs, results marked provisional; `tbd` = never runs in a scored run (check returns not_run with reason `RULE_PENDING`).

Counts (counted from the file by script, 2026-10-22)

- **149 rules** (federal rows). By status: **70 set, 59 proposed, 20 tbd**. By owner: Product 80, Counsel 47, Engineering 22.
- NOTE: the backend README still says "146 rules ... 70 set, 55 proposed, 21 TBD" — out of date (rules were added later, e.g. `vendor_required_clauses`, `exposure_multipliers`, `metric_score_bands`, worst-case values).
- **By section:** Consent 29 · Lead Inspector 24 · General and time 21 · Scoring and gates 16 · Stopping 11 · Data integrity 11 · How you contact 10 · Records 10 · Who you contact 7 · AI conversation review 7 · Provenance and manual imports 3.
- **Place (jurisdiction) rows: 9** — `calling_hours` (FL, OK, MD, CT, TX), `frequency_limits` (FL, OK, MD), `written_consent_required_by_forum` (`5th_circuit`, TBD, provisional true).
- **Keys with strict/lenient variants:** `autodialed_modes`, `sms_treated_as_autodialed`, `seller_absent_effect`, `optout_grace_business_days`, `quiet_hours_apply_to_sms`.
- **counsel_approved: true** on 31 keys (e.g. `clock_skew_seconds`, `seller_match_threshold`, all `li_*` Lead Inspector values, all `worst_case_value.*`).

Important rules (exact values in the file)

key	section	status	counsel item	value / shown
optout_deadline_business_days	Stopping	proposed	A1	10 (business days)
confirmation_window_minutes	Stopping	proposed	A2	5 (one confirmation text, no marketing)
optout_keywords	Stopping	proposed	A3	STOP, QUIT, END, REVOKE, OPT OUT, OPTOUT, CANCEL, UNSUBSCRIBE
optout_scope_rules, optout_grace_business_days	Stopping	tbd	—	not set
optout_observation_days	Stopping	set	—	30
business_day_calendar	General and time	proposed	—	weekdays mon–fri, except us_federal_holidays, time_zone recipient
business_day_tz_fallback	General and time	proposed	—	[recipient_zip, area_code, America/New_York]
calling_hours	How you contact	tbd	A6	no value; provisional federal 08:00–21:00; FL/OK/MD 08:00–20:00; CT 09:00–20:00; TX 08:00–21:00 with Sunday from 12:00
quiet_hours_apply_to_sms	How you contact	tbd	B4	variants strict true / lenient false
frequency_limits	How you contact	tbd	A7	provisional FL/OK/MD: max 3 per 24 h per person, scope subject, solicitations only
state_holiday_restrictions, required_disclosures	How you contact	tbd	—	not set
abandoned_call_max_rate	How you contact	proposed	—	0.03 (2 s connect, 30-day period)
written_consent_types	Consent	proposed	B1	web_cert, signed_form, ivr_keypress_with_disclosures
signed_written_consent_types	Consent	proposed	—	web_cert_with_esignature, signed_form
prior_express_evidence_types, autodialed_modes, state_consent_rule (A18), consent_checklist, seller_absent_effect, inbound_contact_scope, cc_min_font_px, cc_max_sellers_named	Consent	tbd	—	not set
ebr_purchase_months /	Consent	propo	—	18 / 3

key	section	status	counsel item	value / shown
ebr_inquiry_months		sed		
written_consent_required_by_forum	Consent	proposed	B11	true everywhere; 5th_circuit row TBD (Bradford v. Sovereign Pest Control)
cert_unretained_expiry_hours / cert_confirmed_expiry_hours	Consent	set	—	72 / 2160
count_unanswered_calls / count_undelivered_sms	General and time	proposed	B9	true / false
lookback_years	General and time	proposed	—	4
poll_interval_minutes, catchup_interval_minutes, full_sweep_interval_days	General and time	set	—	2 / 60 / 7
urgent_alert_max_minutes	General and time	set	—	15
dnc_check_max_age_days	Who you contact	proposed	—	31
rnd_sample_size	Who you contact	proposed	—	400
record_retention_years	Records	proposed	A16	5
civ_default_retention_years	Records	proposed	—	6
vendor_required_clauses	Records	proposed	A15b	12 clauses, 5 required: consent naming the client, claimable certificate, proof on request within 5 business days kept 5 years, opt-outs passed on, indemnity. Recommended: no resale, delivered within 24 h, DNC checked before sale, sub-vendors named, insurance, audit right, refunds. Shown "12 clauses, 5 required (draft)". Required ones are counted in M07.
grade_bands	Scoring and gates	set	—	A 90 / B 80 / C 70 / D 60
direct_violation_grade	Scoring and gates	proposed	D23	D (shown to clients as direct_contact_cap_grade)
hard_cap_share, hard_cap_grade	Scoring and gates	tbd	—	not set
coverage_cap_floor / coverage_cap_grade	Scoring and gates	proposed	—	0.6 / C

key	section	status	counsel item	value / shown
exposure_multipliers	Scoring and gates	proposed	—	venue states FL OK MD WA TX CT VA GA NJ UT IN MS KY; heavy FL OK WA TX CT; venue bands 0→1.0, 2→1.1, 5→1.2, top 1.3, cap 1.35; volume bands 100k→0.9, 500k→1.0, 1M→1.1, top 1.2 (1.3 multichannel), cap 1.35; targeting cap 1.5
li_some_risk_threshold / li_high_risk_threshold	Lead Inspector	set	—	40 / 70
vs_rate_weights	Lead Inspector	set	—	[0.35, 0.3, 0.15, 0.1, 0.05, 0.05] (must sum to 1)
conv_review_min_confidence / conv_review_accuracy_pass_mark / conv_review_stop_catch_mark	AI conversation review	proposed	—	0.9 / 0.98 / 1.0
calls_per_agent_day, conv_review_partial_requests	—	tbd	—	not set

2. Rulebook app — apps/rulebook (Module 2)

Purpose

Loads the YAML file into the database, answers "what is rule X for this place on this date?" for every check in the system, counts business days / opt-out deadlines, applies approved rule updates with dates, and serves the portal's Rulebook pages and a monthly Rulebook snapshot PDF.

Data it fetches / receives

- Reads `rulebook/*.yaml` from disk (`RULEBOOK_DIR = <backend>/rulebook`) at each container start.
- Receives approved `RuleUpdate`s from `apps.regulatory` (applied via `apply.apply_update`).
- Reads `ContactEvent` (apps.contacts) only for the admin "Rulebook check" client counts.
- No outside network calls.

Tables

- **`rulebook_version`** (`RulebookVersion`): `version` (unique, e.g. "3.0"), `status` (draft "tests only" / provisional "CiV staff only" / approved "shown to clients"), `effective_from`, `source`, `file_sha256` (hash of the loaded file), `revision` (+1 per approved update so cached lookups reload), `loaded_at`. Ordered newest `effective_from` first.
- **`rule_value`** (`RuleValue`, verbose name "Rulebook"): FK `version`, `key`, `jurisdiction` (default "US"; state code or forum like `5th_circuit`), `section`, `meaning`, `owner`, `status`, `counsel_item`, `affects_status`, `counsel_approved`, `value` (JSON), `provisional` (JSON, TBD test value), `variants` (JSON), `shown` (≤300 chars), `effective_from` / `effective_to` (null from = "in force since before the lookback"), `changed_by`,

`approved_by`, `source_url`. Unique (version, key, jurisdiction, effective_from) with `nulls_distinct=False`; index (key, jurisdiction).

- Neither table is row-level-locked (shared rules for all clients).

Flow — loading (step by step)

1. Container start: `deploy/entrypoint.sh` runs `python manage.py load_rulebook`.
2. The command (`management/commands/load_rulebook.py`) loads every `rulebook/*.yaml` in name order and prints "file: vN status, rows, loaded|reloaded|unchanged".
3. `loader.load(path)`: hash the file (SHA-256), parse YAML, run `check(doc)` (refuse bad files, see Logic).
4. Same version + same hash → `unchanged`, nothing done.
5. Version already exists and is `approved` → refused ("add a new version instead").
6. Otherwise, in one transaction: delete the version's old rows, upsert `RulebookVersion`, bulk-create one federal `RuleValue` per rule plus one row per `jurisdictions` entry (a TBD place row gets `shown = "Not set yet"`, others `str(value)`), then `reapply_approved(version)` — every `RuleUpdate` in state approved is applied again, oldest `approved_at` first, so admin-approved changes survive a file reload.
7. On a reload (not first load) the `rules_changed` signal is sent with `from_date=None` (every date).

Flow — lookup

1. A run makes `RuleSet(mode)`; it picks `current_version(mode)` = newest-`effective_from` version whose status is allowed for the mode: `client` → approved only; `internal` (default) → approved or provisional; `draft` → also draft. No version → `NoRulebook`.
2. All rows of the version are read once per process (`lru_cache` keyed by version id + file hash + revision).
3. `get(key, on=date, state=, forum=, variant=)`: keep rows in force on that date; try the state, then the forum, then 'US'; the newest in-force row of the first place that has one wins. A place's own row wins even while TBD (a pending Florida rule is never replaced by the looser federal one). No key → `UnknownRule`.
4. The value returned (`_answer`): in `draft` mode a chosen variant, or the TBD `provisional` test value; in other modes a variant only if requested and the row is not `set`; a TBD row returns `value=None`.
5. `Rule` result: `.pending` (TBD with no value → checks return not_run / RULE_PENDING), `.provisional` (status ≠ set → result marked provisional), `.require()` raises `RulePending` when pending.

Flow — business days / opt-out deadline (business_days.py)

1. `optout_deadline(received_at, time_zone=...)`: take the recipient's local calendar date of the STOP.
2. Read `optout_deadline_business_days` (10) and the `business_day_calendar` in force on that date.
3. Count n business days after that date; the STOP day itself never counts (STOP Friday → Monday = day 1).
4. Business day = weekday in the list and not an observed US federal holiday.

5. Return `Deadline(date, business_days, time_zone, provisional)`; provisional is true while either rule is Proposed.

- Holidays (5 U.S.C. 6103): New Year, MLK (3rd Mon Jan), Washington's Birthday (3rd Mon Feb), Memorial (last Mon May), Juneteenth (from 2021), Independence, Labor (1st Mon Sep), Columbus (2nd Mon Oct), Veterans, Thanksgiving (4th Thu Nov), Christmas. Saturday holiday observed Friday, Sunday observed Monday; next year's New Year may land on Dec 31.
- Time zone order `pick_time_zone(zip_tz, area_code_tz, fallback)`: ZIP, then area code, then America/New_York.
- README example: STOP Fri 20 Nov 2026 → deadline Mon 7 Dec 2026 (Thanksgiving skipped).

Flow — applying an approved update (apply.py)

1. Lock the key's rows. Template row = same place's row, else any row of the key.
2. No row at all: refused unless it is a brand-new rule (`new_meaning` given) → template section "Added by rule update", owner Counsel, affects_status true, counsel_approved false.
3. If the place already has a row starting on or after the new start date → `OverlapRefused` ("pick a later start date").
4. The open-ended / overlapping old row gets `effective_to = new start − 1 day`.
5. Create the new row (copies section/meaning/owner/counsel item/flags from the template; value, shown, status, effective_from, changed_by, approved_by, source_url from the update).
6. `revision += 1` (lookups re-read), then `check_no_overlap(version)` — refuses two values for the same rule, place and day. Invariant: exactly one value per rule, place and day.

Logic and rules (loader refusals, loader.check)

- Unknown version status; a key missing vs. v3.0 (v3.0 is the reference key set for every version); an unknown key ("typo?"); a key listed twice; `vs_rate_weights` not summing to 1; `dedupe_window_seconds` < `clock_skew_seconds`; unknown rule status; a set/proposed rule with no value.
- For an `approved` version only: no TBD keys anywhere (incl. place rows); every `affects_status` key whose owner is not Counsel must have `counsel_approved`. (README adds: before any approved version counsel must approve 5 such keys — cert expiry hours ×2, conv_review ×3; not re-verified by count here.)
- Approved version is immutable once loaded.

Key functions / classes

- `apps/rulebook/loader.py:load` — load one YAML file (loaded / reloaded / unchanged).
- `apps/rulebook/loader.py:check` — refuse invalid files.
- `apps/rulebook/loader.py:reapply_approved` — re-apply admin-approved updates after reload.
- `apps/rulebook/lookup.py:RuleSet` — one version in memory; `.get(key, on, state, forum, variant)`.
- `apps/rulebook/lookup.py:rule` — one-off lookup helper.

- `apps/rulebook/lookup.py:current_version` — version allowed for a mode (client/internal/draft).
- `apps/rulebook/lookup.py:Rule` — result with `.pending`, `.provisional`, `.require()`; constant `RULE_PENDING`.
- `apps/rulebook/business_days.py:BusinessCalendar` — `is_business_day`, `add`, `between`.
- `apps/rulebook/business_days.py:optout_deadline` — opt-out deadline (Deadline dataclass).
- `apps/rulebook/business_days.py:named_holidays` / `us_federal_holidays` / `pick_time_zone` / `local_date`.
- `apps/rulebook/apply.py:apply_update` / `check_no_overlap` / `Update` / `OverlapRefused`.
- `apps/rulebook/health.py:check` — admin Rulebook check data; `history(key)` — every value of a rule.
- `apps/rulebook/snapshot.py:snapshot_pdf` — PDF of rules in force on the last day of a month (reportlab).
- `apps/rulebook/signals.py:rules_changed` — signal (kwarg `from_date`).
- Users of RuleSet across the system (grep): admin_panel/dashboard, captures, certificates, consent (checks, keeping, needs, panel), contacts (calling, dedupe), docai, imports, metrics/conduct, optouts (api, detect, marks, measures), reconcile, reports, scoring (coverage, exposure, portal, scores), sync/health, vendors (contracts, rates).

Signal receivers

- `apps/consent/apps.py` connects `rules_changed` → `apps/consent/keeping.py:rules_changed`, which queues a consent `Recheck` for all clients (client None, reason `rule_changed`, from the start date at 00:00 UTC, or from the beginning when `from_date` is None). Shown to users as "Rulebook change".

API endpoints (all IsClientUser = any signed-in client user, any role)

- `GET /api/rulebook` — rules in force **today** in the current `internal` version (so provisional v3.0 is shown): `{version: "Rulebook v3.0", versionStatus, note (when version not approved: "Some rules are not yet confirmed by a lawyer..."), set, proposed, tbd (federal counts), settings: [{key, place, section, meaning, value (=shown), owner, status, counselItem, since}], impact: []}`. `direct_violation_grade` is renamed `direct_contact_cap_grade`. 404 "No rulebook yet." if none.
- `GET /api/rulebook/calendar?year=YYYY` (2000–2100 else 400) — holidays of the year (observed + actual date), next holiday, `businessDays` (shown fixed "Monday to Friday, except US federal holidays, in the time zone of the person contacted"), `callingHours` and `frequencyLimits` by place (from a `draft` RuleSet, so TBD draft values are shown, formatted e.g. "8am – 9pm; Sunday from 12pm" / "3 per 24 hours per person, solicitations only"), `smsFollowsCallingHours`, `restrictedHolidays`, `optOut {deadlineDays, confirmationMinutes, keywords}`, `timeZone {order: [ZIP, area code, New York time], status}`.
- `GET /api/rulebook/snapshot?period=YYYY-MM` — PDF download `rulebook-snapshot-YYYY-MM.pdf` (bad period → this month); title includes the client's company name. Also made by `apps/reports/make.py` for the Reports page.

Scheduled jobs

- None in this app. Loading happens at container start (entrypoint), not on a timer.

Admin panel parts (Admin → Rules)

- **Rulebook** (`RuleValueAdmin`): every rule row, read only (no add/change/delete). Columns key, place (US → "Federal"), shown, status pill (set green / proposed amber / tbd gray), owner, counsel item, "Changes results", version, History link. Filters status, owner, section, jurisdiction, affects_status, version; search key / meaning / shown / counsel item; 200 per page.
- **Rulebook check** (`/admin/rulebook/rulevalue/check/`, Module 13 step 5, template `rulebook/check.html`): federal counts by status; **Needs a decision** = TBD rules with `affects_status`, each with the number of active clients that made outbound contacts in that place in the last 30 days (US = every client with a contact), most affected first; **Not confirmed by a lawyer** = Proposed, `affects_status`, not `counsel_approved`.
- **History** (`/civ-admin/rulebook/rulevalue/rule-history/<key>/`, own name since 2026-10-23 so it no longer clashes with Django's per-row history; the History button on a rule's page opens it): each value by place with from / until, status, Rulebook version, changed by, approved by, source.
- Admin dashboard tile **"Rules needing a decision"** (`apps/admin_panel/dashboard.py:rulebook_tile`): count of pending; foot "N not set · N proposed · N set"; links to the check page.

Frontend pages fed

- Portal **Rulebook** (`frontend/src/screens/rulebook/RulebookScreen.tsx`, eyebrow "Engagement"): two tabs.
  - **Settings** (`SettingsTab.tsx`): KPI tiles Set / Proposed / TBD, stacked bar by status, table "Settings in force" (rule + meaning, place, value in force, owner · counsel item, status pill) with search and status filter; `ImpactTable` (strict vs lenient contacts that change status) only when `impact` is non-empty — backend always returns `[]`, so it never shows with the real backend. Data: `useRulebook()` → `GET /rulebook`.
  - **Calendar & hours** (`CalendarTab.tsx`): `CanCallBox` ("Can I call now?" → `POST /can-call`, served by `apps/contacts`, not this app; number not stored), `HolidaysCard` (year picker, observed vs actual dates), `PlaceRulesCard` for calling hours and contact limits (TBD shown as "draft, not set yet"), Opt-out timing card, "Which time zone we use" card. Data: `useRulebookCalendar(year)` → `GET /rulebook/calendar?year=`.
  - `RuleStatusPill.tsx`: Set (green, "Decided") / Proposed (amber, "Runs; results are marked provisional until a lawyer confirms it") / TBD (gray, "Not set yet: checks that need it do not run").
- Portal **Reports** (`screens/reports/ReportCard.tsx`): Rulebook snapshot downloads `/api/rulebook/snapshot?period=`.

3. Regulatory app — law changes, rule updates, lawyer reviews (Module 2 step 5–6)

Purpose

Records law changes (FCC orders, court rulings, state laws) and the Rulebook rules each touches, shows them to clients, lets each client's lawyer agree / disagree with draft rule values and suggest law changes, and lets a CiV admin approve a draft so it enters the Rulebook from its start date. Adding a law change never changes a rule by itself.

Data it fetches / receives

- Typed in by CiV staff in the admin panel, or created from a law-watcher item (Module 13).
- From the portal: "read" marks, lawyer reviews, lawyer suggestions.
- Starting data from migrations: `0002_starting_changes` adds the 4 portal sample changes (FCC revocation order 30 Sep 2026; 7th Circuit Steidinger v. Blackstone 14 Jul 2026; Oregon HB 3865 in force 1 Jan 2026; Texas SB 140 in force 1 Sep 2025), suggested_by "Portal sample data", review "to check". `0008_vendor_clauses_draft` adds "CiV draft: clauses each vendor contract must contain" (jurisdiction "CiV draft", status proposed, metrics M07, domain VND) with one RuleUpdate for `vendor_required_clauses` (value from the YAML, shown "12 clauses, 5 required", status set, starts 2026-10-22, proposed_by "CiV (draft)") so clients' lawyers can review it.

Tables

- **`reg_change`** (`RegChange`, shared, not client-locked): `title`, `jurisdiction` (free text e.g. "Federal · FCC", "Oregon"), `status` (proposed / adopted / in_force / court_ruling), `announced_on`, `effective_from` (null = unknown), `effective_text` (shown to clients), `summary`, `rule_keys` (JSON list), `metrics` (JSON, codes like M10), `domains` (JSON, codes like REV), `source`, `source_url`, `review` (to_check / checked by counsel), `suggested_by`, `created_at`, `updated_at`.
- **`rule_update`** (`RuleUpdate`): FK `reg_change` (PROTECT), `key` (blank = brand-new rule), `new_key`, `new_meaning`, `jurisdiction` (default US), `value` (JSON), `shown`, `status` (set / proposed; default set), `effective_from`, `state` (waiting "Draft · waiting for approval" / approved / rejected), `proposed_by`, `approved_by`, `approved_at`, `created_at`. Custom permission `regulatory.approve_ruleupdate` ("Can approve rule updates (changes the Rulebook)").
- **`rule_update_review`** (`RuleUpdateReview`, client-locked RLS): FK client, FK rule_update, FK reviewer (ClientUser, SET_NULL), `reviewer_name`, `verdict` (agree / disagree), `comment`, `reviewed_at`. One per client per update.
- **`reg_change_read`** (`RegChangeRead`, client-locked): client, reg_change, `read_by`, `read_at`. One per client.
- **`client_suggestion`** (`ClientSuggestion`, client-locked): client, `suggested_by`, `title`, `jurisdiction`, `summary`, `source`, `source_url`, `created_at`, `handled`, `civ_note`.
- RLS: migration `0006_lock_client_tables` calls `apps.tenancy.sql.lock_table` on the three client tables.

Flow (step by step)

1. CiV staff add a law change in Admin → Rules → Regulatory changes (or create it from a law item, section 4). `suggested_by` = staff email; add / edit / delete logged (`reg_change_added|changed|removed`).

2. On the change's **Rule updates** tab, staff add one line per new rule value: pick an existing rule ("Modifies rule") or leave it blank and give `new_key` + `new_meaning` for a brand-new rule; place, value (JSON same shape as current), shown, status, start date. `proposed_by` = staff email. State = waiting (draft).
3. Portal: every client sees all law changes (`GET /api/reg-changes`) with their waiting and approved updates (rejected are hidden).
4. Any client user may click "Mark as reviewed" → `RegChangeRead` (company-wide), logged `reg_change_read`.
5. The client's lawyer (`ClientUser.is_counsel`) may Agree / Disagree with an optional comment on each draft; upsert, changeable until CiV decides; logged `rule_update_reviewed`.
6. The lawyer may also "Suggest a change" (a law CiV may have missed) → `ClientSuggestion`, visible only to CiV; logged `law_change_suggested`.
7. Admin sees, next to each draft, "N agree · N disagree" and each reviewer (name, company, verdict, comment).
8. An admin with `approve_ruleupdate` clicks **Approve** → `approve.approve`: in one transaction set state approved, approved_by (email), approved_at; `apply_update` on the current internal version (old value ends the day before, new starts; overlap refused → nothing saved, error message "Not approved: ..."). Then `rules_changed` is sent with `from_date = effective_from` (consent results from that date are rechecked) and `rule_update_approved` is logged.
9. **Reject** → state rejected, kept on record, logged `rule_update_rejected`. A **waiting** draft can be deleted (Delete tick box on its row, logged `rule_update_deleted`); approved and rejected ones stay. A whole law change can be deleted while none of its updates is approved (its drafts go with it, and a law item it came from goes back to New).
10. A later reload of the YAML re-applies all approved updates.

Logic and rules

- `RegChange.clean`: every `rule_keys` entry must exist in `rule_value`; metrics must match `^M\d{2}[a-z]?$`; domains `^[A-Z]{2,4}$`; title, summary, effective_text must not contain banned words.
- `RuleUpdate.clean`: existing key must exist; new key must match `[a-z][a-z0-9_]{2,79}`, must not already exist, needs a one-line meaning; new-rule fields must be blank when modifying an existing rule; two-letter place uppercased; `shown` checked for banned words.
- Banned words (`wording.py`, same list as portal `src/lib/wording.ts`): violation(s), breach(es), illegal, compromised, fake, fraud, fraudulent, genuine, compliant, non-compliant, noncompliant, ensures compliance, protects you, guaranteed, guarantee.
- Once approved or rejected, the update form's fields are disabled (cannot be edited).
- Reviews are advice only: CiV alone approves. A review on a non-waiting update → 400 "CiV has already decided on this update."
- Note: the "suggest" action is a separate new-law suggestion form; there is no per-update "suggest a value" field — the lawyer's alternative goes in the review comment.

Key functions / classes

- `apps/regulatory/models.py:RegChange` / `RuleUpdate` (`as_update()`, `rule_key`, `is_new_rule`) / `RuleUpdateReview` / `RegChangeRead` / `ClientSuggestion` / `LawItem`.
- `apps/regulatory/approve.py:approve` / `reject` — decide a rule update.
- `apps/regulatory/forms.py:RegChangeForm` (rules picked via Select2 from the real Rulebook, metrics / domains as comma text) / `RuleUpdateForm` / `rule_choices`.
- `apps/regulatory/api.py:RegChangesView` / `ConfirmView` / `ReviewView` / `SuggestView` / `IsClientLawyer`.
- `apps/regulatory/wording.py:banned_words`.

API endpoints

- `GET /api/reg-changes` — any client user. List of `{id "rc-N", title, jurisdiction, status ("in force" / "court ruling" / "adopted" / "proposed"), date, effective, summary, settings (rule keys), metrics, domains, confirmed, source, sourceUrl, checkedByCounsel, updates: [{id "ru-N", rule, isNewRule, meaning, place, value (shown), startsOn, state "draft"|"approved", myReview {verdict, comment, by, at}}]}`.
- `POST /api/reg-changes/<rc-N>/confirm` — any client user; "I've read this" for the whole company.
- `POST /api/rule-updates/<ru-N>/review` — company's lawyer only (`is_counsel`); body `{verdict: agree|disagree, comment ≤2000}`.
- `POST /api/reg-changes/suggest` — company's lawyer only; body `{title, jurisdiction, summary, source?, sourceUrl?}`; 201 `{id "cs-N", received: true}`.
- Admin-only: `POST /admin/regulatory/regchange/rule-update/<pk>/<approve|reject>/` (staff with `approve_ruleupdate`).

Scheduled jobs

- None for this part (the law watcher is section 4).

Admin panel parts (Admin → Rules)

- **Regulatory changes** (`RegChangeAdmin`): list title, jurisdiction, status pill, date, rules it touches, counsel pill (checked green / to check amber); filters status, review, jurisdiction; search; date hierarchy. Inline tab **Rule updates** with columns incl. "Clients' lawyers" summary and Approve / Reject buttons ("Waiting for an approver" for staff without the permission). Only waiting drafts can be deleted.
- **Client suggestions** (`ClientSuggestionAdmin`): read only except `handled` and `civ_note`; no add / delete; shows company.

Frontend pages fed

- Portal **Regulatory Changes** (`frontend/src/screens/regulatory/RegulatoryScreen.tsx`, eyebrow "Engagement"): `useRegChanges()` → `GET /reg-changes`; cards sorted unread first, then newest.

- `ChangeCard.tsx`: status pill, jurisdiction · date, title, summary, "Mark as reviewed" button (→ `POST /reg-changes/<id>/confirm`) or "Reviewed" pill; takes effect, Rulebook settings it touches, metric / domain codes, source link; "New rule values" list.
- `RuleUpdateRow.tsx`: rule key, "New rule" pill, place, new value from start date, Draft / "In the Rulebook" pill, the company's own review; lawyer sees "Review as our lawyer" / "Change review" → Agree / Disagree + comment (→ `POST /rule-updates/<id>/review`).
- `SuggestChange.tsx` (lawyer only): form What changed / Where / Summary / Source / Link → `POST /reg-changes/suggest`.
- Lawyer actions hidden when `session.view === 'underwriter'` (read-only view) or the user is not counsel. The backend only checks `is_counsel` (does not check the underwriter view).

4. AI law watcher — Module 13 (steps 1–4), in apps/regulatory

Purpose

Every morning, find new official federal items that may change a calling / texting rule, sort them by importance without AI, (later) let an AI draft summaries and rule values, and let staff turn an item into a law change with one click. Suggestions only: the Rulebook never changes by itself; a CiV admin decides.

Data it fetches / receives

- **Federal Register API** only: `https://www.federalregister.gov/api/v1/documents.json` (free, no key), User-Agent "CiV law watcher (Comply iV)". Two searches, both limited to document types RULE and PRORULE, newest first, 100 per page, up to 5 pages each:

1. CFR title 47 part 64 (`conditions[cfr][title]=47`, `conditions[cfr][part]=64`);
2. term `"Telephone Consumer Protection"`. Fields: title, abstract, publication_date, effective_on, document_number, html_url, type, agencies, docket_ids, raw_text_url.

- **Not fetched:** the FCC's own website (refuses automated readers, 403 on 2026-10-22; every FCC rule is in the Federal Register anyway); courts (CourtListener) and state laws (LegiScan) are "later" per README / docstring.
- When an AI is chosen: the full raw text (`raw_text_url`) of High items, up to 200,000 characters, in memory only.

Tables

- **`law_item`** (`LawItem`): `source` ("federal_register"), `ref` (FR document number; unique with source), `kind` (Rule / Proposed Rule / Notice), `title`, `summary` (source abstract), `agencies`, `published_on`, `effective_on`, `url`, `matched` (calling words matched), `state` (new / used "Turned into a law change" / dismissed), `found_at`; step 2: `rule_keys`, `level` (High / Medium / Low, default Low), `stage` (proposed / final / in_force), `dockets`, `follows` (FK self, the earlier proposal), `text_url`; step 3: `ai_summary`, `ai_drafts` (list of {key, value, shown, effective_from, quote, known}), `ai_confidence`, `ai_model`, `ai_read` (full_text / summary), `ai_done_at`; step 4: FK `reg_change`.

Flow (step by step)

1. Celery beat `law-watcher` runs `apps.regulatory.tasks.watch_laws` daily at **06:15** (crontab minute 15 hour 6; README says UTC) → `watcher.refresh()`.
2. Look-back: first run (no federal_register items yet) 365 days; later runs 7 days (anything missed is caught again; duplicates skipped by the unique (source, ref)).
3. Fetch both searches, merge by document number.
4. Keep only items whose title + abstract match a calling word (word-start regex): telephone consumer protection, tcpa, robocall, robotext, telemarket, do-not-call, do not call, autodial, automatic telephone dialing, artificial or prerecorded, prerecorded, text message, revocation, revoke consent, prior express, reassigned number, caller id, stir/shaken, call authentication, call blocking, one-to-one consent, consent, opt-out, opt out. (Part 64 also covers relay services etc., which are dropped.)
5. Save each new one as a `LawItem` (state new).
6. **Sort (step 2, `sorting.resort`)** — every item again, no AI:
   - `rules_touched`: fixed word → rule table: revoc/revoke/opt-out/stop → `optout_deadline_business_days`, `optout_scope_rules`, `optout_keywords`; calling hours / time of day / quiet hours / 8 a.m / 9 p.m → `calling_hours`, `quiet_hours_apply_to_sms`; frequency / how often / calls per → `frequency_limits`; consent words → `written_consent_types`, `consent_checklist`; autodial / prerecorded / artificial voice / AI-generated → `autodialed_modes`; text / sms / robotext → `sms_treated_as_autodialed`, `quiet_hours_apply_to_sms`; caller ID / STIR/SHAKEN / spoof → metric M21; reassigned → M11; do-not-call / registry → `dnc_check_max_age_days`.
   - **Level:** High = kind "Rule" (final) touching any rule with `affects_status`; Medium = a Proposed Rule, or a final Rule touching only other keys / metrics; Low = the rest.
   - **Stage:** not a Rule → proposed; Rule with effective_on ≤ today → in_force; else final.
   - **Follows:** a final Rule with docket numbers is linked to the latest earlier Proposed Rule sharing a docket.
7. **AI draft (step 3, `drafting.draft_new`)** — up to 20 new, undrafted items. **The AI slot is OFF**: `_drafter` is None until the owner picks the AI (decisions 2026-10-21 / 2026-10-22), so nothing is sent and nothing costs. When on: High items send the full text (falls back to summary if the download fails), others the official summary; input `{title, kind, summary, text, rules: [{key, shown, status}]}` (prompt version `law-v1`); output `{summary, rule_keys, drafts: [{key, value, shown, effective_from, quote}], confidence}`. Drafts without a key or without a quote are dropped; at most 10 kept; each marked `known` if the key is one the item touches / CiV has. Failures are retried the next day.
8. Staff open Admin → Rules → **New law items** (sorted High first, then newest), read the source, and either:
   - **Dismiss** (list action "Dismiss: not a calling-rule change", logged `law_items_dismissed`), or
   - **Create a law change from this item** (confirm page `regulatory/create_change.html`, needs `regulatory.add_regchange`) → `from_item.make_change`.
9. `make_change` (step 4, one transaction; idempotent — returns the existing change if already made): creates a `RegChange` with title, jurisdiction "Federal · FCC", status (not a Rule → proposed; Rule in force

→ in_force; Rule not yet in force → adopted), announced_on = published_on, effective_from = effective_on, effective_text ("From <date>" / "Not law yet (proposed)" / "Date to confirm"), summary = AI summary or official abstract or title, rule_keys (non-M keys), metrics (M keys), source "Federal Register <ref>", source_url, suggested_by "CiV law watcher", review "to check".

    - Only for a final **Rule**: one `RuleUpdate` per AI draft that is `known` and has a quote and a start date (draft's own date, else the item's effective_on; none → skipped, staff add by hand). Status **proposed**, place US, proposed_by "AI draft (<model>), added by <staff email>". A Proposed Rule gets no updates until it becomes final.
    - Item state → used, linked to the change; logged `law_change_from_item`.

10. From here the Module 2 process applies: lawyers review in the portal, CiV approves, approval ends the old value the day before the new start date (`apply.apply_update`) — the "expiry of old rule + new rule effective" mechanism.

Key functions / classes

- `apps/regulatory/tasks.py:watch_laws` — Celery task (daily).
- `apps/regulatory/watcher.py:refresh` / `documents` / `fetch` / `matches` — read and save Federal Register items.
- `apps/regulatory/sorting.py:resort` / `sort_item` / `rules_touched` / `level_of` / `stage_of` / `judging_rules`.
- `apps/regulatory/drafting.py:draft_new` / `draft_one` / `use` / `active` / `full_text` / `rules_for`.
- `apps/regulatory/from_item.py:make_change` / `status_of`.

API endpoints

- None in the portal (staff-only, admin panel). Admin URL: `/admin/regulatory/lawitem/<pk>/make-change/` (GET confirm, POST create).

Scheduled jobs

- `law-watcher` → `apps.regulatory.tasks.watch_laws`, `crontab(minute=15, hour=6)` (config/settings.py line ~192). It runs fetch → sort → AI draft (no-op while off).

Admin panel parts

- **New law items** (`LawItemAdmin`): list level pill (High red, Medium amber, Low gray), published date, stage, title, rules it may touch, state; filters state, level, stage, kind, source; search title / summary / ref; no add; everything read-only except `state`. Detail page: make-change button (or "Open its law change"), stage in words, "Existing rules it may change (value in force today)" (now <shown>, status, since date; M codes marked as metrics), "AI's draft" box ("No AI chosen yet: read the source and write the summary yourself." while off), FCC case numbers, "Follows the proposal" link, calling words found, "Open in the Federal Register" link.
- Admin dashboard tile **"New law items"**: count of new High items; foot "High · N new in all (law watcher)".

Frontend pages fed

- None directly. Its output reaches clients only after staff create a law change → portal Regulatory Changes page.

5. Module 13 step 5 — Rulebook admin (covered in section 2)

"Rulebook check" page, per-rule "History" page and the "Rules needing a decision" dashboard tile (`apps/rulebook/health.py`, `apps/rulebook/admin.py`, `apps/admin_panel/dashboard.py:rulebook_tile`). There is still no in-admin editor for rule values: changes go through the YAML file or an approved RuleUpdate.

Tests present (names only)

`tests/rulebook/test_business_days.py`, `test_health.py`, `test_loader.py`, `test_lookup.py`; `tests/regulatory/test_portal_api.py`, `test_reg_changes.py`, `test_rule_updates.py`, `test_watcher.py`.

4. Data intake: connectors, sync, normalizer, imports, reconciliation, uploads (Modules 3–5)

Read from the code on 22 October 2026. Backend root `/home/shakib/Audit/backend`, frontend root `/home/shakib/Audit/frontend`. Paths below are relative to `backend/` unless they start with `frontend/`.

Overall pipeline (one line): **Connection (locked key)** → **sync engine polls the tool** (Celery, every 2 min) → each record **fingerprinted** (`fingerprint` table, raw bytes hashed, never kept) → phone numbers turned into **phone codes** (HMAC with the client's key) → **hand-off receivers** (normalizer → `contact_event`; imports → `crm_creation`; certificates; opt-outs; consent) → record content dropped. In parallel: **Upload centre** files for sources with no API, **import batches** every 15 min, **records completeness** every night.

1. Connections (apps/connections) — Module 3 setup side

Purpose

One `Connection` = one tool (dialer, texting, certificate account, lead feed/platform, CRM) at one client company, read-only, with the client's key locked (encrypted). Staff add connections in the admin; the system tests them and derives the client's **data access level 0–4**.

Data it receives / where from

- Keys typed by CiV staff in the admin (Account ID / API key / API secret fields) for `api_key` tools, plus a visible **"Time zone or region"** dropdown (`connection.setting`, 2026-10-23): US time zones first, then others, and the Genesys regions; not a secret, editable later. Five9, Convoso and Vicidial read their times in it (older connections that kept the time zone in the secret still work); Genesys uses it as its region.
- For `oauth` tools ("Sign in with…": RingCentral, NICE CXone, Talkdesk, Salesforce, HubSpot, Zoho, GoHighLevel, Dynamics 365) no key is typed; staff email the client's Admins a link. The CRM adapters expect `access_token`, `refresh_token`, `instance_url` inside the locked credentials.
- Test calls (one read-only call per tool) — endpoints listed under testers below.

Providers (apps/connections/providers.py — PROVIDERS, dropdown order; kind; auth; field meanings)

Kinds (`Kind`): `dialer`, `sms` (Texting tool), `certificates`, `lead_feed` (Lead feed or lead platform), `crm`.

Code	Name	Kind	Auth	account →	key →	secret →
five9	Five9	dialer	api_key	Username (reports only)	Password	– (time zone in the setting box)
convoso	Convoso	dialer	api_key	–	Auth token	– (time zone in the setting box)
ringcentral	RingCentral	dialer	oauth			
vicidial	Vicidial	dialer	api_ke	Database server	Read-	Password (server time zone in

Code	Name	Kind	Auth	account →	key →	secret →
			y	host:port/database	only DB user	the setting box)
genesys	Genesys Cloud	dialer	api_key	Client ID	Client secret	– (region in the setting box)
nice_cxone	NICE CXone	dialer	oauth			
talkdesk	Talkdesk	dialer	oauth			
readymode	ReadyMode	dialer	api_key	Account domain	API key	–
aircall	Aircall	dialer	api_key	API ID	API token	–
dialpad	Dialpad	dialer	api_key	–	API key	–
amazon_connect	Amazon Connect	dialer	api_key	Access key ID	Secret access key	Instance ID and region
other_dialer	Other dialer	dialer	api_key	Account ID	API key	API secret
twilio	Twilio	sms	api_key	Account SID	Auth token	–
telnyx	Telnyx	sms	api_key	–	API key	–
bandwidth	Bandwidth	sms	api_key	Account ID	API username	API password
plivo	Plivo	sms	api_key	Auth ID	Auth token	–
vonage	Vonage	sms	api_key	API key	API secret	–
sinch	Sinch	sms	api_key	Service plan ID	API token	–
other_sms	Other texting tool	sms	api_key	Account ID	API key	API secret
trustedform	TrustedForm	certificates	api_key	–	API key	–
jornaya	Jornaya	certificates	api_key	Account code (LAC)	API key	–
lead_feed	Lead feed copy (CiV as extra destination)	lead_feed	api_key	–	Shared secret	–

Code	Name	Kind	Auth	account →	key →	secret →
leadconduit	LeadConduit (ActiveProspect)	lead_feed	api_key	Account ID	API key	–
boberdoo	Boberdoo	lead_feed	api_key	Server address	API key	–
leadspedia	LeadsPedia	lead_feed	api_key	Account	API key	API secret
phonexa	Phonexa	lead_feed	api_key	Account	API key	–
salesforce	Salesforce	crm	oauth			
hubspot	HubSpot	crm	oauth			
zoho	Zoho CRM	crm	oauth			
gohighlevel	GoHighLevel	crm	oauth			
velocify	Velocify	crm	api_key	Username	Password	–
dynamics	Microsoft Dynamics 365	crm	oauth			
pipedrive	Pipedrive	crm	api_key	–	API token	–
other_crm	Other CRM	crm	api_key	Account ID	API key	API secret

34 providers in all. Only 6 have a sync adapter (twilio, convoso, five9, vicidial, salesforce, hubspot) and 6 have a live tester (twilio, convoso, five9, vicidial, trustedform, jornaya). All others save the key and stay "Saved, not tested" / "Not read yet".

Tables

- `connection` (model `Connection`): `client` FK, `provider` (code above), `kind` (set from provider on save, not editable), `label` (optional, e.g. "Main dialer"), `credentials` (TextField: Fernet-encrypted JSON `{account, key, secret}` or OAuth tokens; never shown again), `status`, `last_checked_at`, `last_message` (300), `created_at`, `updated_at`. Ordering client name, kind, provider. It is CiV's own table.
- Writes `client.access_level` (field on `client` table, 0–4, not editable).

Statuses (Status)

`not_connected` (Not connected) · `waiting_client` (Waiting for client — OAuth link sent / new OAuth connection) · `untested` (Saved, not tested) · `connected` · `failed` · `stale` (set by sync health). Admin pill colours: connected green, failed red, stale amber, waiting teal, untested/not_connected gray.

Crypto (apps/connections/crypto.py)

- `lock(values: dict) -> str`: JSON → `MultiFernet.encrypt`. `unlock(token) -> dict`.
- Keys from env `CIV_SECRETS_KEYS` (comma-separated Fernet keys): the **first** encrypts, **any** decrypts (key rotation). Missing env → `ImproperlyConfigured`. The same lock is reused for the client phone-code key (`apps/keyring`).
- `Connection.set_credentials` drops empty values; when a key is edited the admin merges new fields over the old (`{**old, **new}`) so leaving a field empty keeps the saved value.

Access level (apps/connections/access_level.py)

Ladder (each level needs the one below): 1 = dialer **or** sms connected; 2 = + certificates; 3 = + lead_feed; 4 = + CRM. A connection counts when status is `connected` or `stale`. `refresh_access_level(client)` saves the value if changed. Called by signal on every Connection save/delete (`signals.py`) and by sync health every 10 min. Text shown in portal (`sync/api.py LEVEL_TEXT`): 0 nothing connected yet · 1 dialer or texting connected · 2 plus a certificate account · 3 plus a lead feed · 4 plus a CRM.

Testers (apps/connections/testers.py — run_test(connection) -> (status, message), never changes the tool)

- OAuth tool → keeps status, "Connects when the client clicks Allow".
- No credentials → `not_connected`. No tester → `untested` "A live test … comes with its connector".
- `test_twilio`: `GET https://api.twilio.com/2010-04-01/Accounts/{sid}.json` (Basic auth) → connected with friendly_name; HTTP error → failed.
- `test_convoso`: validates time zone (secret), then `POST https://api.convoso.com/v1/log/retrieve` with `auth_token`, last hour, `limit=1` → `success` must be true.
- `test_five9`: validates report time zone, runs the Call Log report for the last minute via the adapter.
- `test_vicidial`: connects with PyMySQL and runs `SELECT <id> FROM vicidial_log LIMIT 1` and same on `vicidial_closer_log`; MySQL 1045 → "refused user/password"; other OperationalError → network; ProgrammingError → "give SELECT on vicidial_log and vicidial_closer_log".
- `test_trustedform` / `test_jornaya`: ask about a made-up certificate (`'0'*40` / zero UUID) through `apps.certificates.vendors`; `NotFound` = key works, `Refused` = bad key. Jornaya requires the account code (LAC).
- Timeout 10 s.

Flow (staff adds a tool)

1. Admin → Client companies → a company → **Connections** tab (inline; there is no separate Connections page).
2. Choose tool (grouped dropdown), optional label, type Account ID / API key / API secret (write-only password widgets). Form refuses keys for OAuth tools ("Leave the key fields empty and use Send connect link").

3. Save (`save_connections`): deleted rows logged `connection_removed`; new keys locked and merged; a new OAuth connection gets status `waiting_client`; access-log entry `connection_added` / `connection_key_changed` / `connection_changed`.
4. If a key changed → test runs at once (`test_and_store`) → status, message, last_checked_at saved; logged `connection_tested`; admin message shows the result.
5. Row buttons: **Test connection now** (api_key tools) or **Send connect link to client Admins** (OAuth tools).
6. Send link: emails every active client Admin (`emails.send_connect_request`, subject "Connect <tool> to CiV", body points to `{PORTAL_URL}/sources`); status → `waiting_client`; logged `connect_link_sent`. No active Admin → warning. (Emails currently go to console — see memory "Users email later".)
7. Signal recomputes access level.

Key functions / classes

- `apps/connections/models.py:Connection` — the row; `tool`, `has_credentials`, `set_credentials`, `get_credentials`.
- `apps/connections/providers.py:PROVIDERS / Provider / Kind / PROVIDER_CHOICES` — tool catalogue.
- `apps/connections/crypto.py:lock / unlock` — Fernet locking.
- `apps/connections/access_level.py:access_level / refresh_access_level` — level 0–4.
- `apps/connections/testers.py:run_test / TESTERS` — live tests.
- `apps/connections/admin.py:ConnectionInline / save_connections / test_and_store / sync_summary` — admin tab.
- `apps/connections/views.py:test_connection / send_connect_link` — POST buttons (perm `connections.change_connection`).
- `apps/connections/emails.py:send_connect_request` — email to client Admins.
- `apps/connections/forms.py:ConnectionForm / ConnectionInlineForm / key_guide` — form + "Which fields each tool needs".

API endpoints

None for clients directly (connections are surfaced through `GET /api/sources` and `/api/data-health`, see §2). Admin-only URLs (staff, `admin_view` + `connections.change_connection` perm):

- `POST /civ-admin/clients/client/<client_id>/connections/<connection_id>/test/`
- `POST /civ-admin/clients/client/<client_id>/connections/<connection_id>/send-link/`

Scheduled jobs

None of its own (health job in sync touches status).

Admin panel parts

Client page → Connections tab: fields provider, label, account, key, secret, Status pill, **Sync** column (Live / Catching up / Stale / Not read yet ("CiV starts reading as soon as the connection test passes"), "last read X ago · N records read · problem: <ErrorType> (n× in a row)"; "Not read yet — its adapter comes when a client needs it" for tools without adapter; "waiting for the client to click Allow" for OAuth without cursor), Saved key ("Saved · hidden"), last checked, last message, action button. Summary tile "data access level" on the client page (README).

Frontend pages fed

Source Registry, Overview Data health card, Setup & Readiness (via sync/onboarding APIs).

2. Live sync engine (apps/sync) — Module 3

Purpose

Reads every connected tool on a timetable, a small batch at a time, so every client takes turns; fingerprints each record, turns phone numbers into phone codes, hands the record to receivers, then drops it. Tracks per-tool health.

Data it fetches

Via adapters (`apps/sync/connectors/*`), detailed below. Only 6 tools have adapters.

Tables

- `sync_cursor` (`SyncCursor`, one per connection, OneToOne): `client`, `connection`, `source_system` (provider code), `cursor` (tool bookmark, ≤500 chars), `sweep_cursor` (weekly full-check bookmark; '' = start), `mode` (`live` / `poll`, default poll), `last_event_at` (last record received), `last_heartbeat_at` (last successful call), `last_catchup_at`, `last_full_sweep_at`, `sweep_started_at`, `backlog` (tool had more when last run stopped), `records_delivered` (count), `failures_in_row`, `last_error` (error **type name only**, ≤120, so no record text leaks).
- `sync_gap` (`SyncGap`, "missed record"): `client`, `source_system`, `source_record_id`, `sha256`, `found_by` (`catchup` / `full_sweep`), `changed_at`, `found_at`, `delay_seconds`. Only written for tools in `live` mode — all tools are poll mode now, so it stays empty.
- Writes `fingerprint` (apps/ledger: client, job_id, source_system, source_record_id, content_kind, source_grade, sha256, fetched_at, anchor/leaf_index/merkle_path for the nightly seal) and `purge_log` on failure.
- Both sync tables are row-level-security locked (`lock_table`).

Adapter contract (apps/sync/connector.py)

- `Record(source_record_id, raw: bytes, content_kind, changed_at, phones: list[str])` — raw = exact bytes (adapters build canonical JSON `sort_keys, separators=(',',':')`), phones in memory only.
- `Page(records, next_cursor, more)`.

- `Connector`: `mode='poll'`, `source_grade='C'` (client systems by API), `min_poll_minutes=0`, `counts=()` (measures the tool can total itself), methods `changes_since(cursor, limit)`, `list_all(cursor, limit)` (default = changes_since from ''), `count_between(measure, start, end)`, `fetch_one(record_id)` (Module 14 re-fetch; None = gone; NotImplementedError = cannot), `parse_event(body, headers)` (live notice; must raise `BadSignature`).
- `CONNECTORS` registry dict; `connector_for(connection)`. Registered via `apps/sync/connectors/__init__.py` imports.

Connectors (one per tool) — what they read

Twilio (`connectors/twilio.py`, texts + calls; creds account=SID, key=auth token; Basic auth)

- `GET https://api.twilio.com/2010-04-01/Accounts/{sid}/Messages.json?DateSent>=…&DateSent<…&PageSize=1000` and `…/Calls.json?StartTime>…&StartTime<…`, follow `next_page_uri`; window 1 day; lag 2 min; timeout 20 s.
- Two streams (texts, calls), each bookmark `<iso time>#<n done at exactly that time>`, stored together as JSON. Texts get half the batch, calls the rest. Time = `date_sent`/`start_time` (fallback `date_created`), RFC 2822.
- Record id = Twilio `sid`; content_kind `text`/`call`; phones = `to`, `from` (only those starting `+`).
- `fetch_one`: `GET …/Messages/{sid}.json` or `…/Calls/{sid}.json` (CA prefix = call); 404 → None.
- `parse_event`: validates `X-Twilio-Signature` = base64 HMAC-SHA1(auth token, full URL + sorted form fields); MessageSid/SmsSid → text, CallSid → call.
- `counts = ()` (no own total); completeness uses Twilio **billing** instead (§5).

Convoso (`connectors/convoso.py`, dialer; key=auth token, secret=account time zone, default America/New_York)

- `POST https://api.convoso.com/v1/log/retrieve` form body `auth_token, start_time, end_time, limit=1000, offset, order=asc` (token in body, not URL). `success` false → `ConvosoError`. Pages by offset until short page or `total_found` reached. Window 1 h, lag 2 min, timeout 30 s.
- Times are account-local `%Y-%m-%d %H:%M:%S` → converted to UTC. Bookmark `<time>#<n>`.
- Record id = Convoso `id`; kind `call`; phones = `phone_number`, `number_dialed`, `caller_id_displayed`.
- `count_between`: same endpoint `limit=1` → `total_found`. `counts=('calls',)`.
- `fetch_one`: `log/retrieve` with `id=<record_id>` — marked "TO CONFIRM on a real account".
- Also used by imports: `POST /v1/leads/search` (`created_at_start`, limit 1000) for dialer list entries.

Five9 (`connectors/five9.py`, dialer; account=reports-only username, key=password, secret=report time zone)

- SOAP `https://api.five9.com/wsadmin/v12/AdminWebService` (Basic auth): `runReport` (folder "Call Log Reports", report "Call Log", criteria time start/end) → `isReportRunning` (timeout 10 s, up to 12 rounds ≈ 2 min; else TimeoutError, retried next turn) → `getReportResultCsv`. SOAP faults → `Five9Error`.
- Window 1 h, lag 5 min, **min_poll_minutes = 10** (read at most every 10 min). Whole window processed at once (one report per window); bookmark = window end ISO.

- Columns matched loosely (CALL ID/CALLID/SESSION ID; TIMESTAMP or DATE + TIME, several formats; ANI, DNIS, NUMBER/NUMBER1). Missing call id → first 32 hex of sha256(raw).
- `count_between`: report rows counted. `counts=('calls',)`. **No `fetch_one`** (re-fetch = "cannot").

Vicidial (`connectors/vicidial.py`, dialer; account=`host:port/database` (db default `asterisk`, port 3306), key=read-only DB user, secret=password; PyMySQL, new connection per query, timeouts 20 s; SELECT only)

- Tables: `vicidial_log` (outbound, id `uniqueid`) and `vicidial_closer_log` (inbound, id `closecallid`); columns uniqueid/closecallid, lead_id, list_id, campaign_id, call_date, start_epoch, end_epoch, length_in_sec, status, phone_code, phone_number, user, term_reason, alt_dial/uniqueid.
- `WHERE (start_epoch > e OR (start_epoch = e AND id > last)) AND start_epoch <= now-120s ORDER BY start_epoch, id LIMIT half-batch`. Bookmark JSON `{outbound: "<epoch>|<id>", inbound: …}`. No time-zone setting needed (epoch).
- Record id `outbound:<id>` / `inbound:<id>`; phone = phone_code + phone_number.
- `count_between`: `SELECT COUNT(*)` on both tables. `fetch_one`: `SELECT … WHERE id = %s LIMIT 1`.
- Also used by imports: `SELECT lead_id, list_id, entry_date, phone_number FROM vicidial_list WHERE lead_id > %s`.

Salesforce (`connectors/salesforce.py`, CRM, OAuth via `OAuthTool`; API v62.0)

- SOQL through `GET {instance_url}/services/data/v62.0/query?q=…` (follows `nextRecordsUrl`): `SELECT FIELDS(ALL) FROM Lead|Contact|Task WHERE [TaskSubtype='Call' AND] (SystemModstamp > t OR (SystemModstamp = t AND Id > last)) AND SystemModstamp <= now-2min ORDER BY SystemModstamp, Id LIMIT ≤200` (200 is the FIELDS(ALL) cap).
- Objects: `leads` (kind `lead`, phones Phone, MobilePhone), `contacts` (kind `crm_record`, Phone, MobilePhone, HomePhone, OtherPhone), `calls` = Tasks of subtype Call (kind `crm_record`, no phones). Bookmark per object `<stamp>|<Id>`.
- `count_between`: `SELECT COUNT() FROM Lead/Contact WHERE CreatedDate in range` (sum). `counts=('crm_records',)`.
- `fetch_one`: `SELECT FIELDS(ALL) … WHERE Id='<15-18 alnum>'`.
- Token renewal: `POST https://login.salesforce.com/services/oauth2/token` (refresh_token grant).

HubSpot (`connectors/hubspot.py`, CRM, OAuth)

- `GET https://api.hubapi.com/crm/v3/properties/{contacts|calls}` (to discover property names) then `POST /crm/v3/objects/{contacts|calls}/search` filter `hs_lastmodifieddate` GTE mark and LTE now-2min, sort ascending, limit 100, paging `after`.
- Properties requested: standard list (contacts: phone, mobilephone, hs_lastmodifieddate, createdate, lifecyclestage, hs_lead_status, hs_analytics_source, hs_latest_source, hs_email_optout, hs_object_source, hs_created_by_user_id, hs_updated_by_user_id, hs_lifecyclestage_customer_date; calls: hs_timestamp, hs_call_direction, hs_call_disposition, hs_call_status, hs_call_to_number, hs_call_from_number, hs_call_duration) **plus** any property whose name contains trustedform, leadid, jornaya, consent, vendor, provider, publisher, affiliate, supplier, lead_source, utm_source.
- Record id `contacts:<id>` / `calls:<id>`; kind `crm_record` for both. Bookmark per object `<ms>#<n>`.

- `count_between`: search `total` with `limit 1` (contacts by `createdate` for crm_records; calls by `hs_timestamp`). `counts=('crm_records','calls')`.
- `fetch_one`: `GET /crm/v3/objects/{path}/{id}?properties=…`; 404 → None.
- Token renewal: `POST https://api.hubapi.com/oauth/v1/token`.

OAuth helper (`connectors/oauth.py:OAuthTool`): `call(url, body=None)` GET or POST JSON with `Bearer access_token`; on 401 renews once with refresh_token + CiV app id/secret from settings, saves new access_token/refresh_token/instance_url locked on the connection; still 401/403 → `Disconnected` ("someone must click Allow again"). Timeout 30 s.

Flow (one poll turn)

1. Beat runs `poll_due` every 2 min (in all-clients scope). For each `syncable()` connection (provider has adapter, status connected/stale, client `can_sync` = stage Trial/Onboarding/Active), if cursor mode is poll and `last_heartbeat_at` is empty or older than the adapter's `min_poll_minutes`, queue `sync_connection(id,'poll')`.
2. `engine.run(connection_id, kind)`: inside `tenant(client)`, get/create cursor; start = `cursor.cursor` (or `sweep_cursor` for full_sweep; a new sweep sets `sweep_started_at`).
3. Call adapter `changes_since` (poll, catchup) or `list_all` (full_sweep) with `SYNC_BATCH_SIZE` (env, default 500).
4. In one DB transaction + `ledger_job`: `process()` each record:
   - full_sweep: tell `handoff.SEEN` receivers all record ids read;
   - skip if (record id, sha256(raw)) already fingerprinted (same content = no double counting);
   - else write a `fingerprint` (grade from adapter), turn phones into tokens with `Tokenizer(client)`, call `handoff.deliver` → every receiver; if tool is in live mode and the id was never seen and found by catch-up/sweep → write `sync_gap`;
   - `records_delivered += new`.
5. Move bookmark (`cursor`, `backlog = page.more`; or `sweep_cursor`), set heartbeat, reset failures; set `last_event_at` if records; set `last_catchup_at` / `last_full_sweep_at` when that kind finished; when a full sweep finishes, call `handoff.SWEEP_DONE` receivers (records not seen since `sweep_started_at` are gone from the tool).
6. Any exception: whole batch rolled back; ledger job deletes its fingerprints and writes one `purge_log` line (error type); cursor `failures_in_row += 1`, `last_error = type name`; return False (retry next turn).
7. If `page.more`, the task re-queues itself at the back of the queue (clients take turns).

Hand-off receivers registered (apps/*/apps.py ready())

`handoff.RECEIVERS` (client, connection, record, phone_tokens): `apps.contacts.normalize.normalize_record`, `apps.contacts.epochs.crm_record_seen`, `apps.imports.creations.creation_seen`, `apps.certificates` `scan_record` (certificate links → 72-h claim queue), `apps.optouts` `optout_seen` (STOP texts, CRM do-not-call boxes), `apps.consent` `proofs_seen` and

`keeping.contact_seen`. `handoff.SEEN`: `consent.keeping.records_seen`. `handoff.SWEEP_DONE`: `consent.keeping.sweep_done`. Receivers get the record in memory and must not keep content.

Logic and rules

- Bookmark moves only after the batch is committed → crash never skips or doubles.
- Each adapter waits a lag (2 min; Five9 5 min) so records still being written are not half-read.
- All adapters start from the client's `start_date` at 00:00 UTC.
- Re-reading the same record with different bytes = processed again (an update), same bytes = skipped.
- Health (`health.py`): `state(cursor)` = `never` (no heartbeat), `stale` (heartbeat older than Rulebook `source_freshness_hours` = 2, default 2), `catching_up` (backlog), else `live`. `check_all()` every 10 min: connected + stale state → status `stale`, last_message "No news for over N hours", access-log `source_stale`; stale + now live/catching_up → back to `connected`, access-log `source_recovered`; recomputes access level for each cursor's client.
- Live notices ("hooks") **OFF** (owner decision 2026-10-16): route `hooks/<connection_id>` is only added in `config/urls.py` when env `SYNC_HOOKS_ENABLED=true`. View `apps/sync/views.py:hook` (csrf_exempt, POST): 404 unless connected/stale with adapter and client can sync; `parse_event` signature fail → 403; else `receive_live` (does not move bookmark) → 204. Only Twilio implements `parse_event`.
- Rulebook keys `poll_interval_minutes` 2, `catchup_interval_minutes` 60, `full_sweep_interval_days` 7 exist but the schedule is hard-coded in `CELERY_BEAT_SCHEDULE` (comment links them); the code does not read them.

Key functions / classes

- `apps/sync/connector.py:Connector / Record / Page / CONNECTORS / connector_for / BadSignature`
- `apps/sync/engine.py:run` — one batch; `receive_live` — live notice; `syncable` — readable connections; `cursor_for`.
- `apps/sync/process.py:process` — fingerprint + dedupe-by-hash + deliver + gap; `connection_grade`.
- `apps/sync/handoff.py:deliver / RECEIVERS / SEEN / SWEEP_DONE`
- `apps/sync/health.py:state / check_all / fresh_hours`
- `apps/sync/tasks.py:sync_connection / poll_due / catchup_all / full_sweep_due / check_health`
- `apps/sync/views.py:hook` — live-notice door (off).
- `apps/sync/api.py:SourcesView / DataHealthView / tool_row / upload_row / records_completeness / conversation_review / proof_requests_waiting`
- `apps/ledger/record.py:ledger_job / LedgerJob.record` — fingerprint writer with cleanup.
- `apps/keyring/tokens.py:Tokenizer` — phone → HMAC-SHA256 token with the client's key (`kid` k1); `apps/keyring/phones.py:normalize` — E.164 (US default, "possible" not "valid").

API endpoints (client portal, IsClientUser = any signed-in client user)

- `GET /api/sources` — Source Registry rows. Tool rows: `{id: conn-<pk>, name (+label), tier (certificates 2, other tools 3), access ("Connected by API · read every N minutes" with N = max(2, min_poll_minutes); "API key"; "Sign in with X"; "Saved"), lastSync ("X ago"), status (current / stale / not_supplied), feeds (domain codes by kind), revocable true, acceptsUpload false, blocks (reason text)}`. Plus 6 upload rows (tier 4, access "Upload", acceptsUpload true, status current once any file of that source was uploaded, lastSync = newest file's time). FEEDS: dialer CON DLR PRV CTF IDD SHK REV REC IDN XBD · sms CON SMS CTF REV REC IDN XBD · crm CON LPV MNA IDN REV REC · certificates CON WEB LPV RET · lead_feed LPV VND AFF WEB. Upload feeds: internal list IDN · policies GOV VND · complaints INC · DNC checks NDN SDN · RND queries RND · vendor lead files LPV VND. Upload "blocks" when not supplied: DNC check records "Blocks M13", RND query logs "Blocks M11".
- `GET /api/data-health` — `{accessLevel, accessLabel, lastChangeAt (max last_event_at), sources {live, stale, notSupplied, total}, recordsCompleteness, completenessFloor, completenessStatus (complete / incomplete / not_available / not_measured), completenessNote, conversationReview {reviewed, optOutsFound, missed, notMarkedFirstCheck} | null, proofRequestsWaiting}`.
- `POST /hooks/<connection_id>` — tools only, only when hooks enabled (currently not routed).

Scheduled jobs (config/settings.py CELERY_BEAT_SCHEDULE, UTC)

- `sync-poll` → `apps.sync.tasks.poll_due` every 120 s.
- `sync-catchup` → `catchup_all` hourly at :40 (queues a `catchup` for every syncable connection; uses the same bookmark as poll).
- `sync-full-sweep` → `full_sweep_due` daily 03:10; queues `full_sweep` only for connections whose last full sweep is empty or older than 7 days (so effectively weekly per tool).
- `sync-health` → `check_health` every 600 s.
- Celery: Redis broker (`REDIS_URL`); without it tasks run eagerly in-process; `acks_late`, prefetch 1.

Admin panel parts

- Connections tab Sync column (above).
- Admin → Records → **Missed records** (`SyncGapAdmin`, read-only, list found_at, company, tool, record id, found_by, "How late"; filters tool/found_by). Empty while all tools poll.
- Admin → Records → **Failed-job cleanups** (`purge_log`, from apps/ledger admin; README).
- `SyncCursor` has no admin page of its own.

Frontend pages fed

Source Registry (`/sources`), Overview Data health card + key numbers + attention banner, top-bar Freshness dot (stale sources), Setup & Readiness (connection checks).

3. Normalizer — Contacts (apps/contacts) — Module 4

Purpose

Turns every synced call or text, from any tool, into one row of the same shape (`contact_event`), with the person as a phone code, a masked label, probable location/time zones, dedupe across tools, and phone-ownership epochs. Also serves the Contact Ledger, the number page (evidence file base) and "Can I call now?".

Data it receives

Records from the sync hand-off (in memory), with raw JSON and phone tokens. No vendor calls of its own.

Tables

- `contact_event` (`ContactEvent`): `client`, `phone_token` (64, indexed) + `phone_token_kid`, `masked_label` ("(916) •••-0001"; outside +1: country code + "•••" + last 4 — owner decision D31), `source_system`, `source_record_id`, `source_grade`, `channel` (call/text), `direction` (outbound/inbound/unknown), `occurred_at` (UTC, indexed), `ended_at`, `answered` (null = tool does not say), `length_seconds`, `disposition` (tool's own result code as is), `sms_status`, `campaign`, `agent_id`, `caller_id_token` (number shown, as phone code), `lead_id`, `dial_mode` (blank = unknown), `prerecorded_or_ai` (null = unknown), `stir_attestation`, `is_test` (CiV test numbers, Module 9), `recipient_state`, `time_zones` (JSON list), `country`, `is_non_us`, `location_confidence` (high/medium/low), `location_source` (area_code), `dedupe_group` (UUID), `is_primary` (count only these), `epoch` FK, created/updated. Unique (client, source_system, source_record_id). Indexes (client, occurred_at), (client, phone_token, occurred_at). RLS locked.
- `phone_epoch` (`PhoneEpoch`): client, phone_token, `number` (1,2,3... time order; unique per client+token), `started_at` (null = before anything seen), `ended_at` (null = still owner), `reason` (first_seen, crm_moved, wrong_number, rnd, new_consent), `signal_source`. RLS locked.
- `crm_phone_holder` (`CrmPhoneHolder`): client, source_system, record_id, phone_token, held_from, released_at — which CRM record held a number when (IDs only). RLS locked.

Flow (normalize_record, per synced record)

1. `translate(provider, record)`: only content_kind call/text/crm_record; per-tool translator returns fields with `person` (raw number, memory only) and `caller_id`; None = not a contact (CRM leads/contacts, Salesforce Tasks, unknown tool, unreadable JSON, no person).
2. Clean number to E.164 (`keyring.phones.normalize`); not a number → stop.
3. Token with the client's key (`codes_for` caches a Tokenizer per client for 300 s).
4. `locate(number)` → state, time zones, country, is_non_us, confidence, source; `mask(number)` → label; caller ID → token.
5. `update_or_create` on (client, tool, record id) — a changed record updates its row, never doubles.
6. `dedupe.group(contact)` then `epochs.attach(contact)`.

Translators (apps/contacts/translators.py)

- **twilio**: direction from `direction` (in/*out*/"manual" → outbound); person = `from` if inbound else `to`; caller = the other. Text: sms_status = disposition = `status`. Call: `ended_at` from `end_time`, length `duration`, disposition `status`, answered = status 'completed' and length > 0.

- **convoso**: call; direction `call_type` (default outbound); person `phone_number` or `number_dialed`; caller `caller_id_displayed`; length `call_length`; answered = length > 0; disposition `status`; campaign `campaign`/`campaign_id`; agent `user_id`/`user`; lead_id.
- **five9**: direction `CALL TYPE`; person ANI if inbound else DNIS; caller the other; length from LENGTH / CALL TIME / TALK TIME (hh:mm:ss); disposition DISPOSITION; campaign CAMPAIGN; agent AGENT / AGENT NAME.
- **vicidial**: direction from record id prefix (inbound:/outbound:); person phone_code+phone_number; ended_at from end_epoch; length length_in_sec; disposition status; campaign campaign_id; agent user; lead_id.
- **hubspot**: only `calls:` records; direction hs_call_direction; person from/to number by direction; length hs_call_duration ms→s; disposition hs_call_disposition or hs_call_status. (occurred_at = record.changed_at, i.e. hs_lastmodifieddate, not hs_timestamp — noted as a possible inaccuracy.)
- **salesforce**: always None.

Location (apps/contacts/location.py)

phonenumbers offline data; US state from geocoder description; time zones from area code with a correction table `SPLIT_AREA_CODES` (219, 850/448, 812/930, 270/364, 423, 931, 906, 915, 432, 785, 620, 308, 701, 605, 208/986, 541/458, 928) and `NANP_COUNTRY_ZONE` for other +1 countries; US territories (PR, VI, GU, AS, MP) not "non-US". More than 4 zones = unknown (empty). Confidence medium = one zone, low = several/unknown; high reserved for ZIP (later). Corrections marked "TO VERIFY against NANPA / tz data".

Dedupe (apps/contacts/dedupe.py)

Same person token + same channel + same direction (or one unknown) + **different tool** + occurred_at within max(Rulebook `dedupe_window_seconds` 120, `clock_skew_seconds` 120) (default 120 s). Joins the nearest candidate whose group has no record from the same tool. Group key UUID; primary = best `SOURCE_RANK` (dialers/Twilio 1, CRMs 2, other

1. then earliest then pk. Before re-grouping, `leave_group` (a changed time/direction re-groups). Nothing deleted.

Epochs (apps/contacts/epochs.py)

- `epoch_for` creates epoch 1 (first_seen) on first sight; `attach` puts the contact in the epoch containing its time.
- `split(client, token, at, reason, source)`: ends current epoch at `at`, inserts new epoch, renumbers in time order (two-pass +1000 trick), moves later contacts. Idempotent if an epoch already starts at `at`.
- Signal now: `crm_record_seen` receiver (lead/crm_record, not calls): tracks `crm_phone_holder`; when a number appears on a different CRM record and every earlier holder has released it → `split(... CRM_MOVED, provider)`. A number on two records at once is a duplicate/household, not a new owner. Other reasons (wrong_number, rnd, new_consent) are called by other modules.

Key functions / classes

- `apps/contacts/normalize.py:normalize_record / mask / codes_for`
- `apps/contacts/translators.py:translate / TRANSLATORS`
- `apps/contacts/location.py:locate / state_code`
- `apps/contacts/dedupe.py:group / choose_primary / leave_group / window`
- `apps/contacts/epochs.py:epoch_for / attach / split / crm_record_seen`
- `apps/contacts/calling.py:window` — allowed calling hours per zone from Rulebook `calling_hours` (draft ruleset; advice only).
- `apps/contacts/proof_elements.py:proof_elements` — 8 proof elements for the number page (uses uploads tables).
- `apps/contacts/api.py:ContactsView / ContactView / NumberView / CanCallView / number_file / list_checks / find_tokens`

API endpoints (all IsClientUser)

- `GET /api/contacts?status=&q=&channel=&direction=` — primary contacts newest first, max 200; status ALL / NOT_CHECKED / VERIFIED|WEAK|CONFLICTING|NO_PROOF|PENDING|NOT_NEEDED; q = full number (tokenized, never stored) or last 4 digits (masked_label endswith).
- `GET /api/contacts/ce-<id>` — one contact row.
- `GET /api/evidence/p-<24 hex>` — number page (contacts, epochs, proof elements, opt-outs, list checks, consent, cameFrom, seal, hold, location, calling window).
- `POST /api/can-call {number}` — location + calling window now; not stored.

Scheduled jobs

None (runs inside sync tasks).

Admin panel parts

None.

Frontend pages fed

Contact Ledger, number/evidence page, Can I call now?, plus counts used by Overview/Setup/Plan & usage.

4. Import batches (apps/imports) — Module 5 step 1 (+ Module 8 step 3)

Purpose

Finds lists that were uploaded by hand into the CRM or dialer (bulk imports), as opposed to typed-by-hand records or automatic feeds, so numbers from uploaded lists can be checked for evidence, re-added opt-outs, and provenance.

Data it receives / fetches

- From sync hand-off: every CRM lead/contact record (`creation_seen`).

- Directly from dialers (every 15 min, `dialer_lists.read_all`): Vicidial `SELECT lead_id, list_id, entry_date, phone_number FROM vicidial_list WHERE lead_id > <max read> ORDER BY lead_id LIMIT 1000`; Convoso `POST /v1/leads/search` with `created_at_start`, limit 1000 (to confirm). Five9: later.

Tables (all RLS locked)

- `crm_creation` (`CrmCreation`): client, source_system (salesforce/hubspot), record_id (unique per client+tool), `created_at`, `created_by` (CRM user ID), `how` (tool label e.g. HubSpot IMPORT/FORM/INTEGRATION), `vendor` (lead vendor name from CRM fields), `sub_id`, `price`, `watched_sha` (fingerprint of opt-out/DNC/lead-source fields, never values), `opted_out` (bool/null), `phone_tokens` (JSON), `batch` FK.
- `import_batch` (`ImportBatch`): client, `system` (crm / crm_update / dialer), source_system, created_by, starts, ends, created_count, `kind` (bulk_import / small_batch / hand_entered / feed), labelled_by_tool, confirmed_by_client (null; "later"), opted_out_readded (M34), updated_count, optouts_cleared, list_id.
- `crm_update` (`CrmUpdate`): client, source_system, record_id, updated_at, updated_by, cleared_optout, phone_tokens, batch FK.
- `dialer_list_entry` (`DialerListEntry`): client, source_system, lead_id (unique per client+tool), list_id, added_at, phone_token, batch FK.

Flow

1. `creation_seen` (hand-off): skip non lead/crm_record and `calls:` ids. Read creation facts: Salesforce `CreatedDate`, `CreatedById`; HubSpot `createdate` (or createdAt), `hs_created_by_user_id`, `hs_object_source` (upper-cased). Extract `vendor_of` (first field whose name contains vendor/provider/publisher/affiliate/supplier/ lead_source/leadsource/utm_source/source_name, excluding category words in `NOT_A_VENDOR` and URLs), `sub_of` (sub_id/subid/sub_source/aff_sub…), `price_of` (lead_price/cost/cpl/price/payout, also `__c`; 0–100000), `watched` (fingerprint of DNC/opt-out (email ignored) and vendor fields; opted-out yes/no; last modifier `LastModifiedById` / `hs_updated_by_user_id`).
2. Existing record: if watched fields changed → write `CrmUpdate` (cleared_optout when it was opted out and now not); update watched_sha/opted_out; union phone tokens; fill vendor/sub/price only if empty. New record → insert.
3. Every 15 min task `rebuild_batches`: `dialer_lists.read_all()` → `rebuild_all()` → `provenance.requests.send` per active client.
4. `rebuild(client)` deletes and recomputes `system='crm'` batches per tool: rows sorted by creator, time; a gap over `import_gap_seconds` (120) or a creator change starts a new batch; `kind_of`: HubSpot IMPORT label → bulk (labelled); labels all in {INTEGRATION, API, APP, EXTENSION, BCC_TO_CRM} or creator is a "feed user" → feed; n ≥ `import_min_batch` (20) → bulk; n = 1 → hand_entered; else small_batch. Feed user = creates records on ≥5 distinct days and ≥10 distinct hours within the last 14 days.
5. `rebuild_updates`: same burst logic over `CrmUpdate` by updater → `system='crm_update'` batches with updated_count and optouts_cleared; feed users the same way.

6. `rebuild_dialer`: dialer entries whose token is not in any CRM creation, grouped by (tool, list) and time gap → `system='dialer'` batches by size.
7. After each client: `apps.optouts.readded.check_batches` (M34 opted-out numbers that came back).
8. `origin(client, token, at)` = batch of latest CRM record holding the token created before `at`, else dialer entry.

Key functions

- `apps/imports/creations.py:creation_seen / creation_facts / watched / vendor_of / sub_of / price_of`
- `apps/imports/batches.py:rebuild / rebuild_updates / rebuild_dialer / rebuild_all / kind_of / feed_users / origin / thresholds`
- `apps/imports/dialer_lists.py:read_all / read_vicidial / read_convoso / save`
- `apps/imports/tasks.py:rebuild_batches`

API endpoints

None of its own. Data surfaces through `GET /api/provenance` (uploads list, ladder; other slice), readiness setup rows (vendors count), records completeness (crm_records count), consent/metrics modules.

Scheduled jobs

`import-batches` → `apps.imports.tasks.rebuild_batches` every 900 s.

Admin panel parts

None registered for imports (no admin.py). Visible indirectly via Lead provenance tile / Proof requests.

Frontend pages fed

Lead Provenance "Uploaded lists" card (`frontend/src/screens/leads/UploadsCard.tsx`, via /api/provenance), Setup checklist vendors row.

5. Records completeness / reconciliation (apps/reconcile) — Module 5 step 3 (M37)

Purpose

"Is CiV seeing every call and text?" Monthly, per connected tool, compares what CiV read with an independent number (bill, tool's own total, or working-day pattern). The lowest ratio is metric **M37**; under the floor the month is "incomplete data" (missing share takes worst-case values in Module 10 scoring). Also stores the month's contact volume (Exposure "Volume" input).

Data it fetches

- Twilio billing: `GET https://api.twilio.com/2010-04-01/Accounts/{sid}/Usage/Records/Daily.json?Category= sms-outbound|calls-outbound&StartDate&EndDate&PageSize=1000` (pages via `next_page_uri`), sums `count` (`reconcile/billing.py:twilio_billed`).

- Tool totals via each adapter's `count_between` (Convoso, Five9, Vicidial, Salesforce, HubSpot — see §2).
- Local tables: contact_event, crm_creation, opt_out_event, Rulebook business-day calendar.

Tables (RLS locked)

- `reconciliation` (`Reconciliation`): client, `period` (first of month), `measure` (calls, texts, vendor_leads, crm_records, days_present, stop_replies), `source_system`, `method` (bill / tool_total / volume / none), `records` (what CiV read), `independent_total`, `ratio`, `available` (False = no comparison; excluded from M37, never failed), `flagged`, `note`, computed_at. Unique (client, period, measure, source_system).
- `period_integrity` (`PeriodIntegrity`): client, period (unique), `completeness` (M37 = min available ratio; null = nothing to compare), `floor` (default 0.9), `incomplete`, `flagged_by` (list of measures that flagged), `contacts` (primary contacts that month).

Flow (check_month(client, period))

1. floor = Rulebook `completeness_floor` (0.9, proposed). Range = month, clipped to yesterday and to client start date.
2. Delete that month's rows; for each connected/stale tool (sorted): a. **bill**: Twilio only (`BILLED_TOOLS = {'twilio': ('texts','calls')}`): CiV outbound count ÷ billed count. Bill read error → row unavailable with note "Bill not read: <Error>". b. **tool_total**: for each measure in adapter `counts` not done by bill: CiV read (calls/texts from contact_event for that tool; crm_records from crm_creation created in range) ÷ `count_between`. Error → falls back to volume with a note. c. **volume** if no measure done (no adapter or no counts): working days with records ÷ working days, plus sudden drop (last 7 days' daily average < 25% of the prior 28 days' average) → flagged. Main measure: twilio texts, salesforce/hubspot crm_records, else calls. Tool with no adapter → note "CiV cannot read this tool yet", available False. d. ratio < floor → flagged; ratio > 1 → note "Above 100%: listed for review"; total 0 → unavailable.
3. All tools together: `days_present` (working days with any contact ÷ working days; flags under floor; not in M37) and `stop_replies` (keyword opt-outs vs outbound texts; flagged when texts > `stop_check_min_texts` 1000 and zero STOPs; not in M37).
4. M37 = min ratio over calls/texts/vendor_leads/crm_records rows that are available. Save `period_integrity`.
5. `check_recent()` nightly: this month and last month for every client with a connected/stale tool.

Key functions

- `apps/reconcile/check.py:check_month / check_recent / by_bill / by_tool_total / by_volume / civ_read / working_days`
- `apps/reconcile/billing.py:twilio_billed`
- `apps/reconcile/summary.py:completeness_tile / completeness_months / tool_name`
- `apps/reconcile/views.py:completeness_page` (admin)
- `apps/reconcile/api.py:OwnInsuredView / attestation / rows`
- `apps/sync/api.py:records_completeness` (Overview box)

API endpoints

- `GET /api/data-health` (IsClientUser) — `recordsCompleteness`, `completenessFloor`, `completenessStatus`, `completenessNote` (e.g. "CiV received only 82% of the calls compared with what Twilio billed · October 2026").
- `GET /api/insureds/<ref>` (IsClientUser or IsBuyerUser) — client's own page: attestation numbers (last 30 days), exposure reading, signed statements, `completeness`, `reconciliation` rows `{label, against, records, total, ratio, result flag|consistent, source ("Billing API · source grade B" / "Tool's own total · source grade C" / "Plausibility (working days)")}`. Buyer users get a shared company's snapshot instead (Module 15).

Scheduled jobs

`records-completeness` → `apps.reconcile.tasks.check_completeness` daily 02:30 UTC.

Admin panel parts

Client page tile **Records completeness** (overall %, Complete/Incomplete, tools checked, lowest tool) → opens `/civ-admin/clients/client/<id>/completeness/` (template `reconcile/completeness.html`): this and last month, one line per tool+measure (how, numbers, %, OK / Below floor / Review / Not available), plus "All tools" extra checks (days with records, STOP replies) marked "not in %".

Frontend pages fed

Overview KeyNumbers "Records completeness" tile (clicks to /sources) and AttentionBanner ("Records completeness below the floor"); insurer pages (`/insureds`) reconciliation table and completeness.

Billing app note

`apps/billing` does **not** read carrier bills: it is CiV's own **client contract** (`contract` table: price, currency, billing period, payment method, start/renewal dates, auto-renew, notice days, billing contact, notes) shown as the "Contract & billing" admin tab. Carrier/tool billing for reconciliation lives in `apps/reconcile/billing.py` (Twilio only).

6. Upload centre (apps/uploads) — files for sources with no API

Purpose

Lets the client upload files for the six sources that have no API: internal do-not-contact list, DNC check records, reassigned-number (RND) query logs, complaint log, policies/training/contracts, vendor lead files (Module 12, added 2026-10-22). Each file is fingerprinted, kept write-once in private storage (deleted after 4 years; fingerprint and read rows kept) and its rows read into tables, numbers as phone codes only.

Data it receives

Multipart upload from the portal: `source` (the source's display name) + `file`. CSV/TXT for all; PDF/DOC/DOCX also for policies. Max 50 MB.

Sources (Source choices)

`internal_list` Internal do-not-contact list · `policies` Policies, training, contracts · `complaints` Complaint log · `dnc_checks` DNC check records · `rnd_queries` Reassigned-number query logs · `vendor_leads` Vendor lead files.

Tables (RLS locked on uploaded_file, internal_list_entry, dnc_check, rnd_query, complaint, training_record;

vendor_lead locked in vendors)

- `uploaded_file` (`UploadedFile`): client, source, `file` (path `client-uploads/<client_id>/<uuid><ext>` under MEDIA_ROOT; emptied on purge), original_name, sha256, received_at, uploaded_by (ClientUser), rows_read, rows_rejected, purged_at.
- `internal_list_entry`: client, upload, phone_token, added_on.
- `dnc_check`: client, upload, phone_token, checked_on, listed (bool), list_name (≤40; default "national").
- `rnd_query`: client, upload, phone_token, queried_on, result (yes / no / no_data).
- `complaint`: client, upload, phone_token (may be blank), received_on, resolved_on, channel (≤30). Never the words.
- `training_record`: client, upload, agent_ref (sha256 of lowercased agent ID), trained_on.
- `vendor_lead` (apps/vendors `VendorLead`): client, upload, row, lead_ref, phone_token, vendor, sub_id, received_at, price.

read.py column detection

- Column keys normalised: lowercase, non-alphanumerics removed. `pick(row, names)`: exact key match first, then any key **containing** a name; first non-empty value.
- CSV dialect sniffed among `, ; tab |`; UTF-8 with BOM tolerated; blank rows skipped.
- PHONE names: phone, phonenumber, number, telephone, mobile, cell, ani, dnis, tn, msisdn.
- Dates: ISO first, then %Y-%m-%d, %m/%d/%Y, %m/%d/%y, %Y/%m/%d, %d-%b-%Y, %b %d %Y, %Y%m%d. `to_datetime` = ISO with UTC default, or a bare date at midnight UTC.
- YES words: y, yes, true, 1, listed, on list, onlist, dnc, match, matched, found, blocked, do not call. NO words: n, no, false, 0, not listed, clean, clear, ok, not found, nomatch, no match, callable, allowed.
- Per source:
  - internal list: phone required; added_on from added/addedon/date/created/since/optout, else upload day.
  - DNC checks: phone + checked (checked/checkedon/scrubbed/scrubdate/date/run/queried) + listed (listed/onlist/dnc/result/status/match/registry as yes/no) required; list name from list/listname/registryname/ state/source.
  - RND queries: phone + queried date (queried/queriedon/querydate/date/checked) + result (result/response/reassigned/ answer/status: "no data"/nodata/unknown/n/a → no_data; yes word → yes; no word → no).

  - complaints: received date (received/receivedon/date/opened/created/filed) required; resolved (resolved/resolvedon/closed/closedon); channel (channel/source/via/type); phone optional.
  - vendor lead files: phone + vendor (vendor/vendorname/supplier/publisher/affiliate/source/leadsource) + received (received/receivedat/delivered/deliveredat/leaddate/created/createdat/date) required; sub_id, lead_ref, price optional; a certificate column (certificate, certificateurl, trustedform..., jornaya, leadidtoken, universalleadid, consentlink, consenturl) → queued for claiming and matched to the number in memory (`apps/vendors/lead_certificates.link_certificates`), becoming a consent proof labelled from a vendor.
  - policies: a non-CSV document → kept and fingerprinted only (0 rows). A **CSV** under policies is read as a **training log**: agent (agentid/agent/employeeid/employee/user/userid/id) + trained date (trained/trainedon/completed/completion/date).
- A row missing a required value is counted in rows_rejected. Rows bulk-inserted in one transaction (batches of 1000).

Flow (POST /api/uploads)

1. Permission: signed-in client user; must be **Admin or the company's lawyer** (`is_admin` or `is_counsel`), else 403.
2. Validate source name and file; extension allowed and ≤ 50 MB, else 400.
3. Read bytes, sha256, save `UploadedFile` (file stored privately), then `read()` → rows_read/rows_rejected.
4. `after_upload`: every phone token in an internal-list or DNC-check file is queued for consent recheck (`consent.keeping.queue(..., 'new_proof')`).
5. Access log `upload_received` ("<source> · <name> · N rows").
6. Response = the file row.

Effects elsewhere

- Consent engine do-not-call need (`consent/needs.py`): on the internal list from its date, or listed in a DNC check within Rulebook `dnc_check_max_age_days` (31, proposed) → DNC listed; a "not listed" check covers that date.
- Number page list checks (`contacts/api.py:list_checks`) and proof elements 7 (internal list checked) and 8 (pre-contact reassigned check) use these tables.
- Source Registry upload rows turn "current" once any file of that source exists.
- Setup & Readiness checks: "Internal do-not-contact list" and "Documents" (DNC check records + RND query logs).
- Insurer attestation counts contacts to numbers on the internal list.

Purge

`effects.purge_old`: files older than 4 years + 1 day (received_at), not yet purged, with a file → skipped if under a legal hold (`evidence.holds.Holds.covers` by client, date, tokens, vendor keys); otherwise file deleted, `file=''`, `purged_at=now`. Fingerprint and rows stay.

Key functions

- `apps/uploads/api.py:UploadsView (get/post) / row`
- `apps/uploads/read.py:read / pick / rows_of / to_date / to_datetime / yes_no / to_price`
- `apps/uploads/effects.py:after_upload / tokens_of / purge_old`
- `apps/uploads/tasks.py:purge_old_uploads`
- `apps/vendors/lead_certificates.py:link_certificates / claims_for`

API endpoints

- `GET /api/uploads` (any client user) — up to 200 newest files: `{artifactId: up-<id>, source (label), fileName, sha256, receivedAt, rowsRead, rowsRejected}`.
- `POST /api/uploads` (client Admin or company lawyer) — multipart `source` (label text) + `file`.

Scheduled jobs

`purge-old-uploads` → `apps.uploads.tasks.purge_old_uploads` daily 04:55 UTC.

Admin panel parts

None.

Frontend pages fed

Source Registry Upload centre (`frontend/src/screens/sources/UploadCentre.tsx`), number page, consent results, Setup & Readiness, Lead Provenance / Vendor pages (vendor leads).

7. Portal screens (frontend)

Routing: `frontend/src/server/backend.ts` forwards to Django when `BACKEND_URL` is set. Fully real (BACKEND_PATHS) includes `sources`, `data-health`, `uploads`, `contacts`, `evidence`, `can-call`, `provenance`… Partly real (MERGE_PATHS: Django fields overlaid on sample data, reads only) includes `readiness`, `insureds`, `settings`.

Source Registry (/sources, frontend/src/screens/sources/)

- `SourcesScreen.tsx` — `useSources()` → `GET /api/sources`. KPIs: Current (x of total), Stale (names), Not supplied (the `blocks` texts). Card "Sources by evidence tier" (stacked bar tiers 1–4). Card "Sources" table.
- `SourceTable.tsx` — columns Source (+blocks), Tier, Access (+ "You can revoke this at any time" when revocable), Last sync, Status pill (current/stale/not_supplied/revoked), Feeds domains. Sorted stale/revoked first, then not supplied, then current.

- `UploadCentre.tsx` — `useUploads()` → `GET /api/uploads`; form posts `apiUpload('/uploads', FormData{source, file})`; select lists sources with `acceptsUpload`; accept `.csv` (plus .pdf/.doc/.docx for "Policies, training, contracts"); table File, Source, Received (UTC), Fingerprint (truncated sha), Rows read, Rows not read (amber pill). Invalidates all queries after upload.
- No Connect/Allow or revoke action exists on this page.

Data health (Overview)

- `frontend/src/screens/overview/DataHealthCard.tsx` — `useHealth()` → `GET /api/data-health`: Connected sources bar (Live/Stale/Not supplied), Data access level "Level N of 4" + label, Last change received (UTC) with text "read every 2 minutes, hourly catch-up", link to Source Registry.
- `KeyNumbers.tsx` — Records completeness tile (`recordsCompleteness`, `completenessNote`; click → /sources) and conversation review numbers.
- `AttentionBanner.tsx` — "Records completeness below the floor" (when < `completenessFloor`), proof requests waiting.
- `components/shell/Freshness.tsx` — top-bar dot from `GET /api/sources` (amber + "N source stale").

Setup & Readiness (frontend/src/screens/setup/)

- `SetupScreen.tsx` tabs Readiness report / Setup checklist, `useReadiness()` → `GET /api/readiness` (merged with sample).
- Backend `apps/onboarding/readiness.py:readiness` (IsClientUser): checks by name — Dialer connection, SMS connection, CRM connection (missing / partial "Not read recently" / ready "Connected and read by API (tools)"; dialer partial when dial mode missing on X% of calls), Consent proof (up to 100 recent numbers), Certificate retention, Consent pages, Lead records (traced), Internal do-not-contact list (uploaded?), History depth (≥4 years ready), Documents (DNC check records + RND logs uploaded); `canStart` = dialer or SMS not missing; `numbersPerMonth` (distinct outbound primary numbers, 30 days); `recommendedPlan` (Starter ≤10k, Growth ≤50k, Scale ≤250k, else Enterprise); `preview` of strict consent statuses; `setup` rows (caller-ids, pages, vendors, campaigns "all treated as marketing", states). `notMeasured` and other fields come from the sample.
- `ReadinessTab.tsx` KPIs: Can the audit start?, Checks ready x of y, Recommended plan, Would be not measured (sample); preview stacked bar; checks table. `ChecklistTab.tsx` setup checklist meter.

Lead Provenance uploads card

`frontend/src/screens/leads/UploadsCard.tsx` — "Uploaded lists" from `GET /api/provenance` (import batches: numbers, with evidence, STOP numbers brought back, opt-out marks cleared, proof rate).

8. Rulebook keys used by intake (rulebook/v3.0.yaml)

clock_skew_seconds 120 (set) · dedupe_window_seconds 120 (set) · source_freshness_hours 2 (set) · poll_interval_minutes 2 / catchup_interval_minutes 60 / full_sweep_interval_days 7 (set; informational — schedule is hard-coded) · import_gap_seconds 120 (proposed) · import_min_batch 20 (proposed) · completeness_floor 0.9 (proposed) · stop_check_min_texts 1000 (proposed) · dnc_check_max_age_days 31 (proposed).

9. Privacy rules applied across intake (from code)

- Raw record bytes are hashed (sha256 in `fingerprint`) and dropped; content never stored.
- Phone numbers → E.164 → HMAC-SHA256 token with a per-client key (locked with CIV_SECRETS_KEYS); only a masked label (area code + last 4) kept on contacts. Typed search numbers tokenized, never stored.
- Error messages stored as type names only (`last_error`, `purge_log.reason`).
- CRM data kept as user IDs, times, fingerprints of watched fields; vendor name/sub-id/price kept as plain text.
- Agent IDs in training logs are hashed.
- Uploaded files are the exception: kept privately for 4 years (owner decision).
- All client-data tables are Postgres row-level-security locked by `civ.client_id` / `civ.scope` (`tenancy/sql.py`).

5. Consent, certificates, page capture, lead provenance and proof requests (Modules 6–8)

Read from the code on 22 October 2026. Backend = `/home/shakib/Audit/backend` (paths below are relative to it unless they start with `frontend/`).

Shared facts that apply to every area below:

- All client tables in these apps are **locked per client** (row-level security) by migrations calling `apps.tenancy.sql.lock_table(...)`: `consent_proof`, `consent_need`, `consent_result`, `consent_result_change`, `consent_recheck`, `certificate_claim`, `consent_page`, `page_capture`, `proof_request`, `provenance`. Background jobs open them with `apps.tenancy.scope.all_clients()` or `tenant(client)`.
- Phone numbers are never stored. Everything is keyed by `phone_token` (the client-keyed phone code from Module 1) and shown as `masked_label` (area code + last 4).
- Client API permission everywhere here is `apps.accounts.permissions.IsClientUser` (any signed-in client user, own company only via `request.user.client`). Extra rule: only Admins or the company lawyer (`is_counsel`) can answer proof requests.
- All client URLs are mounted under `/api/` (`config/urls.py`: `include('apps.consent.urls')`, `include('apps.captures.urls')`, `include('apps.provenance.urls')`). The certificates app has no URLs of its own; its list is served by the consent app (`/api/certificates`).
- The Rulebook (`rulebook/v3.0.yaml`) supplies the numbers; many are drafts (`status: proposed / tbd`, `counsel_approved: false`). Code passes a default when a key has no value.

1. Certificates (apps/certificates) — finding and claiming TrustedForm / Jornaya certificates

Purpose (plain words)

Leads often carry a "consent certificate" link from TrustedForm or a Jornaya LeadiD token: a third-party recording that the person filled in a form. Those certificates disappear after about 72 hours unless someone "claims" (retains) them. This app spots every certificate in every synced record and claims it automatically, with the client's own certificate-account key, before the deadline. It also asks TrustedForm whether the phone number in the lead was the one typed into the form ("phone match"). Owner decision 2026-10-21: claiming is automatic, no on/off switch (covered by the service agreement).

Data it fetches / receives, and from where

- **Receives** every synced record from the live sync engine (Module 3) through the hand-off list `apps.sync.handoff.RECEIVERS` (registered in `apps/certificates/apps.py: ready`). Each record has `raw` bytes, `content_kind` (lead / crm_record / call / text...), `source_record_id`, `changed_at`, and (in memory only) `phones`.
- **Fetches** from the providers with the client's key (from `apps.connections.models.Connection` with `provider='trustedform'` or `'jornaya'`, status CONNECTED / STALE / UNTESTED):

- TrustedForm: `POST https://cert.trustedform.com/<40-hex id>` with Basic auth `API:<key>`, header `api-version: 4.0`, body `{"retain": {"reference": "civ-client-<id>"}, "insights": {"properties": ["created_at","expires_at","page_url"]}, "match_lead": {"phone": "<digits, no leading 1>"}}` (match_lead only when a phone is in hand). Built from public docs. Lead-matching shape is **marked "to confirm"** on a real account.
- Jornaya: `GET https://api.leadid.com/SingleQuery?lac=<account>&lak=<key>&id=<token>` — **marked "to confirm"** (Jornaya docs are customer-only). No phone match for Jornaya.
- Answers are read "loosely" (several possible JSON paths tried). HTTP 401/403 → `Refused`; 404/410 → `NotFound`; 5xx → retried.

Tables

- `certificate_claim` (model `CertificateClaim`): `client`, `provider` (trustedform / jornaya), `cert_ref` (cert ID or token; evidence ID, not personal data), `seen_at` (first found), `deadline_at` (record time + 72 h), `state` (`waiting` / `claimed` / `no_key` / `lost`), `attempts`, `last_error` (error type name only, e.g. `NotFound`), `deadline_alerted`, `claimed_at`, facts read: `cert_created_at`, `cert_expires_at`, `page` (host + path only, never the query string), `gone_at` (no longer opens: expiry passed), `phone_match` (JSON `{phone_token: true/false}`). Unique: (client, provider, cert_ref). Index (state, deadline_at).

Flow step by step

1. A record is synced. `scan.scan_record` runs `found_in(raw)`:
   - TrustedForm regex `https?://cert.trustedform.com/([0-9a-f]{40})`.
   - Jornaya regex: a UUID only when it sits in a field named `leadid_token`, `leadid`, `lead_id_token`, `universal_leadid` or `jornaya*` (never any UUID).
2. Each certificate is created once per client with `deadline_at = record.changed_at + cert_unretained_expiry_hours` (Rulebook, value 72; fallback constant 72 if no Rulebook). Duplicates reuse the existing claim.
3. If the record is a lead / CRM record (not a HubSpot `calls:` record), `match_phones` runs **immediately while the numbers are in memory**: each phone of the record is normalised, turned into its token, and `claim.match_now` is called for every claim in that record.
   - TrustedForm only; skipped if that token already has an answer.
   - If the claim is still waiting / no_key → it is claimed right now, with the phone (claim + match in one request).
   - If already claimed → a match-only request (`match_trustedform`). Provider down → stays "not measured", never retried (the number is gone from memory).
4. Every minute (`claim_certificates` task) `claim.claim_due` takes up to 200 claims in `waiting` or `no_key`, nearest deadline first, and calls `claim_one` under that client's tenant:
   - Past the deadline → `lost`, access-log event `certificate_lost`.
   - No usable connection → `no_key` (stays queued; a key added before the deadline still works).

    - Calls the provider. `NotFound` → `lost` with `last_error='NotFound'` (logged). Any other error → `last_error` set, retried next minute until the deadline.
    - Success → the provider's raw answer is written to the fingerprint ledger (`ledger_job`, content kind CERTIFICATE, **source grade B**) and then dropped; claim set to `claimed` with created / expires dates and page host+path; `note_phone` stores the phone answer.
5. `warn_close_deadlines`: any waiting / no_key claim within 12 h of its deadline → access-log event `certificate_deadline_close` once (`deadline_alerted=True`).
6. `note_phone` calls every receiver in `claim.PHONE_CHECKED` (the consent app registers `keeping.phone_checked`) → recheck of that number's results.
7. Nightly (consent app, `mark_gone`): claimed certificates whose `cert_expires_at` has passed get `gone_at` set (no provider request; uses the expiry date given at claim time) and their numbers are queued for recheck.

Logic and rules

- Deadline window = Rulebook `cert_unretained_expiry_hours` (status `set`, 72).
- Alert 12 h before the deadline (`ALERT_BEFORE`). Up to 200 claims per minute (`PER_RUN`).
- Phone match is TrustedForm-only; Jornaya lead matching "to confirm".
- Custody (`apps/consent/sources.py: custody_of`): claimed → "Claimed in the client's account"; lost with NotFound → "Provider does not have it"; lost → "Not claimed in time"; waiting → "Being claimed"; no_key → "Waiting for the certificate account". A vendor-only holder cannot be seen via API → never guessed.
- Retention: CiV claims in the client's own account; how long the provider keeps it depends on the client's account (expiry comes back from the provider). Results warn when it expires within `cert_expiry_warning_days` (30).
- After expiry the certificate no longer opens, but CiV holds the sealed fingerprint of what it read → `WK_FINGERPRINT_ONLY` (weak) instead of conflicting.
- Test connection (README): asks about a made-up certificate; "not found" = key works, "refused" = bad key (code in the connections app; not read here).

Key functions/classes

- `apps/certificates/models.py:CertificateClaim` — one certificate to claim, and facts read from it.
- `apps/certificates/scan.py:found_in` — finds TrustedForm links / Jornaya tokens in raw text.
- `apps/certificates/scan.py:scan_record` — hand-off receiver; queues claims; triggers phone match.
- `apps/certificates/scan.py:match_phones` — phones of a lead vs each certificate, in memory.
- `apps/certificates/scan.py:window_hours` — reads `cert_unretained_expiry_hours`.
- `apps/certificates/claim.py:claim_one` — claim/read one certificate, set state, ledger fingerprint.
- `apps/certificates/claim.py:match_now` — phone match now (claim with phone or match-only).
- `apps/certificates/claim.py:note_phone` — store phone answer, notify `PHONE_CHECKED`.

- `apps/certificates/claim.py:claim_due` — the urgent queue, nearest deadline first.
- `apps/certificates/claim.py:warn_close_deadlines` — 12 h warning log.
- `apps/certificates/vendors.py:claim_trustedform / match_trustedform / read_jornaya / host_and_path` — provider calls.
- `apps/consent/keeping.py:mark_gone` — nightly "no longer opens".

API endpoints

- `GET /api/certificates` (consent app, `CertificatesView`, IsClientUser): up to 200 claims, newest first: `id` (`cc-<pk>`), provider, `ref` (first 8 chars + "..."), state + label, seenAt, deadlineAt, createdAt, expiresAt, page, `usedBy` (distinct strict results that used it), `problem` (last_error when lost), `source` (label from the first proof's source), `phoneMatch` (null if no answer, else all answers true).

Scheduled jobs

- `claim-certificates` → `apps.certificates.tasks.claim_certificates`, every 60 s.
- (`consent-nightly` 03:00 UTC runs `mark_gone` — see consent.)

Admin panel parts

- No admin.py in this app. Admin client page has a **Certificates** tile (`apps/clients/admin.py: certificates_tile`): claimed count plus waiting / no key / lost counts.
- Onboarding readiness (`apps/onboarding/readiness.py`) has "Certificate retention" check (lost count, certificates kept less than a year) and "Consent pages" check.
- Alerts (`apps/alerts/raise_.py`) count claims expiring within 30 days.

Frontend pages fed

- Consent Integrity → Summary tab → `CertificatesCard` (`frontend/src/screens/consent/CertificatesCard.tsx`, hook `useCertificates` → `/certificates`).
- Consent Integrity → Certificate expiry tab (`ExpiryTab`, uses `/consent expiring`).
- Evidence file → Consent card proofs (provider, state, expiry, phone match, page).

2. Consent engine (apps/consent) — Module 6

Purpose (plain words)

For every outgoing call or text, decide: did the client have permission on record, and how strong is that record? It (a) collects "proofs of permission" from synced records, (b) works out what proof each contact needed under the rules on that date, (c) runs five checks and gives a status, and (d) keeps results current when new facts arrive, without silently rewriting history.

Data it fetches / receives, and from where

- Synced records via hand-off receivers (`apps/consent/apps.py: ready`):

    - `handoff.RECEIVERS`: `proofs.proofs_seen` (collect proofs) and `keeping.contact_seen` (freeze on changed facts). Order: after the certificate scanner and after the contact normalizer.
    - `handoff.SEEN`: `keeping.records_seen` (weekly full check found these records).
    - `handoff.SWEEP_DONE`: `keeping.sweep_done` (weekly full check finished → missing records are deleted).
- Signals: `certificates.claim.PHONE_CHECKED` → `keeping.phone_checked`; `captures.capture.CAPTURED` → `keeping.page_captured`; `post_save(OptOutEvent)` → `keeping.optout_saved`; `post_save(PhoneEpoch)` → `keeping.epoch_saved`; `apps.rulebook.signals.rules_changed` → `keeping.rules_changed`.
- Reads: `contacts.ContactEvent` (the calls/texts), `contacts.PhoneEpoch` (who owned the number when), `optouts.OptOutEvent` (STOPs), `uploads.InternalListEntry` / `uploads.DncCheck` (client's DNC lists), `imports.CrmCreation` / `ImportBatch` (how a record was created, upload batches), `certificates.CertificateClaim`, `captures.PageCapture` (wording checklist), the Rulebook (`RuleSet`).
- Module 8 evidence (`apps/provenance/evidence.py: found`) and proof requests (`apps/provenance/requests.py`) also create `ConsentProof` rows.

Tables

- `consent_proof` (`ConsentProof`): `client`, `phone_token`, `epoch` (owner), `kind`, `proof_type` (Rulebook word: web_cert, crm_flag, purchase, inbound_call, client_unchecked, signed_form…), `certificate` (FK claim), `captured_at` (consent time), `date_known` (False = record time standing in until the certificate is read), `source_system`, `source_record_id`, `lead_id`, `proof_request` (FK), `source_hint` (how the tool says the record was made: FORM / IMPORT / Web…), `source_seen_at`, `withdrawn_at`, created/updated. Unique: (client, source_system, source_record_id, phone_token, kind, certificate), nulls not distinct.
    - **ConsentProof kinds**: `certificate` (TrustedForm/Jornaya), `self_asserted` (CRM consent field only — never enough alone), `ebr_purchase` (existing customer: purchase), `ebr_inquiry` (existing customer: inquiry), `inbound_call` (person called first), `inbound_sms` (person texted YES), `form_submission` (client's own form, no certificate), `client_supplied` (file attached to a proof request).
- `consent_need` (`ConsentNeed`): one row per contact **and variant** (`strict` / `lenient`): `category` (always marketing for outbound), `robocall`, `bases` JSON `[{basis, needs:[proof types] or ['any']}]`, `labels`, `is_legal_basis`, `rulebook_version`, `evaluated_at`. Unique (contact, variant).
- `consent_result` (`ConsentResult`): one row per contact and variant: `status`, `primary_code`, `codes`, `bases` JSON `[{basis, status, codes, proof}]`, `proof` (winning proof FK), `labels`, `is_legal_basis`, `evaluation_as_of`, `rulebook_version`, `inputs` (contact facts when checked), `reeval_reason`, `frozen`, `frozen_reason`, `frozen_at`, `source_seen_at`, `evaluated_at`. Unique (contact, variant). Index (client, variant, status).
- `consent_result_change` (`ResultChange`): history: old/new status, old/new code, `reason`, `at`.
- `consent_recheck` (`Recheck`): queue: `client` (null = all), `phone_token` (blank = all numbers), `from_at`, `to_at`, `reason`, `after_contact` (resume point).

Statuses

`verified` · `weak` · `pending` (certificate not read yet; left out of metrics) · `conflicting` · `no_proof` · `not_needed` (inbound, or CiV test numbers). Portal shows them upper-case; a contact with no result yet shows `NOT_CHECKED`.

Variants (strict vs lenient)

Every contact is evaluated twice, once per variant, because many Rulebook values are undecided and carry `variants: {strict, lenient}`. **Portal, metrics, alerts and scoring use strict only.** Examples: `optout_grace_business_days` strict 0 / lenient 10; unknown dial mode counts as autodialed except where `autodialed_modes` is `none` (lenient).

Reason codes (apps/consent/words.py: REASONS)

- `VF_OK` — all checks CiV can run passed.
- Check 1 (no fitting proof → `no_proof`): `NP_NONE`, `NP_SELF_ASSERTED`, `NP_NO_EVIDENCE_FOUND` (number from an uploaded list with no evidence), `NP_BEFORE_CONSENT`, `NP_AFTER_OPT_OUT` (person said STOP before this contact), `NP_RELATIONSHIP_LAPSED`, `NP_WRONG_TYPE`. Order of priority = `CHECK1_ORDER`.
- Check 2/4 (`conflicting`): `CF_NOT_FOUND`, `CF_EXPIRED`, `CF_PHONE_MISMATCH`, `CF_CERT_AFTER_DELIVERY`.
- Check 3 (`no_proof`): `NP_NO_CONSENT_TEXT`, `NP_PRETICKED` (from Module 7 checklist).
- Weak: `WK_CLIENT_PROOF_UNCHECKED`, `WK_WORDING`, `WK_TIMING_UNCERTAIN`, `WK_RETENTION`, `WK_FINGERPRINT_ONLY`.
- Pending: `PD_NOT_READ_YET`, `PD_NO_CERT_ACCOUNT`.
- Frozen reasons: `FZ_SOURCE_DELETED`, `FZ_INPUTS_CHANGED`.
- Labels: `EBR_ONLY`, `pre_engagement`, `category_unknown`, `dial_mode_unknown`, `dnc_list_not_supplied`, `no_consent_needed`, `warn:CC_ABOVE_BUTTON` / `warn:CC_NO_SCROLL`, `draft:consent_checklist`, and `not_measured:*` (`consent_wording`, `ai_wording`, `phone_match`, `page_match`, `lead_trace`, `seller_name`).

Flow step by step

A. Collecting proofs (`proofs.proofs_seen`, per synced record):

1. Parse `record.raw` as JSON; get the source hint (`sources.hint_from_record`: HubSpot `hs_object_source`, Salesforce `LeadSource`) and `lead_id` (`lead_id` / `Id` / `id`).
2. For every certificate found in the record that already has a claim → a `certificate` proof (proof_type `web_cert`) per phone token in the record; date = certificate creation time if claimed, else the record's `changed_at` with `date_known=False`.
3. Lead / CRM records (not `calls:`): a field whose name contains "consent" (not "opt", not "revok") set to a yes value (`true, yes, y, 1, on, consented, opted in, opted_in`) → `self_asserted` (`crm_flag`); its date stays the date CiV first saw it. HubSpot `hs_lifecyclestage_customer_date` → `ebr_purchase` (`purchase`).
4. Module 8 evidence in the record (`provenance.evidence.found`) → `inbound_call` / `inbound_sms` / `form_submission`.

5. `_save` puts each proof on the owner (epoch) of the number **on the consent date**; a new proof queues a recheck from its date minus 2 minutes; a changed date on an existing proof freezes results that used it (`FZ_INPUTS_CHANGED`) and queues a recheck.
6. Proofs from this record that were not seen this time (link removed, box unticked, number changed) are **withdrawn** and results that used them frozen (`FZ_INPUTS_CHANGED`).
7. Every 5 min `refresh_certificate_dates`: proofs with `date_known=False` whose claim now has `cert_created_at` take the real date and the right epoch, then recheck.

B. What a contact needs (`needs.work_out` / `profile`, per variant, rules on the contact's date):

1. Inbound or test number → no bases, `not_needed`.
2. Category = **marketing** for every outbound contact (no campaign map; label `category_unknown`).
3. Robocall? prerecorded/AI voice → yes; text → `sms_treated_as_autodialed` (default True); call → `autodialed_modes` (default `every_mode_except_manual`); missing dial mode → label `dial_mode_unknown`, counts as autodialed unless modes = `none`.
4. Bases:
   - `robocall_marketing` when robocall → needs `written_consent_types` (if `written_consent_required_by_forum`, default True; value `web_cert, signed_form, ivr_keypress_with_disclosures`) else `prior_express_evidence_types` (tbd, not set).
   - `dnc_listed` when a STOP is in force (`opt_out_in_force`) or the number is on the client's internal list / a DNC check said listed → needs `signed_written_consent_types` + `ebr_active`.
   - `dnc_unknown` when no DNC check covers the date (within `dnc_check_max_age_days`, default 31) → needs `any`; label `dnc_list_not_supplied`.
   - `state_law` per matching `state_consent_rule` row for the recipient state and channel → its `consent_type_required` (default `any`).
   - `civ_policy` (needs `any`) when none of the legal bases or dnc_unknown applies.
   - `is_legal_basis` = any of robocall_marketing / robocall_informational / dnc_listed / state_law.
5. Saved to `consent_need` (strict + lenient).

C. The five checks (`checks.evaluate`, per need/variant; time checks use `evaluation_as_of`, not the clock): For each basis: run check 1 on every non-withdrawn proof of the owner's evidence folder; judge the survivors with checks 2–5; keep the best proof (status rank, then newest). The contact's status = the **worst** basis.

1. **A fitting proof** (`check_one`): self-asserted → `NP_SELF_ASSERTED`; unchecked client file → only blocked by an opt-out; timing: proof after the contact → `NP_BEFORE_CONSENT` (linked by same `lead_id`: no tolerance; unlinked: `clock_skew_seconds` 120 s, inside tolerance = "uncertain"); an opt-out in force (calculated with `newer_consent_lifts=False`) and the proof is EBR or dated before the STOP → `NP_AFTER_OPT_OUT`; EBR out of window → `NP_RELATIONSHIP_LAPSED`; otherwise `meets()` false → `NP_WRONG_TYPE`. No survivors → `no_proof` with codes in CHECK1_ORDER (`NP_NONE` when no proofs; plus `NP_NO_EVIDENCE_FOUND` when the number came from an uploaded list and only a CRM box backs it).

2. **Opens at its source** (certificates): lost + NotFound → `CF_NOT_FOUND` (conflicting); lost → `CF_EXPIRED` (conflicting; label `pre_engagement` if the deadline was before the client's `start_date`); not claimed → `pending` (`PD_NO_CERT_ACCOUNT` when no_key, else `PD_NOT_READ_YET`); claimed but gone / expired by as_of → `WK_FINGERPRINT_ONLY` (weak).
3. **The page really asks for permission** (Module 7 checklist from CiV's own capture within `page_snapshot_max_gap_days`=1 of the consent): `NP_NO_CONSENT_TEXT` / `NP_PRETICKED` → no_proof. No capture close enough → not measured, never worse.
4. **Sources agree**: provider says the number was not typed into the form → `CF_PHONE_MISMATCH`; certificate made after the lead was delivered (CRM creation time) by more than the tolerance → `CF_CERT_AFTER_DELIVERY`; within tolerance → uncertain.
5. **Complete**: `WK_TIMING_UNCERTAIN`, `WK_RETENTION` (no expiry date, or expires within `cert_expiry_warning_days` 30), `WK_WORDING` (button name, number mentioned, font size, contrast). None → `verified` / `VF_OK`.

- An unchecked client file (`proof_type='client_unchecked'`) is always `weak` / `WK_CLIENT_PROOF_UNCHECKED`.
- Labels: `EBR_ONLY` only if every winning proof is an existing-customer one. `not_measured:phone_match` removed when the provider answered for every certificate used. When wording was measured: removes `not_measured:consent_wording` and `not_measured:seller_name`, adds `not_measured:ai_wording` and `draft:consent_checklist`.

D. `meets()` — which proof kind can satisfy which basis (`needs.py`):

- self_asserted: never.
- client_unchecked file: always (but weak).
- EBR (purchase / inquiry): never for `robocall_marketing`; only for a **live, hand-dialed call** when `ebr_allowed_contact_types` includes `live_manual_call`; only inside the window (`ebr_purchase_months` 18 / `ebr_inquiry_months` 3, both proposed); and only where the basis needs `any` or `ebr_active`.
- inbound_call / inbound_sms / form_submission (`evidence_meets`): only where the basis needs `any` or lists the proof type (they are never written consent, so never a robocall basis); inbound_call covers live, non-robocall calls for 30 days after it (`INBOUND_DAYS`, draft, counsel B12); inbound_sms covers texts; form_submission only when `source_of(proof)` = client_form.
- Everything else (certificate, validated client proof): `any` in needs or its `proof_type` listed.

E. Opt-out in force (`needs.opt_out_in_force`): latest counted (`counts=True`), not-lifted opt-out before the contact. Not in force for the one confirmation text within `confirmation_window_minutes` (5); not before `optout_grace_business_days` business days end (strict 0, lenient 10; local time zone of the contact); for the "STOP list" basis a newer certificate consent after the STOP lifts it. A purchase or CRM box never lifts it. In the five checks the newer-consent lift is turned off and handled per proof (a proof dated after the STOP survives).

F. Evaluation timing

- New contacts: every minute `work_out_needs` takes up to 2,000 primary contacts with no need yet (oldest first), works out needs and runs the five checks right away → each call's result within minutes.
- Rechecks: every minute `keep_results_up_to_date` = `run_rechecks` (up to 20 queue rows, 1,000 contacts per minute, resumable via `after_contact`) + `recheck_pending` (pending results whose certificate is no longer waiting/no_key).
- Nightly 03:00 UTC `recheck_time_passed`: `mark_gone`, then verified results whose certificate expires within the warning days (up to 5,000 contacts) → `time_passed`.
- Every 5 min `refresh_proof_dates`.

G. Keeping results current (`keeping.py`) — what queues a recheck (reason):

- new proof / proof date changed → `new_proof` / `certificate_read` (from proof date − 2 min);
- phone match answer → `new_proof` (whole number);
- new page capture → `page_captured` for numbers whose certificate from that page has a consent time within ±1 day;
- new opt-out → `late_opt_out` (from the opt-out time);
- new owner epoch → proofs re-assigned to the right owner by their date, then `owner_changed` from the split;
- Rulebook change signal → `rule_changed` for all clients from the change date;
- proof request answered / reviewed → `new_proof`.
- A recheck re-runs `work_out` (needs can change) then `check_contact`. Frozen results are never recomputed.
- History: `note_change` writes `consent_result_change` only when status or primary code changed; the first result is the row itself. `CHANGE_REASONS` turns reasons into words ("certificate read", "STOP received"...).
- Freezing: a contact's record comes back with different facts (`phone_token, occurred_at, channel, direction, dial_mode, lead_id`) → `FZ_INPUTS_CHANGED`. Weekly full sweep (`records_seen` stamps `source_seen_at`; `sweep_done`) → results and proofs whose record was not found → `FZ_SOURCE_DELETED` (proofs withdrawn). Frozen results stay visible but are left out of totals.

Key functions/classes

- `apps/consent/models.py:ConsentProof / ConsentNeed / ConsentResult / ResultChange / Recheck` — tables above.
- `apps/consent/proofs.py:proofs_seen` — collects proofs from each synced record.
- `apps/consent/proofs.py:_save` — saves a proof on the right owner; queues recheck / freezes on date change.
- `apps/consent/proofs.py:refresh_certificate_dates` — real certificate dates onto proofs.
- `apps/consent/proofs.py:evidence_folder` — owner's proofs + upload batch + `manual_import_no_evidence`.

- `apps/consent/proofs.py:consent_fields` — CRM "consent = yes" detector.
- `apps/consent/needs.py:profile / work_out / work_out_new` — what each contact needs.
- `apps/consent/needs.py:opt_out_in_force / dnc_lists / is_robocall / ebr_active / meets / evidence_meets`.
- `apps/consent/checks.py:check_one / judge / evaluate / check_contact / wording_for` — the five checks.
- `apps/consent/keeping.py:queue / run_rechecks / recheck_pending / recheck_time_passed / mark_gone / freeze / withdraw / contact_seen / records_seen / sweep_done / note_change` — keeping current.
- `apps/consent/sources.py:source_of / hint_from_record / custody_of` — where a proof came from.
- `apps/consent/words.py:REASONS / check_states / reason_text / proof_text / label_words / portal_status`.
- `apps/consent/panel.py:consent_panel / worst_of / optouts_of` — consent block of the number page.
- `apps/consent/api.py:summary / ConsentView / CertificatesView` — client API.

Source labels (sources.py, owner decision 2026-10-21)

`client_form` ("Client's own form"), `vendor` ("From a vendor"), `uploaded_list` ("From a vendor (uploaded list)"), `unknown`. Rules: `vendor_lead_file` source → vendor; HubSpot FORM/FORMS → client_form, IMPORT → uploaded_list, INTEGRATION/API → vendor; else by the record's Module 5 batch (bulk/small → uploaded_list, feed → vendor); Salesforce LeadSource containing "web" and not in a batch → client_form; else unknown. The label never skips a check.

The five checks in the portal (words.check_states)

Each contact row gets five states: `pass` · `fails` · `gap` · `not_reached` · `not_measured`. E.g. `NP_*` → `[fails, not_reached×4]`; `CF_NOT_FOUND/EXPIRED` → `[pass, fails, …]`; `PD_*` → `[pass, gap, …]`; `CF_PHONE_MISMATCH` / `CF_CERT_AFTER_DELIVERY` → `[pass, pass, not_measured, fails, not_reached]`; `NP_NO_CONSENT_TEXT/PRETICKED` → `[pass, pass, fails, …]`; check 3 shown as `pass` when wording was measured and passed.

API endpoints

- `GET /api/consent` (`ConsentView`, IsClientUser) → this month's strict summary: `contacts` (counted), `mix` (VERIFIED / WEAK / CONFLICTING / NO_PROOF count + share + note), top 8 non-VF_OK `reasons` with text, `legalShare`, `legalCount`, `policyCount`, `dncUnknownShare`, `certificateValidity` (% of contact–certificate pairs without a `CF_` code), `certificatePairs`, `expiredBeforeEngagement`, 4-month `trend`, `expiring` within 7/30/90 days (claimed certificates), `noProofLegal`, `pending`, `frozen`, `notNeeded`, `period`, `real: true`. Pending, frozen and not_needed are counted apart and left out of every share.
- `GET /api/certificates` — see Certificates.

- Served by the contacts app but built from consent code: `GET /api/contacts` (Contact Ledger rows: status, code, reasonText, five `checks`, `bases`, `proof`, flags; `?status=` incl. `NOT_CHECKED`), `GET /api/contacts/<ref>`, `GET /api/evidence/<ref>` (number page: `worstStatus`, counts, `consent` block from `consent_panel`: owner, proofs with kind/provider/madeAt/dateKnown/expiresAt/state/tool/withdrawn/usedBy/source/phoneMatch/page, latest contact's `needs`, last 20 `history` changes, frozen/pending counts, `notMeasured` texts, `draftRules`).

Scheduled jobs (config/settings.py CELERY_BEAT_SCHEDULE)

- `consent-needs` → `apps.consent.tasks.work_out_needs`, every 60 s.
- `consent-recheck` → `apps.consent.tasks.keep_results_up_to_date`, every 60 s.
- `consent-nightly` → `apps.consent.tasks.recheck_time_passed`, 03:00 UTC daily.
- `proof-dates` → `apps.consent.tasks.refresh_proof_dates`, every 300 s.

Admin panel parts

- No admin.py in the consent app (no direct admin screens for proofs/results found). Onboarding readiness shows a preview of consent results on recent numbers (`apps/onboarding/readiness.py`).

Downstream users (read-only use of consent results)

- `apps/metrics/overview.py` — M01 consent coverage, M10 contacts after opt-out (`codes contains NP_AFTER_OPT_OUT`).
- `apps/alerts/raise_.py` — "Contacts after opt-out" high alert (strict, not frozen, `NP_AFTER_OPT_OUT`) and certificates expiring within 30 days.
- `apps/scoring/exposure.py` — strict, not frozen, legal-basis results.
- Also referenced from `metrics/who.py`, `metrics/conduct.py`, `findings/examples.py`, `vendors/rates.py`, `vendors/disputes.py`, `buyers/attestation.py`, `reconcile/api.py`, `anchoring/position.py` (details in those slices).

Frontend pages fed

- **Consent Integrity** (`frontend/src/screens/consent/ConsentScreen.tsx`), tabs Summary / Certificate expiry / Wording catalogue. Summary: `StatusMixCard`, `StatusTrendCard`, `ReasonCodes`, `LegalBasisCard` (all `/consent`), `CertificatesCard` (`/certificates`), `FiveChecks` (static explainer text). Expiry: `ExpiryTab` (`/consent expiring`). Wording: `WordingTab` (`/consent-pages`).
- **Contact Ledger** (`frontend/src/screens/ledger/LedgerScreen.tsx`, `/contacts` + `/consent`).
- **Evidence file / number page** (`frontend/src/screens/evidence/EvidenceFileScreen.tsx`, `/evidence/<ref>`): Standing card (worst status), `ConsentCard` (consent block), `ContactTimeline`, `ProofElements`, "Came from" line.
- Frontend proxy: `consent`, `certificates`, `consent-pages`, `provenance`, `proof-requests`, `evidence`, `contacts` are full backend paths (`frontend/src/server/backend.ts BACKEND_PATHS`); `leads` and `vendors` are MERGE paths (real fields over sample fields).

3. Page capture (apps/captures) — Module 7 steps 3, 4, 6

Purpose (plain words)

CiV takes its own "photos" of the public consent form pages named in the client's certificates, about once a day, at phone and computer size. It keeps the screenshot, the page words, the consent paragraph, and measurements (checkboxes, buttons, font size, colours, positions). A rule-based checklist reads these measurements to decide if the page really asks for permission. Owner decision D32a: CiV keeps screenshot, text, and measurements.

Data it fetches / receives, and from where

- Page addresses from `certificate_claim.page` (host + path only, from claimed certificates).
- Opens each page with its own Chromium browser via **Playwright** (`apps/captures/browser.py`), in a separate container `capture` (`deploy/Dockerfile.capture`, Celery queue `capture`, `CHROMIUM_PATH` env; empty = Playwright's own browser). The main worker never opens a browser (`CELERY_TASK_ROUTES`).

Tables

- `consent_page` (`ConsentPage`): `client`, `address` (host+path), `first_seen_at`, `last_seen_at` (latest certificate naming it), `last_captured_at`, `client_mark` (`approved` / `unmarked` / `rejected`, default unmarked; the client's own note, never changes a result), `created_at`. Unique (client, address).
- `page_capture` (`PageCapture`): `client`, `page`, `captured_at`, `viewport` (`390x844` phone / `1440x900` computer), `ok`, `problem` (why not captured; never held against the client), `screenshot` (private file at `captures/<client>/<page>/<YYYYmmdd-HHMMSS>-<viewport>.png`), `screenshot_sha256`, `text`, `text_sha256`, `consent_block`, `consent_block_sha256`, `measurements` JSON, `fingerprint_sha256` (ledger fingerprint), `purged_at`. Index (client, page, captured_at).

Flow step by step

1. Hourly at :20 (`capture_pages`, queue `capture`): `pages_from_certificates` turns every distinct claim page into a `ConsentPage` (or updates `last_seen_at`).
2. `due()`: pages named in the last 30 days (`ACTIVE_DAYS`) not captured within `page_capture_interval_hours` (24), oldest capture first; up to 25 pages per run (`PER_RUN`).
3. For each page and each viewport in `cc_viewports` (default `[[390,844],[1440,900]]`) `Browser.capture`:
    - Safety: opens only `http/https` on ports 80/443 whose host resolves only to public (global) IPs; every request the page makes is checked the same way and aborted otherwise; a redirect to a non-public address keeps nothing; service workers blocked; phone viewport uses `is_mobile` + touch; user agent `CiV-PageCapture/1.0`. Never types into or submits a form.
    - Loads (timeout 30 s, wait for `load`), HTTP ≥400 → not ok; waits 1.5 s for late scripts; full-page PNG screenshot, body text (max 200,000 chars), runs `MEASURE_JS`.
    - `MEASURE_JS` returns: title, viewport, pageHeight, number of forms, `phoneFields` count, `checkboxes` (checked, visible, label, position), visible `buttons` (text, position), `consentBlock` (the visible element of 40–3,000 chars with the most permission words — consent, agree, authorize, autodial,

prerecorded, TCPA, "calls and texts", etc., at least 2 hits — with its text, position, fontSize, colour, background).

4. Each capture is saved with SHA-256 hashes; if ok the screenshot is stored and a JSON of the facts (url, viewport, time, hashes, measurements) is written to the fingerprint ledger (`civ_capture`, content kind PAGE_CAPTURE, **source grade A**), sealed with the nightly seal.
5. `last_captured_at` set. If any viewport was ok → `CAPTURED` receivers fire (`consent.keeping.page_captured` queues rechecks of numbers whose certificate from that page has a consent time within ±1 day).
6. Nightly 04:45 UTC `purge_old_captures`: captures older than 4 years (365×4+1 days) have screenshot, text and consent block deleted (`purged_at` set); hashes and fingerprint kept. **Skipped if under a legal hold** (Module 14 `Holds.covers` by client, time, phone tokens of proofs using the page, and the page's vendor).

Logic and rules — the consent checklist (checklist.py)

Per viewport, then the **worse** answer across viewports (rank pass < not_measured < fail):

- `CC_CONSENT_TEXT`: no consent block but the page has a phone box and a button → **fail** (`NP_NO_CONSENT_TEXT`); no phone box → everything not measured (form may continue on a later step).
- `CC_UNTICKED`: a visible checkbox within 250 px (+block height) of the consent words is ticked → fail (`NP_PRETICKED`).
- Wording items (any fail → `WK_WORDING`): `CC_BUTTON_MATCH` ("By clicking X" names a real button; not measured if no such phrase), `CC_NUMBER_SHOWN` (mentions the phone number), `CC_FONT_SIZE` (≥ `cc_min_font_px`, draft 10 px, counsel A15a), `CC_CONTRAST` (WCAG ratio ≥ `cc_min_contrast_ratio` 4.5, or 3.0 for text ≥ 24 px).
- Warnings only: `CC_ABOVE_BUTTON`, `CC_NO_SCROLL` (seen without scrolling).
- Information only: `CC_SELLER_NAMED` (client's own name minus Inc/LLC etc.; no brand list).
- AI items always not measured: `CC_OPT_OUT`, `CC_NOT_CONDITION`, `CC_AUTODIAL_DISCLOSED`.
- For results (`for_certificate`): uses CiV's capture of the certificate's page closest to the consent time and within `page_snapshot_max_gap_days` (1); none → not measured.
- Whole checklist marked **draft** (not approved by counsel).

Key functions/classes

- `apps/captures/models.py:ConsentPage / PageCapture`.
- `apps/captures/capture.py:pages_from_certificates / due / capture_page / capture_due / purge_old`.
- `apps/captures/browser.py:Browser.capture / is_public / MEASURE_JS`.
- `apps/captures/checklist.py:one_viewport / evaluate / for_certificate / contrast`.
- `apps/captures/api.py:page_row / page_source / page_vendor / wording_changed_on / latest`.
- `apps/captures/admin.py:ConsentPageInline / open_photo`.

API endpoints (IsClientUser, own client only)

- `GET /api/consent-pages` — up to 100 pages, newest seen first: `id` (`cpg-<pk>`), address, `mark`, `vendor` (most common CRM vendor name on its leads), `changedOn` (last time consent words changed between photos), `checklistPassed` / `checklistTotal` (measured, non-AI items), `missing`, `source` + `sourceLabel` (most common proof source), first / last seen, `leads` (certificates from this page), `capturedAt`, `problem`, `photos` (Phone / Computer URLs), `wording` (consent block), `items` (every checklist item: code, label, effect no_proof/wording/warning/info/ai, result), `draft: true`.
- `PATCH /api/consent-pages/<ref> {mark: approved|unmarked|rejected}` — client's own mark (restored feature, owner request 2026-10-21); never changes a result.
- `GET /api/consent-pages/<ref>/photo/<viewport>` — the latest PNG (served only through this API, no public address).

Scheduled jobs

- `capture-pages` → `apps.captures.tasks.capture_pages`, hourly at minute 20 (queue `capture`).
- `purge-old-captures` → `apps.captures.tasks.purge_old_captures`, 04:45 UTC daily.

Admin panel parts

- Admin → client page → **Consent pages** tab (`ConsentPageInline`, read-only, shown on GET only, left out of saves): address, "Seen in certificates" date range, latest photo time or why it could not be captured, Phone / Computer photo buttons (`admin:clients_client_page_photo`, needs `captures.view_consentpage`, `Cache-Control: private, no-store`), checklist (draft) with ✅ / ❌ / ⚠️ / i / –.

Frontend pages fed

- Consent Integrity → **Wording catalogue** tab (`WordingTab.tsx`, `ConsentPageCard.tsx` with mark buttons → PATCH).
- Lead Provenance: "Most often missing" + "Consent completeness" (`/api/leads`, `/api/provenance missing`).
- Alerts: "Consent page changed" medium alert (WEB domain) when `wording_changed_on` is recent.

4. Lead provenance (apps/provenance — ladder + evidence) — Module 8 steps 1–3, 5

Purpose (plain words)

For every number the client called, work out where it came from: a named vendor, a website, an uploaded list, or "untraced". Also find non-certificate evidence that the person reached out first (called in, texted YES, used the client's own form).

Data it fetches / receives, and from where

- Reads `contacts.ContactEvent` (first outbound, primary, non-test contact per number + owner), `imports.CrmCreation` (lead records: created_at, `vendor`, batch), `imports.CrmUpdate` (update uploads),

`imports.DialerListEntry` (dialer list uploads), `imports.ImportBatch` (kinds bulk / small / feed), `consent.ConsentProof` (certificates with a page).

- Evidence is read from synced records during the consent proof collection (`evidence.found`).

Tables

- `provenance` (`Provenance`): `client`, `phone_token`, `epoch`, `first_contact`, `first_contact_at`, `basis`, `source_name` (vendor as the CRM names it, website, or upload name), `own_website`, `lead_system`, `lead_record_id`, `lead_received_at`, `batch`, `note`, `computed_at`. Unique (client, phone_token, epoch). Index (client, basis, first_contact_at).
- **Ladder bases (strongest first)**: `feed` (lead platform feed — no reader yet, never matches), `crm_field` (CRM lead source names a vendor), `invoice` (vendor invoice — skipped, API only; Document AI later), `cert_domain` (certificate's website), `manual_import` (uploaded list), `untraced`.

Flow step by step (ladder)

1. Every 30 minutes, per active client (`rebuild_provenance`): `first_contacts` → each (phone token, epoch) and its first outbound counted contact.
2. `climb`: latest `CrmCreation` with this phone token created at or before the first contact.
3. Lead names a vendor → `crm_field`, source = vendor name.
4. Else latest non-withdrawn certificate proof (same epoch, dated ≤ first contact, with a page; prefer the one from the lead's own record) → `cert_domain`, source = website host without `www.`; `own_website` true when its source is client_form.
5. Else an upload before the contact (`upload_before`: the lead's own bulk/small batch, an update upload that touched the number, or — when no lead — a dialer list entry) → `manual_import`, source = "Update upload · ...", "Dialer upload · list N · date" or batch kind + date.
6. Else `untraced` with a note: "Arrived by an automatic feed; the record names no vendor." / "The CRM record names no vendor and carries no certificate." / "No CRM record before the first contact."
7. `update_or_create` the row.

Evidence for numbers without a certificate (evidence.py)

- `inbound_call`: a call record whose contact is inbound + call → proof dated the call time.
- `inbound_sms`: Twilio inbound text whose **whole** body (trailing spaces/./! stripped) is YES, Y, START, UNSTOP or SUBSCRIBE (owner decision 2026-10-21) → proof for the sender's token.
- `form_submission`: CRM record made by the client's form (HubSpot creation source FORM/FORMS; Salesforce LeadSource web / website / web form) with a creation time → proof per token.
- Only the fact, date and phone code kept. Not written consent. Used by consent `evidence_meets` (see §2 D). `INBOUND_DAYS = 30` (draft). Recordings wait for AI (Module 9); purchases are the Module 6 EBR proofs.

Key functions/classes

- `apps/provenance/ladder.py:first_contacts / climb / upload_before / upload_name / website / rebuild / untraced`.
- `apps/provenance/evidence.py:found / yes_word`.
- `apps/provenance/api.py:ProvenanceView / LeadsView / SearchView / VendorsView / missing_items / uploads`.
- `apps/provenance/admin.py:provenance_tile`.

API endpoints (IsClientUser)

- `GET /api/provenance` — this month: `period`, `numbers`, `ladder` (each basis: count, share, `available` false for feed and invoice), `topSources` (top 10 named, excluding manual_import), `untraced` (masked number, first contact time, tool, CRM record ID, note; max 200), `uploads` (last 50 upload batches: label, system, numbers, `withEvidence`, `stopBroughtBack`, `optOutsCleared`, `asked`, `proofRate`), `missing` (Module 7 checklist items failing, counted in certificates per page).
- `GET /api/leads` — `inspected` (CRM records created this month), `completeness` (% of this month's certificates whose page passed every no_proof/wording item, among pages with a photo), `missing`.
- `GET /api/vendors` — Vendor Ledger (delegates to `apps.vendors.portal.vendors_page`, Module 12).
- `GET /api/search?q=` — numbers (≥4 digits; masked) → evidence file refs; vendor names (≥2 chars) → vendor refs.

Scheduled jobs

- `provenance` → `apps.provenance.tasks.rebuild_provenance`, every 30 min (`crontab(minute='*/30')`).

Admin panel parts

- Admin client page top row **Lead provenance** tile: untraced share this month and number of proof files to check; links to the Proof requests list filtered to that client.

Frontend pages fed

- **Lead Provenance** (`frontend/src/screens/leads/LeadsScreen.tsx`): KPIs (Leads inspected, Consent completeness, Page match — sample), "Leads by signal level" and "Most frequent signals" (sample), "Most often missing" (`/leads`); `LadderCard`, `TopSourcesCard`, `UntracedCard`, `UploadsCard` (`/provenance`); `ProofRequestsCard` (`/proof-requests`).
- Evidence file "Came from" line and the "lead source" proof element (`/evidence/<ref> cameFrom`).
- Overview attention strip: `proofRequestsWaiting` via `/api/data-health` (per README).

5. Proof requests (apps/provenance/requests.py) — Module 8 step 4

Purpose (plain words)

For numbers that came from an uploaded list (or a vendor lead file) and have no evidence, CiV asks the client to prove consent for a fixed random sample. The client attaches a file or says "we have none". CiV

staff open the file and mark it valid (with the consent date) or not valid. The proof rate per list = valid proofs ÷ numbers asked.

Tables

- `proof_request` (`ProofRequest`): `client`, `batch_key` (stable upload key `system|tool|user-or-list|start`, or `leadfile|<upload id>`), `batch_label`, `batch_size`, `phone_token`, `masked_label`, `sent_at`, `due_at` (+30 days), `answer` (`proof` / `none`), `answered_at`, `answered_by` (ClientUser), `file` (private, `client-proofs/<client>/<uuid><ext>`), `file_type` (`signed_form` / `recording` / `email` / `other`), `original_name`, `file_sha256`, `review` ('' waiting / `valid` / `invalid`), `consent_date`, `review_note`, `reviewed_by` (staff email), `reviewed_at`, `purged_at`. Unique (client, batch_key, phone_token).
- Linked `consent_proof` rows of kind `client_supplied`.

Flow step by step

1. Every 15 minutes, after the import batch rebuild (`apps.imports.tasks.rebuild_batches`), `send(client)` runs for every active client.
2. For each upload batch (bulk / small) not sampled yet: numbers in the list → those called outbound → minus those with any non-withdrawn evidence proof (all kinds except self_asserted and client_supplied) → pool.
3. Sample size 50, or 100 when the list has more than 10,000 numbers; seeded by sha256(client|batch_key) so it is always the same numbers. Requests created with due date +30 days.
4. Same for Module 12 vendor lead files (`send_lead_files`, `UploadedFile` source VENDOR_LEADS).
5. Client answers (`POST /api/proof-requests/<ref>/answer`): Admin or company lawyer only; not after CiV reviewed.
   - `{"none": true}` → answer none; any client_supplied proof for it deleted.
   - File (≤25 MB; .pdf .jpg .jpeg .png .heic .mp3 .wav .m4a .eml .msg .txt .doc .docx) + `fileType` → file stored privately with SHA-256; a `client_supplied` proof with `proof_type='client_unchecked'`, no date, on the owner at the time the request was sent → that number's calls become **weak / WK_CLIENT_PROOF_UNCHECKED** (it never lifts a STOP).
   - Access-log event `proof_request_answered`; recheck queued (`new_proof`).
6. CiV staff review in admin (`review`): valid needs a consent date → proof gets `proof_type` `signed_form` (signed form file) / `verbal_recording` (recording) / `client_document` (email/other), dated 23:59:59 UTC of that date, owner re-assigned; invalid → proof deleted. Access-log `proof_reviewed`; recheck queued.
7. Nightly 04:50 UTC `purge_old_proof_files`: files answered more than 4 years ago deleted (`purged_at`); fingerprint and review kept; skipped under a legal hold.

Logic and rules

- Unanswered after `due_at` counts as "overdue" / not produced.
- `rates(client)` per list: asked, attached, valid, waitingReview, none, overdue, rate = valid ÷ asked.

- Owner decision 2026-10-21: CiV keeps the file (departure from spec's "never store uploaded files", noted for counsel). Every file opening logged (`proof_file_opened`).

Key functions/classes

- `apps/provenance/models.py:ProofRequest`.
- `apps/provenance/requests.py:send / send_lead_files / rates / answer / review / purge_old / batch_key / batch_label / tokens_of`.
- `apps/provenance/api.py:ProofRequestsView / AnswerView / FileView / can_answer`.
- `apps/provenance/admin.py:ProofRequestAdmin / ReviewForm`.

API endpoints

- `GET /api/proof-requests` (IsClientUser, everyone at the company) → `canAnswer`, `lists` (rates), `requests` (max 500: id `pr-<pk>`, list, masked number, sentAt, dueAt, answer, answeredAt/By, fileType, fileName, fileUrl, review, reviewNote, consentDate).
- `POST /api/proof-requests/<ref>/answer` (Admin or lawyer) — multipart `{file, fileType}` or JSON `{none: true}`.
- `GET /api/proof-requests/<ref>/file` (IsClientUser, own company) — download, logged, `private, no-store`.

Scheduled jobs

- Sending: inside `import-batches` → `apps.imports.tasks.rebuild_batches`, every 15 min.
- `purge-old-proof-files` → `apps.provenance.tasks.purge_old_proof_files`, 04:50 UTC daily.

Admin panel parts

- **Admin → Proof requests** (`ProofRequestAdmin`, eyebrow "Clients", as in the menu): list (sent, company, masked number, list, answer — Waiting / Overdue / Proof attached / We have none — and check pill amber/green/red), filters (answer, review, file type, client), search. No add, no delete. Detail: read-only facts + "Open file" button (logged, needs `provenance.view_proofrequest`); editable `review`, `consent_date`, `review_note`. Validation: valid needs a consent date; only an attached file can be checked. Saving a review calls `requests.review`.
- Lead provenance tile on the client page links here.

Frontend pages fed

- Lead Provenance → `ProofRequestsCard` / `ProofRequestRow` (`frontend/src/screens/leads/`): state pills ("Due …", "Overdue", "Proof attached · CiV checking", "Checked: valid / not valid", "You have none"), file picker with type (signed form, call recording, email, other), "We have none".
- `UploadsCard` shows `asked` and `proofRate` per upload.

Cross-area event map (who triggers whom)

- Sync record → certificates.scan_record (claim queue + phone match) → consent.proofs_seen (proofs + Module 8 evidence) → consent.keeping.contact_seen (freeze if facts changed).
- Claim read → refresh_proof_dates (5 min) → recheck; pending results rechecked every minute.
- Phone answer → recheck (`new_proof`).
- New page capture → recheck (`page_captured`).
- Opt-out saved → recheck (`late_opt_out`); new owner epoch → recheck (`owner_changed`); Rulebook change → recheck (`rule_changed`).
- Proof request answer/review → recheck (`new_proof`).
- Weekly full sweep → freeze `FZ_SOURCE_DELETED`.

6. Opt-outs, AI conversation review and contact checks (Module 9)

Read from the code on 22 October 2026. Backend root: `/home/shakib/Audit/backend` (apps under `apps/`). Frontend root: `/home/shakib/Audit/frontend/src`.

Shared vocabulary:

- **phone_token** = the "phone code": a per-client keyed token of the number (`apps.contacts.normalize.codes_for(client).token(number)` / `apps.keyring.tokens.Tokenizer`). Numbers and message words are never stored.
- **Portal address** of a number = `p-` + the first 24 hex chars of its phone token (`apps/contacts/api.py:number_ref`).
- **Locked** table = Postgres row-level security per client, via `apps.tenancy.sql.lock_table` in a migration; Celery jobs run each client inside `with tenant(client)`.
- **Client API permission** = `apps.accounts.permissions.IsClientUser` (any signed-in client user: Admin or Member; `IsClientAdmin` = Admin only).
- Rulebook = `backend/rulebook/v3.0.yaml`, read with `apps.rulebook.lookup.RuleSet()` (or `RuleSet('draft')` to use provisional values of TBD rules).

1. The STOP list (apps/optouts) — detection, deadline, re-added numbers

- **Purpose (plain words):** keep one list per client of every phone that asked not to be contacted: when, how and from which tool. Every other part (consent engine, metrics, alerts, number pages) reads this list.
- **Data it receives, and from where:**
  - The sync engine's hand-off (`apps/sync/handoff.py:deliver → RECEIVERS`). `OptoutsConfig.ready()` (`apps/optouts/apps.py`) adds `detect.optout_seen` to `handoff.RECEIVERS`, so every record CiV polls from a client tool (polled every 2 min, no webhooks) passes through it.
  - Twilio inbound texts (the `body` and `from` are read **in memory only**).
  - Salesforce lead / contact (`content_kind lead` or `crm_record`) with `DoNotCall is True`.
  - AI conversation review (`apps/conversations/review.py:act`) adds plain-word stops and "wrong number" (see §2).
  - Not used: the dialer's own result codes like "DNC" (the docstring says "the AI judges them").
- **Tables:**
  - `opt_out_event` (model `OptOutEvent`, locked): `client`, `phone_token` (indexed), `at` (when the person opted out), `channel` (`text`/`call`/`crm`), `method` (`keyword` = texted a stop word, `crm_flag` = "Do not call" ticked in the CRM, `ai` = said in plain words), `keyword` (which stop word; for AI rows `'wrong number'` or `''`), `source_system`, `source_record_id`, `lifted_at` (person allowed contact again; from then not in force), `counts` (False = found by an AI that had not passed its exam: still on the list, still checked in

tools, NOT counted against the client), `created_at`. Unique: (client, source_system, source_record_id, phone_token, method) = `one_optout_per_source_record`. Ordered newest `at` first.

- `optout_readded` (model `ReaddedNumber`, locked): `client`, `phone_token`, `source_system`, `record_id` (the CRM/dialer record that brought the number back), `record_created_at`, `opted_out_at`, `found_at`. Unique (client, source_system, record_id, phone_token) = `readded_once`.

- **Flow — keyword / CRM detection (`apps/optouts/detect.py`):**

1. A polled record arrives at `optout_seen(client, connection, record, phone_tokens)`; `record.raw` is parsed as JSON (bad JSON → ignored).
2. Twilio + `content_kind == 'text'` + `direction` starting with `inbound`: `stop_word(body)` strips trailing spaces / dots / `!`, upper-cases, collapses spaces, and returns the keyword only if the **whole message equals** one Rulebook stop word ("Stop!" yes; "Stop by tomorrow?" no). The `from` number is normalised, tokenised, and an `OptOutEvent(method=keyword, channel=text, keyword=<word>, at=record.changed_at)` is saved.
3. Salesforce + `DoNotCall is True`: one `OptOutEvent(method=crm_flag, channel=crm)` per phone token on the record.
4. `_save()` swallows `IntegrityError` (already on the list).
5. A post_save signal (`apps/consent/apps.py` → `keeping.optout_saved`) queues a consent re-check from `at` with reason `late_opt_out`, so later contacts get `NP_AFTER_OPT_OUT`.

- **Logic and rules:**
  - Stop words = Rulebook `optout_keywords` (status *proposed*, counsel item A3): STOP, QUIT, END, REVOKE, OPT OUT, OPTOUT, CANCEL, UNSUBSCRIBE. Same list hard-coded as `DEFAULT_KEYWORDS` if no Rulebook is loaded.
  - **Deadline** = `apps/rulebook/business_days.py:optout_deadline(received_at, time_zone, rules)`: Rulebook `optout_deadline_business_days` (10, *proposed*, A1) business days on the business-day calendar, counted in the person's local date (day 1 = the first business day after the opt-out day, per README). Returns `Deadline(date, n, tz, provisional)`; `provisional` is true while the rule or calendar is not counsel-approved.
  - **Time zone for the deadline** = `pick_time_zone(None, first time zone of the latest contact to that number, ['recipient_zip','area_code','America/New_York'])` (Rulebook `business_day_tz_fallback`, *proposed*). ZIP is never known yet (passed as None), so it is the area-code zone, else New York.
  - **When an opt-out is "in force" for the consent engine** (`apps/consent/needs.py:opt_out_in_force`): the latest `OptOutEvent` before the contact with `counts=True` and not lifted by then; NOT in force for (a) the single confirmation text within `confirmation_window_minutes` (5, *proposed*, A2) after the opt-out, (b) inside `optout_grace_business_days` (TBD, A4: strict 0, lenient 10), (c) once a newer consent **certificate** was captured after the opt-out. A later purchase or CRM box never lifts it. Contacts with an opt-out in force get the code `NP_AFTER_OPT_OUT` (`apps/consent/checks.py`), shown as "The person had said STOP before this contact."

  - **"Any reasonable means" revocation:** the paths are: whole-message stop word, Salesforce DoNotCall, and the AI judging plain words on texts and calls (§2).

- **Flow — re-added numbers (M34, `apps/optouts/readded.py:check_batches`):**

6. Called from `apps/imports/batches.py` after each batch rebuild (Celery `import-batches`, every 15 min).
7. For every upload batch of kind BULK or SMALL, `entries(batch)` yields (tool, record id, time, phone tokens): CRM updates (`crm_update`, id = `record@timestamp`), dialer list entries (`dialer`, id = `list X lead Y`), else CRM creations.
8. For each token, if an `OptOutEvent` exists **before** that record's time → `ReaddedNumber.get_or_create`.
9. The batch's `opted_out_readded` count is updated; first finds are written to the access log (`optout_readded`).

- **Key functions:**
  - `apps/optouts/detect.py:keywords` — Rulebook stop words (fallback list).
  - `apps/optouts/detect.py:stop_word` — whole-message stop-word match.
  - `apps/optouts/detect.py:optout_seen` — hand-off receiver for Twilio STOP and Salesforce DoNotCall.
  - `apps/optouts/readded.py:check_batches` — M34 re-added numbers per upload.
  - `apps/consent/needs.py:opt_out_in_force` — whether a STOP applies to a given contact.
  - `apps/rulebook/business_days.py:optout_deadline` / `pick_time_zone` — deadline date and its zone.
- **Admin panel parts:** no admin page for `opt_out_event`, `optout_readded` or `optout_mark_check` (no `admin.py` in `apps/optouts`). Re-added counts show on upload batches (`ImportBatch.opted_out_readded`).

2. Did the tools mark the stop? (apps/optouts/marks.py, Module 9 step 3)

- **Purpose:** after each opt-out, check each of the client's connected tools (dialer, texting, CRM) to see if the number was marked "do not contact". If not → urgent alert; still not by the deadline → "missed" (M38).
- **Data it fetches, and from where:**
  - Vicidial (read-only SQL): re-reads the number from the tool's own call row (`SELECT phone_code, phone_number FROM <call table> WHERE <id col> = %s`), checks the token matches, then `SELECT … FROM vicidial_dnc … UNION ALL … vicidial_campaign_dnc` with the 10-digit and full forms.
  - Convoso API: `log/retrieve` (number of the last call) then `dnc/search?phone_number=` (marked **TO CONFIRM** on a real account).
  - Twilio: no API call. A keyword stop from Twilio = marked (Twilio blocks texts after a stop word itself); plain-word stops = "not checked".
  - Salesforce and HubSpot (CRM): no extra calls; uses last-synced `CrmCreation.opted_out`, or an existing `crm_flag` OptOutEvent from that CRM.

  - Five9 and others: `check()` returns `(None, "CiV can't check this tool yet")`.
  - The number is only held in memory; never stored.

- **Table:** `optout_mark_check` (model `MarkCheck`, locked): `client`, `optout` (FK), `connection` (FK), `marked` (True / False / None = tool can't tell, **never a fail**), `note` (why not checked), `deadline_at` (end of the deadline day in the person's zone), `first_checked_at`, `checked_at`, `marked_at` (first time seen marked), `urgent_at`, `day_check_done`, `missed` (M38). Unique (optout, connection). Indexes (client, missed), (client, urgent_at).
- **Flow (`run(client, now)`), per client, every 5 minutes:**

1. Tools = the client's connections of kind dialer / SMS / CRM with status CONNECTED or STALE.
2. **First check:** opt-outs not lifted, created at least 10 minutes ago (`GRACE`), with fewer MarkChecks than tools. For each missing tool: `check()` → new MarkCheck with `deadline_at = deadline_of(event)`. If `marked is False` and CiV learned of the opt-out within the last day (`NEW`) → `urgent_at = now`. (Older opt-outs, e.g. found at first sync, are checked but raise no urgent flag.)
3. **Follow-ups** on every open MarkCheck (not missed, opt-out not lifted):
   - final once checked after the deadline → skip;
   - marked → one more check a day after `marked_at` (`day_check_done`); if it is now unmarked → `urgent_at` set;
   - not marked / not checked → re-checked every hour (`AGAIN`) until the deadline;
   - not marked and deadline passed → `missed = True`.
4. Any tool error → `(None, "The tool did not answer (<Error>)")`, i.e. "not checked this time".

- **Logic and rules:** timing 10 min + 5-min run ⇒ urgent within 15 minutes; urgent is shown in the portal only (no email — owner decision). `missed_optouts(client, start, end)` = **M38 Missed opt-outs**: opt-outs in the period (counts=True) with any `missed` MarkCheck; "Not measured" until any tool gave a True/False answer.
- **Key functions:**
  - `apps/optouts/marks.py:run` — one pass for one client.
  - `apps/optouts/marks.py:check` — picks the per-tool checker; wraps errors.
  - `apps/optouts/marks.py:vicidial` / `convoso` / `twilio` / `crm` — per-tool "is it marked?" (`CHECKS` map).
  - `apps/optouts/marks.py:number_from` — re-reads and verifies the number against the token (`NotChecked` otherwise).
  - `apps/optouts/marks.py:deadline_of` — end-of-day deadline in the person's zone.
  - `apps/optouts/marks.py:missed_optouts` — M38.
- **Scheduled job:** `optout-mark-check` → `apps.optouts.tasks.check_marks`, every 300 s, loops active clients inside `tenant(client)`.
- **Frontend fed:** Revocation page (Opt-outs table "Dialer marked", tool list, urgent flag), Overview card "Opt-outs your dialer missed", Alerts page (High "Opt-out not marked"), Who You Contact M41/M42, scoring (M38 saved in monthly snapshots via `apps/scoring/snapshots.py`).

3. M08 / M09 and the Revocation page numbers (apps/optouts/measures.py, step 5)

- **Purpose:** measure how fast the client's tools stop contacting after a real opt-out (owner decision 2026-10-21: **no paid CiV test lines**; measured from real opt-outs). Labelled `source: "Measured from your real opt-outs"` (Tier 3).
- **Data:** `OptOutEvent` with `counts=True` + their `MarkCheck` rows + `ContactEvent` (outbound, primary) after the deadline.
- **Logic:**
  - `hours_to_mark(event)`: ignore checks with no answer; if any tool has no `marked_at` → `NEVER` (∞) if a deadline passed, else None (undecided); otherwise the max over tools of `(marked_at − event.at)` in hours.
  - **M08 Opt-out propagation time** (`propagation_time`): median of those hours (display "x.x h" or "NEVER"), note includes the 90th percentile (`_p90` = sorted[int(0.9·n)]).
  - **M09 Opt-outs unprocessed at deadline** (`unprocessed`): only opt-outs whose deadline passed. `m09a` = not marked in some tool (`missed`), `m09b` = contacted (outbound primary) after the deadline, `notHonored` = either, `pastDeadline` = total. Display "a · b".
  - `revocation_page(client)`: last **90 days** (`WINDOW_DAYS`): `tests` (opt-outs with a time), `medianHours`, `p90`, `notSuppressedByDeadline` (= m09a), `contactedAfterDeadline` (= m09b), `deadlineBusinessDays` (Rulebook, default 10). If any times exist, also `systems` (tools; CRMs last; `contacting` = not a CRM), `rows` (row names: "SMS reply — “STOP”", "Ticked “do not call” in the CRM", "Spoken on a call", "SMS reply — plain words") × tools with median hours per cell (`hours` null when never/none, `tested`, `tests`), and `distribution` buckets: Under 1 h, 1–6 h, 6–24 h, 1–3 days, Over 3 days, Never.
- **Key functions:** `measures.py:propagation_time`, `unprocessed`, `hours_to_mark`, `tool_hours`, `row_name`, `revocation_page`.
- **API endpoints:**
  - `GET /api/revocation` (`apps/clients/api.py:RevocationView`, IsClientUser): `revocation_page(client)` + `authorisation.signedOn` = latest signed Annex C client document date (or "—").
  - `GET /api/metrics?period=YYYY-MM` (`apps/metrics/api.py:MetricsView`): includes M08 and M09 via `apps/metrics/overview.py:overview_metrics` (each with a 4-month history), plus M01 and M10. A month with a saved snapshot (Module 10) is read from it.
- **Frontend fed:** Revocation Integrity KPIs (Median time to stop, Slowest 10%, Not suppressed by deadline, Contacted after deadline), the matrix (`RevocationMatrix.tsx`), "Time to stop" chart; Overview "Median time to stop" (M08). The page is a **merge path**: fields Django does not send (`wording` list, `minTestsPerCell`, `authorisation.windowFrom/windowTo/channels`) stay sample data (`WordingCoverage.tsx`, `TestAuthorisation.tsx`). Before any opt-out is checked the KPIs show "Not measured" and matrix/chart keep the sample.

4. Opt-outs card API (apps/optouts/api.py)

- **Purpose:** the Opt-outs table at the top of Revocation Integrity: everyone who asked to stop, newest first, with the deadline, which tools marked it, and any contact after.
- **Flow (`OptOutsView.get`):**

1. Loads `RuleSet()` (None if no Rulebook).
2. Walks `OptOutEvent` newest first, keeps **one row per number** (its latest opt-out), up to 200 (`LIMIT`).
3. `row(event)`: zone as in §1; deadline + `deadlineProvisional`; `businessDaysLeft` = business days from today (person's zone) to the deadline (0 if passed); `phoneDisplay` = masked label of the latest contact (or "Number hidden"); `phoneE164` = portal address `p-…` (named E164 for the frontend, but is NOT the number); `how` = "Text · keyword" / "CRM · do not call" / "Call · AI review" / "Text · AI review"; `words` = the stop word.
4. `marks_of(event)`: `dialerMarked` (dialers only: False if any dialer said no, True if some said yes, null if none could tell), `tools` [{name, marked, note}], `urgent` (any tool with `urgent_at` and still `marked False`), `counted` (= `counts`).
5. `contactsAfter`: counted outbound primary contacts after the opt-out: count, channel and time of the latest, `afterDeadline`.

- **Endpoint:** `GET /api/optouts` — IsClientUser.
- **Frontend:** `screens/revocation/OptOutsCard.tsx` + `OptOutsTable.tsx` (`useOptOuts`). Filters All / Open (days left > 0) / Late (deadline passed and dialer said no, or contact after the deadline) / Contacted after STOP. Rows click through to `/numbers/<p-…>`. AI rows with `counted=false` show "AI result · not counted in your score yet".

5. Alerts on opt-out problems (apps/alerts/raise_.py)

- **Purpose:** "alert immediately on contact after STOP" (owner decision 2026-10-16) — implemented as portal alerts, raised every 5 minutes, **no email**.
- **Rules (all severity High, domain `REV`):**
  - `unmarked-<optout>-<connection>` "Opt-out not marked": any MarkCheck with `marked=False` and `urgent_at` set. *Lasting*: resolves by itself once the tool marks it.
  - `after-optout-<day>` "Contacts after opt-out": strict, unfrozen consent results with code `NP_AFTER_OPT_OUT` for contacts in the last 7 days, grouped by day (count of contacts and numbers). *Event*: stays until a person reviews it.
  - `readded-<day>` "Opted-out numbers re-added": `ReaddedNumber` found in the last 7 days.
  - Events older than 7 days at first sight raise nothing (`RECENT`).
- **Table:** `alert` (client, key unique per client, kind, severity, title, detail, domain, count, first_at, at, resolved_at, reviewed, reviewed_by, reviewed_at).
- **Job:** `alerts` → `apps.alerts.tasks.raise_alerts`, every 300 s.
- **Endpoints:** `GET /api/alerts`, `POST /api/alerts/<ref>/review` (`apps/alerts/urls.py`).

6. AI conversation review (apps/conversations, Module 9 steps 1, 2, 4; Module 16 consent flag)

- **Purpose:** read every conversation where the customer spoke (texts) and every recorded call, and let an AI decide whether the person still wants contact: asked to stop, a limit, wrong number, allowed contact again, unclear or nothing. Keep only the result — never words, never the number. Same for every client (no per-client settings).
- **Data it fetches, and from where:**
  - Text threads: `ContactEvent` rows (channel text, primary, not test) for one number + tool over the last 30 days; the words are re-read **by message ID from the tool, in memory** — `readers.READERS = {'twilio': twilio_text}` (Twilio `GET /2010-04-01/Accounts/{sid}/Messages/{id}.json → body`).
  - Call recordings: `readers.RECORDINGS`: Twilio (`Recordings.json?CallSid=` then download `.mp3` with Basic auth), Vicidial (`SELECT location FROM recording_log WHERE vicidial_id = %s`, then download the URL), Convoso (`log/retrieve` row's `recording` link, *TO CONFIRM*). Five9 not built. Max download 200 MB, timeout 120 s.
  - Connection must be CONNECTED or STALE.
- **The two slots (both OFF today; owner decision 2026-10-21):**
  - `ai.py`: `active()` returns the plugged-in reviewer or None. A reviewer is a callable `reviewer(thread) → (result, index of deciding turn or None, confidence 0–1[, extras dict])`. Extras today: `{"attorney": turn index}` (M45) and `{"consent": turn index}` (Module 16). "One listen, one request, several answers": future questions join the same request. `PROMPT_VERSION='v1'`, `MIN_CONFIDENCE=0.90`, `RESULTS = opted_out, partial_request, wrong_number, opted_back_in, none`. Setting `CONVERSATION_REVIEW_MODEL` (env, empty) + an adapter would switch it on — **no adapter exists yet**; only tests plug in a pretend reviewer via `ai.use()`.
  - `speech.py`: `active()` returns a transcriber `transcriber(audio bytes) → [{who: customer|business, text, at: seconds}]` (must tell customer from agent). Planned vendors: AssemblyAI or Deepgram. Off.
- **Tables:**
  - `conversation_review` (model `ConversationReview`, locked): `client`, `phone_token`, `channel` (text/call), `source_system`, `thread_end` (FK last ContactEvent read), `thread_end_at`, `messages_read`, `result` (`opted_out` Asked to stop · `partial_request` Asked for a limit · `wrong_number` · `opted_back_in` Allowed contact again · `unclear` · `none` · `not_reviewed`), `moment` (FK deciding message), `moment_at`, `moment_seconds` (calls), `confidence`, `model`, `prompt_version`, `thread_sha256` (fingerprint of the words, then dropped), `note`, `attorney_at` (M45: customer named a lawyer), `consent_at` + `consent_check` (waiting / confirmed / rejected) + `consent_checked_by` (Module 16 spoken consent — only a flag; never counts as consent on its own), `trusted` (AI had passed its exam for this channel), `approval` (waiting / approved / rejected), `approved_by`, `approved_at`,

`reported_wrong`, `reviewed_at`. Unique (client, channel, phone_token, thread_end_at). Index (client, result, reviewed_at).

- `conversation_exam_run` (model `ExamRun`, CiV-wide, not per client, kept forever): channel, model, prompt_version, ran_at, cases, right, stops, stops_caught, passed, mistakes [{case, expected, got}], note.
- `conversation_exam_case` (model `ExamCase`): extra exam questions added by staff — channel, turns JSON `[{"who","text","at"}]`, expected (opted_out / wrong_number / opted_back_in / partial_request / none), moment (deciding turn index), note, added_by, added_at. Made-up words only.

- **Flow — texts (`review.py`):**

1. `due(client)`: (token, tool) pairs with an inbound text in the last 30 days, whose last message is older than 10 minutes (`QUIET`, quiet thread), and with no review whose `thread_end_at` ≥ that last message.
2. `review_one`: no reader/connection → save `not_reviewed` "This tool can't share message text"; reader error → `not_reviewed` with reason.
3. Build thread `[{who: customer|business, text}]`, fingerprint it (SHA-256 of sorted JSON), call `judge()`, then `del thread`.
4. Save the review with moment, `attorney_at`, consent flag, `trusted = exam.trusted('text', reviewer)`.
5. `act()`.

- **Flow — calls:**

6. `calls_due(client)`: every call (primary, not test) between 30 days and 10 minutes ago not yet reviewed (`thread_end` not used). No skipping of short calls (owner decision).
7. `review_call`: no recording reader → `not_reviewed` "This tool can't share call recordings yet"; `NoRecording` → "No recording"; other errors → reason.
8. Audio → transcriber → `del audio`; turns → `judge()`; moment seconds → `moment_at = call start + seconds`; attorney / consent seconds likewise; `del turns`.
9. Save, then `act()`.

- **`judge(thread, reviewer)` rules:**
  - Unknown result → `unclear`.
  - opted_out / wrong_number / opted_back_in whose deciding turn is not a **customer** turn (e.g. the agent said "stop") → `unclear`.
  - Confidence < 0.90 and result ≠ none: a possible stop or wrong number → **`opted_out`** (the safe side, owner decision 2026-10-21); anything else → `unclear`.
  - Extras kept only if they point at a customer turn.
- **`act(client, review)` rules:**

  - `opted_out` or `wrong_number` with a moment → new `OptOutEvent(method=ai, channel=review.channel, keyword='wrong number' or '', source_system='ai_conversation', source_record_id='review-<pk>', counts=review.trusted)`.
  - `wrong_number` or `opted_back_in` with a moment: trusted → `carry_out()` at once; not trusted → `approval='waiting'` for CiV staff.
  - `carry_out`: wrong number → `apps.contacts.epochs.split(...)` starts a **new owner (phone epoch)** at the moment (reason WRONG_NUMBER, source ai_conversation); opted back in → sets `lifted_at = moment_at` on earlier un-lifted opt-outs.
  - `partial_request`, `unclear`, `none` → only counted (no action).
- **Staff decisions:**
  - `approve(review, by, yes)`: only when waiting; sets approved/rejected + who/when; yes → `carry_out` + `recount(review, True)` (the stop now counts; consent re-check queued `late_opt_out`).
  - `report_wrong(review, by)`: marks `reported_wrong`; a wrong "allowed again" (already applied) is undone (`lifted_at=None`, re-check queued); a wrong stop/wrong-number stays on the STOP list but `counts=False`; a waiting answer becomes rejected; if the active AI's model matches → `exam.fail_now()` (AI not trusted until it passes again).
- **Accuracy gate (`exam.py`, step 4):**
  - Cases: `exam/text.json` (300) and `exam/call.json` (300) made-up conversations + `ExamCase` rows.
  - Marked through the same `judge()`. A stop counts as caught only with the right moment (deadline counts from it).
  - **Pass** = cases ≥ 300 (`MIN_CASES`, Rulebook `ai_accuracy_min_items`) AND every stop caught (`STOP_MARK` 1.0, Rulebook `conv_review_stop_catch_mark`) AND ≥ 98 % right (`OVERALL_MARK`, Rulebook `conv_review_accuracy_pass_mark`). Texts and calls separately; per model + prompt version.
  - `trusted(channel, reviewer)` = latest run for that model/version passed. `due()` = no run, or older than 30 days. AI errors count as wrong answers.
- **Key functions:**
  - `apps/conversations/review.py:due` / `calls_due` — what is ready to read.
  - `apps/conversations/review.py:thread_of` — rebuilds a text thread in memory.
  - `apps/conversations/review.py:judge` — applies the AI answer rules.
  - `apps/conversations/review.py:review_one` / `review_call` — one text thread / one call.
  - `apps/conversations/review.py:act` / `carry_out` — writes the STOP list, new owner, lift.
  - `apps/conversations/review.py:approve` / `recount` / `report_wrong` — staff decisions.
  - `apps/conversations/review.py:consent_flag` — Module 16 spoken-consent flag.
  - `apps/conversations/review.py:review_due` — per client: up to 200 texts, then up to 200 calls (calls only if speech slot on).

  - `apps/conversations/exam.py:run` / `trusted` / `due` / `fail_now` — the exam.
  - `apps/conversations/readers.py:reader_for` / `recording_reader_for` — per-tool readers.
- **Scheduled jobs:**
  - `conversation-review` → `apps.conversations.tasks.review_conversations`, every 60 s; returns 0 and sends nothing while no AI is chosen.
  - `conversation-exam` → `apps.conversations.tasks.run_exams`, daily 02:10 UTC; text always (if AI set), call only if speech slot set; runs only when due; ~300 conversations per channel sent to the AI per run.
- **Admin panel parts (`apps/conversations/admin.py`, eyebrow "AI"):**
  - *AI exam results* (`ExamRun`, read-only): result pill green "Passed" / red "AI failed the exam", stops caught "x of y", right "%".
  - *Extra exam questions* (`ExamCase`): add/edit; `added_by` = staff email.
  - *AI answers* (`ConversationReview`, read-only): columns company, channel, result, confidence, exam passed, staff pill (waiting/approved/rejected/Reported wrong), **Read** button; filters approval, result, channel, trusted, reported_wrong, client. Actions: Approve, Reject, Report as wrong. Each action is written to the access log (`ai_answer_approved` / `ai_answer_rejected` / `ai_answer_reported_wrong`).
  - *Read* view `/<pk>/read/` (needs `conversations.view_conversationreview`): re-reads the conversation from the client's tool (or re-transcribes the call) for one look, template `conversations/read.html`, `Cache-Control: private, no-store`, logged as `conversation_read`; nothing saved.
  - Client page tile `ai_tile(client)` (used in `apps/clients/admin.py`): "AI review" — stops this month, reviews read, waiting answers, exam state for texts and calls (or "no AI chosen yet"); links to *AI answers* filtered to the client.
- **API endpoints:** none of its own. Results feed `GET /api/data-health` → `conversationReview` (`apps/sync/api.py:conversation_review`, last 30 days: `reviewed`, `optOutsFound` (AI, counted), `missed`, `notMarkedFirstCheck`; null until anything other than not_reviewed exists), `GET /api/optouts`, `/api/revocation`, `/api/who-contact`, scoring.
- **Frontend fed:** Overview key number "Opt-outs your dialer missed" (`screens/overview/KeyNumbers.tsx`, "Not yet measured" / "Comes with AI review of calls and texts" while null); Revocation Opt-outs rows "Call · AI review" / "Text · AI review"; Who You Contact M41/M42/M45/M48.

7. "Who You Contact" page backend (apps/metrics/who.py)

- **Purpose:** show which "who may we contact" checks pass or fail, from data CiV really holds, over the last 30 days; plus which services/lists the client uploaded, and the client's own statements per lead source.
- **Data:** outbound primary non-test `ContactEvent` in the last 30 days (`window`); `ConsentResult` (strict); uploads `DncCheck`, `InternalListEntry`, `RndQuery`, `UploadedFile`; `CrmUpdate.cleared_optout`; `OptOutEvent` + `MarkCheck`; `ConversationReview`; `CrmCreation.vendor`; `Provenance` (`cert_domain`); `ImportBatch`.

- **Table owned:** `lead_source_check` (model `LeadSourceCheck`): client, `source_key` (`vendor:<name>` or `upload:<batch key>`), `column` (national · state · internal · reassigned), `checked`, `set_by` (ClientUser), `set_at`. Unique (client, source_key, column).
- **Areas and checks (each check = {state pass/warn/fail, name, result, source records/list, metric, why}; names match the portal's sample names so each replaces its own row):**
  - **`dnc`** — M40 "Expired customer relationship used" (`NP_RELATIONSHIP_LAPSED`). Only with uploaded DNC check records: M13 "National registry checked" (share of contacts with a check ≤ 31 days before; pass ≥ 99 %, warn ≥ 90 %), M39 "Called after your provider flagged the number" (+ masked flags), M49 "Registry copy older than 31 days", "State lists checked (CA, AZ, TX, FL, NV)" (list names other than national).
  - **`idn`** (internal list) — M44 "Removed from the list too early" (CRM opt-out marks cleared); `stops_reached`: **M41** "Stop requests that never reached your list" (counted keyword/AI opt-outs in 30 days with a tool answer; failed = any `missed`) and **M42** "Added within 10 business days" (share marked in time among those whose deadlines passed; median days), shown only when something was checked or the AI has run; with an uploaded internal list: M12 "Called after joining your list" (+ flags). Flags for M41 = contacts after an unreached stop ("Asked to stop, never added", from "AI review" or "Texting tool replies").
  - **`rnd`** (reassigned) — **M48** "Contacted after a “wrong number” reply" (AI wrong-number opt-outs, counted; shown once AI has run), M47 "Consent older than 12 months" (share of proofs > 365 days before contact), with uploaded query logs M11 "Covered by the safe harbor" (an RND query answer "no" within 31 days before).
  - **`lit`** (litigation) — **M45** "Contacted after naming an attorney": trusted reviews with `attorney_at`, contacts after it (first per number).
  - `services`: DNC check records / Internal do-not-contact list / Reassigned-number query logs — "Uploaded <date> · N numbers" or "Not supplied · blocks M13/M12/M11", with `covers` per column.
  - `leadSources`: top 20 CRM vendors (else certificate websites from provenance) + last 20 bulk/small uploads, each with the four column switches; id = `ls-` + 12 hex of SHA-256 of the key.
- **Endpoints (`apps/metrics/api.py:WhoContactView`):**
  - `GET /api/who-contact` — IsClientUser.
  - `PATCH /api/who-contact/<ls-id> {column, checked}` — client Admin or the company lawyer (`is_admin or is_counsel`), else 403; 404 if unknown source/column; saves `LeadSourceCheck`, writes access log `lead_source_marked`, returns the row.
- **Frontend fed:** `screens/who-contact/WhoContactScreen.tsx` (`useWhoContact`; read-only when the session view is `underwriter`), `AreaTile`, `ChecksList`, `WhoCheckedWhat`, `LeadSourceSwitch` (PATCH via `useSend`), `components/drawers/WhoAreaDrawer.tsx`. `who-contact` is both a MERGE path (GET merged over sample by area id / check name) and a WRITE path (PATCH goes to Django) in `frontend/src/server/backend.ts`.

8. Contact Conduct: calling hours, frequency, robo, abandon (apps/metrics/conduct.py)

- **Purpose:** the Contact Conduct metrics (M15–M21, M31) from contacts CiV holds.
- **Data:** outbound, primary, non-test `ContactEvent` in the chosen month (`?period=YYYY-MM`) or last 30 days; Rulebook **draft** values (`RuleSet('draft')`) because calling hours and frequency limits are still TBD; `ConsentNeed` / `ConsentResult` (strict).
- **Rules:**
  - Each contact is checked in **every** time zone its number could be in; outside in any zone = outside (the stricter wins). Several zones or low location confidence → counted as **"possible"**, not confirmed.
  - **M16 out of hours**: state `calling_hours` (draft: 08:00–21:00 federal; FL, OK 08:00–20:00, etc.; optional `sunday_start`); federal breaches skipped for contacts whose strict consent is verified/weak. Display "federal · state", value = contacts outside either; note says "draft hours, not confirmed by a lawyer". Charts: top 10 states (with window text), hours-of-day histogram (flagged before 8 or from 21).
  - **M17 frequency**: per (number, owner epoch), campaigns together; for each contact count earlier contacts inside the state's `frequency_limits.window_hours` (default 24); ≥ `max` (default 3) → over; low confidence → possible. Looks back 7 extra days for earlier contacts.
  - **M18 holidays**: always "Not measured — Rule pending: state holiday restrictions not set yet (counsel item A8)".
  - **M19 robo**: robocalls/robotexts (strict ConsentNeed `robocall=True`) whose strict result is not verified/weak; notes how many had unknown dial mode (counted as autodialed).
  - **M21 caller ID**: always "Not measured"; note shows the STIR/SHAKEN attestation mix if the dialer gives it.
  - **M15 cross-border**: contacts to non-US numbers, top 3 countries.
  - **M31 abandoned rate**: answered calls whose disposition contains "abandon"/"drop"/"abnd" (rough until each dialer's codes are mapped); limit = `abandoned_call_max_rate` (0.03); by campaign chart.
  - Each metric gets a 4-month `history`.
- **Key functions:** `conduct.py:span`, `months_before`, `contacts`, `Rules`, `outside`, `hours_check`, `frequency_check`, `holiday_check`, `robo_check`, `caller_id_check`, `cross_border`, `abandon_check`, `conduct_metrics`, `conduct_page`.
- **Endpoints:**
  - `GET /api/conduct?period=` (`ConductView`, IsClientUser) → `{metrics: [codes], byState, possibleLocationUncertain, hoursOfDay, byCampaign, abandonLimit}` (fully real).
  - `GET /api/metrics?period=` (`MetricsView`) → conduct metrics + overview metrics (M01, M10, M08, M09); saved month snapshot if one exists.

- **Scheduled:** none of its own; nightly `metric-snapshots` (04:00 UTC, `apps.scoring.tasks.snapshot_metrics`) saves `conduct_metrics` + `overview_metrics` + M38 per month (F4 "Calling conduct" factor uses M15–M20, M31).
- **Frontend fed:** `screens/conduct/ConductScreen.tsx` (`useConduct`, `useMetrics`), `MetricTile.tsx`; metric drawer. `conduct` is a backend path; `metrics` is a merge path (sample kept for metrics not sent).

9. "Can I call now?" and the number's calling window (apps/contacts)

- **Purpose:** for a typed number, say where the person probably is and whether **now** is inside allowed calling hours. Advice only — CiV never blocks a call.
- **Data:** the typed number (normalised in memory, never stored); offline area-code data (`phonenumbers` library) with CiV corrections (`apps/contacts/location.py`: split area codes like 850, 812, 270, 208, 541, 928...; NANP countries; US territories); Rulebook draft `calling_hours`.
- **Flow (`CanCallView.post`):**

1. `clean_number(request.data['number'])`; invalid → 400 "Enter a full phone number."
2. `locate(number)` → recipient_state, country, time_zones, location_confidence (`medium` one zone; `low` several/unknown; `high` reserved for ZIP, later), is_non_us.
3. `window(state, zones, is_non_us)` (`apps/contacts/calling.py`): non-US → verdict `unknown` + note; no zones → unknown; no Rulebook → unknown; reads `RuleSet('draft').get('calling_hours', state)`; for each zone local time + allowed; verdict `ok` only if allowed in all zones, else `not_safe`; notes for several zones ("the stricter one decides") and draft hours ("not confirmed by a lawyer yet"); `holidays`: "State holiday restrictions: not set yet (counsel item A8)".
4. Returns `{number: masked, state, country, timeZones, confidence, checkedAt, verdict, hours, status, draft, place, zones[], note, holidays}`.

- **Logic note:** the can-call check does **not** look at the STOP list, DNC lists or consent — only location and hours. (Opt-out status is shown on the number page instead.)
- **Endpoint:** `POST /api/can-call {number}` — IsClientUser. Nothing stored.
- **Frontend:** `screens/rulebook/CanCallBox.tsx` (`useCanCall` mutation), shown in the Rulebook → Calendar tab (`CalendarTab.tsx`), not on a page of its own.

10. Numbers page (evidence file) — opt-out parts (apps/contacts/api.py:number_file)

- **Purpose:** one number's page: contacts, owners, consent, opt-outs, list checks, calling window, seal.
- **Endpoint:** `GET /api/evidence/<p-24hex>` (`NumberView`, IsClientUser); 404 if unknown.
- **Opt-out related fields:** `optOuts` (`apps/consent/panel.py:optouts_of` → [{at, channel, method label}] oldest first, all opt-outs including lifted/uncounted); `listChecks` (`list_checks`: "Internal do-not-contact list" = on it since the earliest of first STOP or internal-list entry; "DNC check records" = latest uploaded check or "not supplied"); `callingWindow` (same `window()` as §9, for the latest contact's state/zones); `epochs` (owners — a "wrong number" from the AI starts a new one); `onHold`.

- **Frontend:** route `/numbers/[phone]` → `screens/evidence/EvidenceFileScreen.tsx` (`useEvidenceFile`), `CallingWindowCard.tsx`, `OwnersCard.tsx`, `ContactTimeline.tsx`, `ConsentCard.tsx`, `ProofElements.tsx`. Also the evidence-file PDF/JSON (`/api/evidence-file/...`, Module 14) and refetch button. Reached from Opt-outs rows and the Contact Ledger.
- Related: `apps/anchoring/position.py` adds an "Opt-out history" row (latest opt-out before a date) to the "position on a date" view.

11. Scoring uses (for cross-reference)

- `apps/scoring/extra.py` adds per saved month: M34 re-added (`ReaddedNumber` by `record_created_at`), M41/M42 (same logic as Who You Contact but period-based), M48, M45 ("Not measured — Comes with AI review of calls and texts" until the AI has run), M12, M11.
- `apps/scoring/snapshots.py` saves M38 (`missed_optouts`) with conduct + overview metrics.
- README: M10 and M34 weigh double in F3; any contact after an opt-out (M10) holds F3 at 75 or worse; M08 bands ≤24 h → 100, ≤72 h → 50, else/NEVER → 0; M09 share not honored 0 → 100, ≤10 % → 50.
- `apps/buyers/attestation.py` and `apps/reconcile/api.py` count `NP_AFTER_OPT_OUT` and re-added numbers for the insurer pages.

12. Frontend wiring summary

Portal page	Route	Endpoints	Real vs sample
Revocation Integrity	`/revocation`	`GET /api/optouts` (backend path), `GET /api/revocation` (merge path)	Opt-outs table fully real; KPIs/matrix/chart real once data exists; wording coverage and test-authorisation window stay sample (Annex C signed date real)
Who You Contact	`/who-contact`	`GET /api/who-contact` (merge), `PATCH /api/who-contact/<id>` (write)	Real checks replace sample rows by name; unbuilt checks keep sample
Contact Conduct	`/conduct`	`GET /api/conduct` (backend), `GET /api/metrics?period=` (merge)	Charts real; metrics real where sent
Numbers (evidence file)	`/numbers/[phone]`	`GET /api/evidence/<p-…>` (backend)	Real
Rulebook → Calendar "Can I call now?"	`/rulebook`	`POST /api/can-call` (backend)	Real, not stored
Overview	`/`	`GET /api/data-health` (`conversationReview`), `GET /api/metrics` (M08, M10)	Real when measured
Alerts	`/alerts`	`GET /api/alerts`	Real

Routing is decided by `frontend/src/server/backend.ts`: `BACKEND_PATHS` (forwarded to Django when `BACKEND_URL` is set), `MERGE_PATHS` (GET merged over the sample with `overlay()`: lists matched by `code` / `id` / `name`), `WRITE_PATHS`.

7. Metrics, scoring, findings, alerts, reports and litigation (Modules 10–11)

Read from the code on 22 October 2026. Backend root: `/home/shakib/Audit/backend` (paths below are relative to it unless they start with `frontend/`). Everything here is what the code does today.

Common facts for this whole slice:

- All client endpoints are under `/api/` (each app's `urls.py` is included at `api/` in `config/urls.py`).
- `IsClientUser` = any signed-in client-portal user (`accounts.ClientUser`), Admin or Member. Extra checks inside a view are noted per endpoint (`is_admin`, `is_counsel` = "company lawyer" flag).
- Celery runs in UTC (`CELERY_TIMEZONE = 'UTC'`, `TIME_ZONE = 'UTC'`). Schedules are in `config/settings.py` (beat).
- Per-client tables are "locked" by a `0002_lock` migration (`apps.tenancy.sql.lock_table`, row-level security); each nightly/5-minute job runs per client inside `with tenant(client):`. `litigation_month` is public (no lock).
- Rulebook = `rulebook/v3.0.yaml` (status provisional). `RuleSet()` default mode is `internal` (may use provisional); Contact Conduct uses `RuleSet('draft')` (may use draft/TBD test values).
- The portal (Next.js) proxies `/api/*`. For "merge paths" (`metrics, who-contact, score, domains, actions, reports, litigation, …`) the real backend answer is overlaid on the portal's sample data item by item (`frontend/src/server/ backend.ts: overlay`): lists are matched by `code`, then `id`, then `name`; an item the backend does not send keeps its sample. `alerts` and `conduct` are full backend paths (no merge). Writes to `who-contact`, `actions`, `reports` always go to the backend.

Metrics (apps/metrics + metric producers in other apps)

Purpose

Count each spec metric (codes M01…M49) for one client over a window, in the shape the portal shows: `{code, value, display, note, notMeasured, evidence?, history?, worstCase?}`. The app also serves the Contact Conduct page charts and the "Who You Contact" checks page, and stores the client's own statements about its lead sources.

Data it uses, and from where

- `contacts.ContactEvent` (Module 4): outbound, `is_primary=True` (not duplicate copies), `is_test=False`; fields used: `occurred_at, time_zones, recipient_state, location_confidence, is_non_us, country, channel, answered, disposition, campaign, dial_mode, stir_attestation, phone_token, epoch_id, masked_label`.
- `consent.ConsentResult` / `ConsentNeed` (Module 6): `variant='strict'`, `frozen=False`; status `verified / weak / no_proof / conflicting / pending / not_needed`; `is_legal_basis`; `codes` (e.g. `NP_AFTER_OPT_OUT`, `NP_RELATIONSHIP_LAPSED`); `ConsentNeed.robocall`.

- `optouts.OptOutEvent`, `MarkCheck`, `ReaddedNumber` (Modules 5/9): opt-outs that `counts=True`, per-tool mark checks (`marked, marked_at, missed, deadline_at`).
- `uploads.DncCheck, InternalListEntry, RndQuery, UploadedFile` (Upload centre, no-API sources).
- `imports.CrmCreation, CrmUpdate, ImportBatch`; `provenance.Provenance` (lead sources).
- `conversations.ConversationReview` (Module 9 AI review: `attorney_at`, `trusted`, `result`).
- `vendors` (Module 12): contracts, dispute candidates, vendor months.
- Rulebook keys: `calling_hours`, `frequency_limits`, `abandoned_call_max_rate` (draft 0.03), `optout_deadline_business_days`.

Tables

- `lead_source_check` (`metrics.LeadSourceCheck`): `client, source_key` (`vendor:<name>` or `upload:<batch key>`), `column` (`national | state | internal | reassigned`), `checked` bool, `set_by` (ClientUser), `set_at`. Unique (client, source_key, column). It is the client's own statement ("this vendor checks the national DNC list"), shown to the insurer as a statement, never counted as proof.
- Metric values themselves are not stored by this app; they are saved monthly by `apps/scoring` (`metric_snapshot`).

Windows

- `conduct.span(period)`: `'YYYY-MM'` → that calendar month (UTC); no period → the last 30 days.
- `months_before(period, 3)` → the three earlier months; every live metric carries `history` = 3 earlier values + now.
- Who You Contact (`who.py`) always uses the last 30 days (`DAYS = 30`), no month picker.

Full metric table

"Saved" = included in the nightly monthly snapshot (`scoring.snapshots.measure`). "Scored" = gets a 0–100 score in `scoring/scores.py`. "Factor" = which Exposure factor uses its score. "Live only" = computed only for a page, not saved.

Code	Name (portal)	How it is measured (code)	Not-measured / worst-case rules	Saved	Scored as	Factor	Status
M01	Consent Coverage	`metrics/overview.py:coverage`. Strict, non-frozen consent results of primary contacts in the window, excluding `pending` and `not_needed`. value = % of legal-basis contacts with status `verified` or `weak`.	No checked contacts → "Not measured". If rows exist but none are legal-basis, value None, display "—".	yes	share (higher better)	F1	Real

Code	Name (portal)	How it is measured (code)	Not-measured / worst-case rules	Saved	Scored as	Factor	Status
		Note gives the overall % incl. CiV-policy contacts; evidence "X of Y legal-basis contacts".					
M02	Consent Status Mix	—	—	no	—	—	Sample only (portal mock)
M04	Form Wording Pass Rate	not computed in snapshots	—	no	share (listed)	F1	Not saved
M05	Form Change Alerts	— (alerts raise "Consent page changed" instead)	—	no	—	—	Sample
M06	Lead Traceability	not saved	—	no	share (listed)	F5	Not saved
M07	Vendor Terms on File	`vendors/contracts.py:terms_on_file`: active vendors with an unexpired contract on file ÷ active vendors, %. Note adds how many have every required clause (`vendor_required_clauses`) when the Module 16 AI reader checked clauses, else "clauses not checked yet".	No vendor named in CRM → not measured; no contracts uploaded → not measured "Contracts not supplied".	yes	share	F5	Real (Module 12)
M08	Opt-out Propagation Time	`optouts/measures.py:propagation_time`: per counted opt-out, hours until every checkable tool marked the number (max over tools); value = median hours (1 decimal); note gives n and	No opt-out checked → not measured.	yes	band M08: ≤24 h → 100, ≤72 h → 50, else 0; display "NEVER" → 0	F3	Real

Code	Name (portal)	How it is measured (code)	Not-measured / worst-case rules	Saved	Scored as	Factor	Status
		90th percentile. A tool still unmarked past deadline → "NEVER". CiV checks 10 min after, then hourly (accurate to about an hour).					
M09	Opt-outs Unprocessed at Deadline	`optouts/measures.py:unprocessed`: opt-outs whose deadline passed; M09a = not marked in some tool, M09b = contacted after the deadline; display "a · b"; also `pastDeadline`, `notHonored` (missed or contacted). value is always None.	No opt-out past its deadline → not measured.	yes	band M09 on notHonored ÷ pastDeadline: 0 → 100, ≤0.10 → 50, else 0	F3	Real
M10	Contacts After Opt-out	`metrics/overview.py:after_optout`: count of strict results with code `NP_AFTER_OPT_OUT`; note = distinct numbers.	Always measured (0 if none).	yes	count per 10k (double weight)	F3 (×2), F3 cap	Real
M11	Pre-Contact Reassigned Checks	`scoring/extra.py:reassigned_checks`: % of month's contacts preceded by an uploaded reassigned-number query with answer "no" in the 31 days before. Live twin in `who.py` ("Covered by the safe harbor").	No RND query logs uploaded → worst case ("not checked"); no contacts → not measured.	yes	share	F6	Real (upload)
M12	Internal List Contacts	`scoring/extra.py:internal_list`: contacts	No internal list uploaded → worst case.	yes	count per 10k	F2, F2 ca	Real (upload)

Code	Name (portal)	How it is measured (code)	Not-measured / worst-case rules	Saved	Scored as	Factor	Status
		after the number's `InternalListEntry.added_on`. Live twin in `who.py` "Called after joining your list" with flagged masked numbers.				p	
M13	DNC Scrub Record Check	`who.py:dnc_area` only: % of last-30-day contacts with a DNC check record within 31 days before (pass ≥99, warn ≥90, else fail); also lists state lists checked.	No DNC check records uploaded → the row is not sent (portal keeps "not supplied").	no (live only)	count per 10k (listed in COUNTS)	F2 (listed)	Live on Who You Contact; never saved, so F2 effectively = M12 only
M14	Litigator List Contacts	—	—	no	—	—	Sample only
M15	Cross-border Contacts	`conduct.py:cross_border`: contacts with `is_non_us=True`; note top 3 countries.	Always measured.	yes	count per 10k	F4	Real
M16	Out-of-Hours Contacts	`conduct.py:hours_check`: per US contact, draft `calling_hours` for the recipient state; outside the window in any possible time zone counts. Federal (only if not verified/weak consent) and state counted; each contact once (`either` = value). Low location confidence or >1 time zone → "possible", not counted. Display	No federal `calling_hours` value → notMeasured "Rule pending: calling hours not set yet (counsel item A6)" (saved state = not_measured, since the note has no word "pending"). Every number labelled "draft hours, not confirmed by a lawyer".	yes	count per 10k	F4	Real (draft rule)

Code	Name (portal)	How it is measured (code)	Not-measured / worst-case rules	Saved	Scored as	Factor	Status
		"federal · state". Also feeds hour-of-day and by-state charts.					
M17	Over-Limit Contacts	`conduct.py:frequency_check`: per person (phone_token + epoch_id), calls sorted; a contact counts when the earlier contacts inside `window_hours` (default 24) ≥ `max` (default 3) of the state's draft `frequency_limits`; looks 7 days back for context; low confidence → possible.	Always measured; note says when no state had a limit.	yes	count per 10k	F4	Real (draft)
M18	Restricted-Holiday Contacts	`conduct.py:holiday_check`: not computed.	Always "Rule pending: state holiday restrictions not set yet (counsel item A8)" → saved state rule_pending.	yes	none (rule pending)	F4	Rule pending
M19	Robo-contact Required-Proof Count	`conduct.py:robo_check`: strict `ConsentNeed.robocall=True` contacts whose result is not verified/weak (excluding pending/not_needed); note shows how many had unknown dial mode (counted as autodialed).	0 robo contacts → value 0 measured.	yes	count per 10k	F4	Real
M21	Caller ID Quality	`conduct.py:caller_id_check`: value always None; note = STIR/SHAKEN attestation mix	notMeasured "Test dials of each caller ID come with Module 9" (paid test lines were dropped,	yes	— (not measured)	F6	Not measured

Code	Name (portal)	How it is measured (code)	Not-measured / worst-case rules	Saved	Scored as	Factor	Status
		when the dialer gives it.	so it stays not measured).				
M22a / M22b / M23/ M24/ M25a	Records & controls metrics	—	—	no	—	—	Sample only
M26 / M27 / M28	Lead provenance metrics	—	—	no	listed	F5	Not saved in this slice
M29	Vendor Grade Distribution	`vendors/score.py:grade_mix` (vendor page only): vendors at each grade this month, lowest named. value None.	No vendor → not measured.	no	—	—	Real on Vendor Ledger, not saved
M30	Disputable Lead Value	`vendors/disputes.py:disputable_value`: $ total of dispute candidates still inside their window; note leads in window, leads with no price, vendors with no contract.	No vendor leads → not measured.	yes	not scored (not in any list)	—	Real (display)
M31	Abandoned-Call Rate	`conduct.py:abandon_check`: answered calls whose `disposition` contains `abandon`, `drop` or `abnd`; value = % of answered; note = campaigns above the limit (`abandoned_call_max_rate`, draft 3%). Chart by campaign (top 10).	No answered calls known → "Your dialer does not say which calls were answered".	yes	band M31: ≤1% → 100, ≤3% → 50, else 0	F4	Real (rough, codes not mapped per dialer)
M34	Re-added	`scoring/`	Always measured.	yes	count per 10k	F3	Real

Code	Name (portal)	How it is measured (code)	Not-measured / worst-case rules	Saved	Scored as	Factor	Status
	opted-out numbers	`extra.py:readded`: `ReaddedNumber` rows with `record_created_at` in the month.			(double weight)	(×2)	
M36	Evidence coverage	`scoring/coverage.py:coverage` — computed from the other metrics, stored in `snapshot_run.score.coverage`, not as a metric row. See Scoring.	No scored metric → not measured.	in run.score	—	cap	Real (Module 14)
M38	Missed opt-outs	`optouts/marks.py:missed_optouts`: opt-outs in the period still unmarked in some tool at deadline.	Nothing checked → not measured.	yes	count per 10k (scored) but in no factor/domain; never a finding (`SKIP`, same as M41)	—	Real, display only
M39	Called after provider flagged	`who.py:dnc_area` (live)	needs DNC records	no	—	—	Live only
M40	Expired customer relationship used	`who.py:dnc_area` (live): strict results with `NP_RELATIONSHIP_LAPSED` (3 months after inquiry / 18 after purchase)	—	no	—	—	Live only
M41	Stop requests never reached every tool	`scoring/extra.py:stops` (monthly) + `who.py:stops_reached` (live): keyword/AI opt-outs with at least one checked tool; count where some check `missed`. Display "n of m".	No stop request checked → not measured.	yes	count per 10k (scored, no factor)	—	Real, display only (can become a finding)
M42	Added within 10 business	same functions: of stop requests whose deadlines	None checked / no deadline passed → not measured.	yes	share	—	Real, display only

Code	Name (portal)	How it is measured (code)	Not-measured / worst-case rules	Saved	Scored as	Factor	Status
	days	passed, % with no tool missed; median days to mark.					
M44	Removed from list too early	`who.py:internal_area` (live): `CrmUpdate.cleared_optout=True` count	—	no	—	—	Live only
M45	Contacts after naming an attorney	`scoring/extra.py:attorneys` + `who.py:litigation_area`: people whose trusted AI review has `attorney_at`, contacted again later. Display "again of named".	AI review not running → not measured "Comes with AI review...". Demand letters not counted (complaint log has no letter type).	yes	count per 10k (scored, no factor)	—	Real when AI on
M47	Consent older than 12 months	`who.py:reassigned_area` (live): % of contacts whose proof time is >365 days earlier	—	no	—	—	Live only
M48	Contacts after "wrong number"	`scoring/extra.py:wrong_numbers` + `who.py:wrong_number_check`: contacts after an AI opt-out with keyword "wrong number".	AI not running → not measured.	yes	count per 10k (scored, no factor)	—	Real when AI on
M49	Registry copy older than 31 days	`who.py:dnc_area` (live): contacts whose latest DNC check is older than 31 days	needs DNC records	no	—	—	Live only

Notes on states (`scoring/snapshots.py:state_of`): `worst_case` if `worstCase`; `rule_pending` if value is None and the note contains "pending"; `not_measured` if `notMeasured` is set or display is "Not measured" with value None; otherwise `measured`. Owner rule: not built / not measured is "not yet measured", never treated as worst case; worst case only when the client did not supply data (M11, M12 today).

Flow

1. Portal asks `GET /api/metrics?period=YYYY-MM`.
2. If the period is given and a saved snapshot exists → `scoring.snapshots.saved_metrics` returns the saved rows, with a four-month `history` from saved months and `revisedOn` when the run is a revision.
3. Otherwise counts live: `conduct_metrics` (M16, M17, M18, M31, M19, M21, M15) + `overview_metrics` (M01, M10, M08, M09), each with a four-month history computed live.
4. Portal overlays these on its 33-metric sample by `code`.
5. `GET /api/conduct` → `{metrics: [M16,M17,M18,M31,M19,M20,M21,M15], byState, possibleLocationUncertain, hoursOfDay, byCampaign, abandonLimit}` for the chosen window.
6. `GET /api/who-contact` → areas `dnc`, `idn`, `rnd`, `lit` with checks `{state pass/warn/fail, name, result, source, metric, why}` and up to 50 flagged numbers (masked), plus `services` (what was uploaded) and `leadSources`.
7. `PATCH /api/who-contact/<ls-id>` {column, checked} saves a `lead_source_check` row and logs `lead_source_marked`.

Key functions

- `apps/metrics/api.py:MetricsView / ConductView / WhoContactView` — the three endpoints.
- `apps/metrics/conduct.py:span, months_before, contacts, Rules, outside` — window, history, contact set, draft rule cache, any-time-zone test.
- `apps/metrics/conduct.py:hours_check, frequency_check, holiday_check, robo_check, caller_id_check, cross_border, abandon_check` — M16, M17, M18, M19, M21, M15, M31.
- `apps/metrics/conduct.py:conduct_metrics, conduct_page` — metrics with history; page charts.
- `apps/metrics/overview.py:coverage, after_optout, overview_metrics` — M01, M10, plus M08/M09 from optouts.
- `apps/metrics/who.py:dnc_area, internal_area, stops_reached, wrong_number_check, litigation_area, reassigned_area, services, lead_sources, who_contact` — Who You Contact.
- `apps/optouts/measures.py:propagation_time, unprocessed, revocation_page` — M08, M09 (and Revocation page).
- `apps/optouts/marks.py:missed_optouts` — M38.
- `apps/vendors/contracts.py:terms_on_file` (M07), `apps/vendors/disputes.py:disputable_value` (M30), `apps/vendors/score.py:grade_mix` (M29).

API endpoints

- `GET /api/metrics?period=YYYY-MM` — any client user.
- `GET /api/conduct?period=` — any client user.
- `GET /api/who-contact` — any client user.
- `PATCH /api/who-contact/<ref>` — client Admin or company lawyer only (403 otherwise); logged.

Scheduled jobs

None of its own; saved monthly by `scoring.tasks.snapshot_metrics` (04:00 UTC).

Admin panel

None for this app.

Frontend pages fed

Overview (key numbers), Audit Scorecard (metric list), MetricDrawer, DomainDrawer, Contact Conduct (`useConduct` + `useMetrics`), Who You Contact (`useWhoContact`, WhoAreaDrawer).

Scoring (apps/scoring) — Module 10

Purpose

Every night, save a copy of every metric for each client and month, turn each into a 0–100 score, compute the six Exposure factors F1–F6, defect D, Exposure = D × Venue × Targeting × Volume with grade and caps, M36 evidence coverage with the coverage cap, and the portal's domain scores and Audit Score. Pages read the saved copies instead of counting again. One engine for all scores (owner decision 2026-10-21).

Data it uses

All metric producers above; `ContactEvent` (counts, states, channels for venue/volume); `ConsentResult` (hard cap); Rulebook keys `metric_score_bands`, `exposure_multipliers`, `hard_cap_share`, `grade_bands`, `coverage_cap_floor`, `coverage_cap_grade`; `findings` (projection).

Tables

- `snapshot_run` (`SnapshotRun`): `client, period` (first day of month), `version` (1,2,...), `ran_at, locked, revised, fingerprint` (SHA-256 of the sorted metric JSON), `contacts` (outbound primary non-test contacts in the month = base for per-10,000), `score` JSON `{factors: [{key,name,weight,value,pending,metrics,heldBy?}], defect, partial, coverage: {code:'M36',value,display,mix}, exposure:{venue,targeting,volume,caps: {hard,coverage},value,raw,clamped,grade,heldBy?}}`. Unique (client, period, version). Locked (RLS).
- `metric_snapshot` (`MetricSnapshot`): `client, run, code, value, display (≤80), state` (`measured | not_measured | worst_case | rule_pending`), `score` (0–100 or null), `data` (the metric JSON as the portal gets it). Unique (run, code).

Flow (nightly, step by step)

1. 04:00 UTC `snapshot_metrics` loops active clients inside `tenant(client)`.
2. `run_nightly`: for every month from the client's `start_date` to now (`months`, newest first) call `snapshot`. Sunday (`weekday()==6`) → `recheck_locked=True`.
3. `snapshot`: lock day = first day of next month + 5 days (`LOCK_DAYS = 5`).
    - Latest run is locked and not Sunday → skip.

- Locked + Sunday → `measure` again; same fingerprint → nothing; different → save a **new version** with `locked=True, revised=True` (old kept, never overwritten).
- Open (or never saved) → `measure` and overwrite the open run in place (its metric rows deleted and re-created); `locked` set when today ≥ lock day.

4. `measure(client, period)`: `conduct_metrics + overview_metrics + [M38] + extra_metrics (M12, M34, M11, M41, M42, M48, M45) + [M07, M30]`; drops `history`; adds `score` per metric (`scores.score` with state, month's contact base, bands for that month).
5. `scored`: `factors(metrics)` → F1–F6, D, partial; `coverage(metrics)` → M36; `cap(M36, period)` → coverage cap; `exposure(...)` → venue, targeting, volume, caps, value, grade. Stored in `snapshot_run.score`.
6. Same task then runs `vendors.score.save_month` (Module 12), `findings.build.refresh` (Module 11), `buyers.attestation.save` (Module 15) for the client.

Logic and rules

Metric score 0–100 (`scores.py`, 100 = best) — bands `[[limit, score], …]`: first limit the value is ≤ gives the score; above all → 0. Draft bands (`DRAFT_BANDS`, overridable by Rulebook `metric_score_bands`, status proposed, "Draft bands"):

- `count_per_10k`: [[0,100],[5,50]] — counts are converted to per 10,000 outbound contacts in the month: 0 → 100, ≤5 → 50, more → 0. If the month has 0 contacts: value 0 → 100, else None.
- `M08` [[24,100],[72,50]] (hours; "NEVER" → 0) · `M09` [[0,100],[0.1,50]] on notHonored/pastDeadline · `M31` [[1,100],[3,50]] (%) · `M32` [[5,100],[20,50]] · `M35` [[0,100],[5,50]] per 10k.
- Shares (higher better): `SHARE_GOOD = M01, M03, M04, M06, M07, M11, M25b, M27, M28, M42` → score = the % clamped 0–100.
- Counts: `COUNTS = M10, M12, M13, M15, M16, M17, M18, M19, M20, M21, M26, M34, M38, M41, M45, M48`.
- State rules: `worst_case` → 0 (owner 2026-10-12); `not_measured` / `rule_pending` → None (left out); anything else (e.g. M30, M29) → None.

Factors and D (`factors.py`, Exposure Indicator v2.2; higher = worse)

Factor	Name	Weight	Metrics
F1	Consent defensibility	0.28	M01, M03, M04
F2	Suppression integrity	0.20	M12, M13
F3	Revocation and opt-out	0.17	M08, M09, M10 (×2), M34 (×2)
F4	Calling conduct	0.16	M15, M16, M17, M18, M19, M20, M31
F5	Vendor and lead provenance	0.11	M06, M07, M25b, M26, M27, M28, M32
F6	Call integrity	0.08	M11, M21, M35

- Factor value = 100 − weighted mean of its scored metrics' scores (M10 and M34 weight 2), 1 decimal.

- Factor with no scored metric → `pending` (value None), left out of D; its weight is spread over measured factors (D = Σ value·weight / Σ measured weights); `partial = true` when any factor is pending.
- Hold rule: F3 held at 75 (if lower) when M10 is violated; F2 held at 75 when M12 is violated (violated = worst case or value > 0); sets `heldBy`. Also applies when the factor was pending (value None → 75).
- D = 0.28·F1 + 0.20·F2 + 0.17·F3 + 0.16·F4 + 0.11·F5 + 0.08·F6 (normalised over measured factors).
- Display-only metrics (M30, M38, M41, M42, M45, M48) are never in a factor.

Exposure (`exposure.py`) — `Exposure = min(100, D × Venue × Targeting × Volume)`, 1 decimal; `raw`, `clamped`. Grade (higher = worse): A < 25 · B 25–<40 · C 40–<49 · D 49–<67 · F ≥ 67. Draft multipliers (Rulebook `exposure_multipliers`, status proposed):

- **Venue** from `recipient_state` of the month's contacts. Venue states: FL, OK, MD, WA, TX, CT, VA, GA, NJ, UT, IN, MS, KY. Reached count 0 → 1.00, 1–2 → 1.10, 3–5 → 1.20, 6+ → 1.30; also 1.30 when ≥10% of contacts go to any heavy state (FL, OK, WA, TX, CT); cap 1.35. Returns states reached and heavy states.
- **Volume** from outbound primary non-test contacts in the 365 days before the month end: ≤100k 0.90 · ≤500k 1.00 · ≤1M 1.10 · more 1.20, or 1.30 when both calls and texts exist; cap 1.35.
- **Targeting** always 1.00 ("No lawsuit data yet"); `targeting_cap` 1.5 defined but unused. Waits for CourtListener.
- **Hard cap**: share of legal-basis checked contacts with `no_proof` or `conflicting` > `hard_cap_share` → `held`. `hard_cap_share` is status `tbd` (provisional 0.05, counsel item D8) → `require()` raises RulePending → the cap is `rule_pending` today. No checked rows → `not_measured`.
- **Coverage cap**: from M36 (below).
- Any cap `held` and grade better than C → grade set to C, `heldBy` lists the caps.
- D is None → exposure value/grade None (multipliers still returned).

M36 evidence coverage + coverage cap (`coverage.py`, Module 14, owner decision 2026-10-22)

- Each scored input gets a source grade: `SOURCE = {M07:D, M11:D, M12:D, M13:D, M01:C, M03:B, M04:A, M27:A, M28:A}`, default C (read by API); worst case → N. Grade values: A = B = C = 1.0, D = 0.7 (client uploads), S = N = 0 (statements / missing). No metric is mapped to S in code today.
- Weight of each input = factor weight × its share inside the factor (M10, M34 double). Not-measured / rule-pending inputs are outside coverage.
- M36 = Σ weight·value / Σ weight (4 decimals); `mix` = share of weight per grade. No scored input → not measured.
- Cap (`cap`): floor = Rulebook `coverage_cap_floor` (0.6, proposed), grade = `coverage_cap_grade` (C). M36 < floor → `held` (overall grade held at C); else `clear`; None → `not_measured`.

Portal scores (`portal.py`, higher = better)

- 25 domains, each = (family, metrics): CON (M01,M02,M03), WEB (M04,M05,M28), LPV (M06,M26,M27), VND (M07,M29,M30), AFF (M06,M26), RND (M11), IDN (M12), NDN (M13), SDN (M13), XBD (M15), DLR (M19,M31), PRV (M19,M20), SMS (M10,M16), CTF (M16,M17,M18), IDD (M21), SHK (M21), VLT (none), REV (M08,M09,M10), REC (M22a,M22b), RET (M23), SEC (none), GOV (M25a), INC (M24), PLT (none), MNA (M25b). Families: Permission, Who you contact, How you contact, Stopping, Records, Company controls.

- Domain score = mean of its scored metrics (domains with none are left out). Bars: pass ≥ 90, warn 60–89, fail < 60, notRun = unscored.
- Contact-after-stop cap (`STOP_CAP`): REV held by M10, IDN held by M12 (value > 0 or worst case) → score set to `grade_bands.C - 0.1` (grade D at best), `held: true`. (Rulebook `direct_violation_grade` = D describes this; code hard-codes D.)
- **Audit Score** = mean of measured domain scores excluding NDN and SDN (shown, never in the overall). Grade from Rulebook `grade_bands` (A 90 · B 80 · C 70 · D 60, else F; status set). `rawGrade` vs `grade`: grade held at C when the hard cap or coverage cap is held and raw is A/B.
- Score page also returns: checkpointsRun/Total, domainsMeasured/Total (25), family means, history (3 earlier saved months + this), `previous`, three caps with plain text (Hard cap, Contact-after-stop cap, Coverage cap), `partial {factorsMeasured, factors}`, `draftBands: true`, `exposure {value, grade, defect}`, `evidenceCoverage` (M36), `projected`, `cap`.
- **Projection** (Module 11 step 4): top 5 (`TOP=5`) unresolved ranked findings treated as fixed (their metrics score 100, value 0, no hold) → projected score & grade; still held at C by the hard cap unless M01 is among them (`clearsCap`). `impact(client, metric)` = Audit Score rise from fixing that one metric (used as "+8.3" on findings).
- `exposure_reading(client)`: latest run's factors (pending = null), venue, targeting, volume, source label "Measured from records · <Mon YYYY> · partial · draft bands" — used by `GET /api/insureds/<ref>` (reconcile app, `exposure.inside`) for the factor map and insurer pages.
- Month picker: `snapshots.periods(client)` → `/api/session` `periods` [{id 'YYYY-MM', label 'Mon YYYY', revisedOn?}], every month since start date, newest first.

Key functions

- `apps/scoring/snapshots.py:months, lock_day, state_of, measure, fingerprint, latest, scored, save, snapshot, run_nightly, saved_metrics, months_back, periods`.
- `apps/scoring/scores.py:bands, banded, score`.
- `apps/scoring/extra.py:internal_list, readded, reassigned_checks, stops, wrong_numbers, attorneys, extra_metrics`.
- `apps/scoring/factors.py:FACTORS, violated, factors`.
- `apps/scoring/exposure.py:settings, grade, venue, volume, hard_cap, exposure`.
- `apps/scoring/coverage.py:grade_of, coverage, cap`.
- `apps/scoring/portal.py:DOMAINS, cutoffs, grade_of, run_for, Fixed, domain_scores, overall, domains_page, score_page, projection, impact, exposure_reading`.
- `apps/scoring/tasks.py:snapshot_metrics` — nightly task.

API endpoints

- `GET /api/score?period=YYYY-MM` — any client user; latest saved month when no period; `{}` when nothing saved.

- `GET /api/domains?period=` — any client user; list of domain objects with 4-point `history`; `[]` when nothing saved.
- (Indirect) `GET /api/metrics?period=` reads saved months; `GET /api/session` gives `periods`; `GET /api/insureds/<ref>` gets `exposure.inside`.

Scheduled jobs

- `metric-snapshots` → `apps.scoring.tasks.snapshot_metrics`, daily 04:00 UTC (locked months re-checked Sundays). Also runs vendor scorecards, findings refresh, risk attestation save.

Admin panel

No admin pages for `snapshot_run` / `metric_snapshot` (none registered in `apps/scoring`).

Frontend pages fed

Overview (score, grade, caps, projection, history), Audit Scorecard (`useScore`, `useDomains`, `useMetrics` with month picker), DomainDrawer, Action Queue (score for projection), insurer pages via `exposure_reading`.

Findings (apps/findings) — Module 11 steps 1, 2, 4

Purpose

Turn the latest saved month's metrics into plain-word "findings" (one per client per problem), let the client's Admin or lawyer assign, date, decide on and dispute them, and show a CiV draft "common response" for their lawyers to review.

Data it uses

Latest `SnapshotRun` + its `metric_snapshot` rows; example numbers from `ContactEvent`, `ConsentResult`, `InternalListEntry`, `OptOutEvent`, `ConversationReview`, `ReaddedNumber`; `contacts.api.number_ref` for number-page links.

Tables

`finding` (`Finding`), unique (client, key), locked (RLS):

- `key` (metric code, or `<code>-worst` for missing data), `metric, domain, severity` (High/Medium/Low), `title` (noun phrase), `why` (≤400), `affected` (count for ordering), `numbers` (distinct numbers), `examples` [{label, ref}] ≤5, `period` (month last found), `status` (`open | resolved | reopened`), `first_seen, last_seen, resolved_at`.
- Client work fields: `decision` (`accepted | declined | alternative`), `decision_note`, `note_sha256`, `decided_by`, `decided_at`, `assignee` (ClientUser), `due`, `work_status` (`open | in_progress | resolved`), `dispute_note`, `dispute_by`, `dispute_at`.

Catalog (catalog.py)

Metric	Title	Severity	Domain	Unit
M10	Contacts after an opt-out	High	REV	contact

Metric	Title	Severity	Domain	Unit
M12	Contacts after joining your internal do-not-call list	High	IDN	contact
M48	Contacts after a “wrong number” reply	High	RND	contact
M45	Contacts after naming an attorney	High	LIT	person
M34	Opted-out numbers brought back by an uploaded list	High	REV	number
M16	Out-of-hours contacts	Medium	CTF	contact
M18	Contacts on restricted holidays	Medium	CTF	contact
M01	Contacts without consent proof that covers them	Medium	CON	contact
M08	Slow opt-out handling across your tools	Medium	REV	—
M09	Opt-outs not handled by the deadline	Medium	REV	opt-out
M41	Stop requests that never reached every tool	Medium	REV	stop request
M42	Stop requests added to your tools late	Low	REV	—
M15	Contacts to numbers outside the United States	Low	XBD	contact
M17	Contacts over state frequency limits	Low	CTF	contact
M19	Robocalls or robotexts without the proof their category needs	Low	DLR	contact
M31	Abandoned-call rate above the limit	Low	DLR	—
M11	Contacts without a recent reassigned-number check	Low	RND	—

Worst-case findings (severity Medium): `M12-worst` "Internal do-not-call list not supplied" (IDN), `M11-worst` "Reassigned-number query logs not supplied" (RND), each with an Upload-centre "why". `SKIP = {M38}` (same as M41).

Common responses (`RESPONSES`): one CiV draft per metric and per worst-case key, worded "Companies commonly…", never "you must"; shown as `fixStep` with `fixStepDraft: true` ("Draft · for your lawyers to review"; no CiV lawyer — clients' lawyers review).

Flow

1. Nightly after snapshots (`scoring.tasks.snapshot_metrics` → `findings.build.refresh`).
2. `found_in`: for each metric row of the latest run (by period desc, version desc):
   - worst_case and in `WORST` → `<code>-worst` finding, affected 0, no examples;
   - measured, score < 100, in `MEASURED` → finding with examples (`examples.flagged`), `affected = int(value)` when the metric has a unit, `why` = "CiV found N units (M numbers) in <Month YYYY>." + the metric note;
   - never for not measured / rule pending.
3. `refresh`: create new rows; update existing ones in place (`last_seen`, `period`); a `resolved` row found again → `reopened`. Rows not found any more → `resolved` with `resolved_at`. Returns count open.

4. `ranked`: unresolved first, then severity (High, Medium, Low), then most affected, then title.
5. `GET /api/actions`: each row with rank, impact (`+X.X` score gain from `scoring.portal.impact`, else "N contacts", "Missing data" or "—"), fix step, assignee, due, overdue (due < today and not resolved), status (`resolved` when CiV closed it, else `work_status` — CiV's own close wins), dispute, decision, metric, examples, numbersAffected, since, reopened, resolvedOn.
6. `PATCH /api/actions/f-<id>` changes any of: `assignee` (name of an active user of the same client), `due` (YYYY-MM-DD or empty), `status` (open/in_progress/resolved), `decision` {choice, note} (note SHA-256 saved), `dispute` {note} (required). Logged as `finding_changed`; the log carries only the first 16 chars of the note fingerprint, never the note. Decision notes stay in the client's workspace and are never sent to insurers.

Examples (`examples.py`, up to 5, masked label + number-page ref, never the number): M10 (`NP_AFTER_OPT_OUT`), M01 and M19 (legal-basis results with no_proof/conflicting — note M19 uses the same filter as M01), M12, M48, M45, M15, M16 (first 5,000 contacts scanned), M34 (latest contact of each re-added number). Others → no examples.

Key functions

- `apps/findings/build.py:found_in, refresh, ranked, plural`.
- `apps/findings/examples.py:shown, flagged`.
- `apps/findings/catalog.py:MEASURED, WORST, SKIP, RESPONSES`.
- `apps/findings/api.py:row, can_decide, ActionsView, ActionView`.

API endpoints

- `GET /api/actions` — any client user (no lawyer-only mode since 2026-10-22).
- `PATCH /api/actions/f-<id>` — Admin or company lawyer only (403 otherwise).

Scheduled jobs

Inside `metric-snapshots` (04:00 UTC daily).

Admin panel

None registered for `finding`.

Frontend pages fed

Action Queue (`/actions`, ActionsScreen + ActionDrawer PATCH), Overview "Things to fix", Sidebar badge (actions count), DomainDrawer and MetricDrawer (related actions).

Alerts (apps/alerts) — Module 11 step 3

Purpose

Every 5 minutes collect what needs a person to look now into alerts. **Portal only, no email** (owner 2026-10-16; the Alerts page's email box says "Off · Alerts are shown in the portal only").

Data it uses

`optouts.MarkCheck` (unmarked with `urgent_at`), `consent.ConsentResult` (`NP_AFTER_OPT_OUT`), `optouts.ReaddedNumber`, `captures.ConsentPage` + `captures.api.wording_changed_on`, `certificates.CertificateClaim` (`cert_expires_at`), `connections.Connection` (status STALE), `reconcile.PeriodIntegrity` (incomplete), `evidence.RefetchCheck` (result changed), `vendors.score.drops` (vendor score drop ≥ `vs_alert_drop_points`, default 10).

Tables

`alert` (`Alert`), unique (client, key), locked: `key` (≤80), `kind, severity` (High/Medium/Low), `title, detail` (≤400), `domain, count, first_at, at` (last raised/updated), `resolved_at, reviewed, reviewed_by, reviewed_at`. Ordering newest `at` first. Never deleted.

Alert kinds (raise_.gather)

Key	Kind	Severity	Domain	Lasting?	Trigger
`unmarked-<optout>-<conn>`	Opt-out not marked	High	REV	yes	MarkCheck marked=False with `urgent_at`
`after-optout-<day>`	Contacts after opt-out	High	REV	no	strict results with NP_AFTER_OPT_OUT, contacts in last 7 days, one per day (count, numbers)
`readded-<day>`	Opted-out numbers re-added	High	REV	no	ReaddedNumber found in last 7 days, per day
`page-<id>-<yyyymmdd>`	Consent page changed	Medium	WEB	no	wording changed within 7 days
`certs-expiring`	Certificate expiry	Medium	CON	yes	claims (not gone) expiring within 30 days
`stale-<conn>`	Source stale	Medium	REC	yes	connection status STALE
`incomplete-<YYYY-MM>`	Incomplete data	Medium	REC	yes	PeriodIntegrity incomplete (shows completeness %)
`refetch-changed-<ISO year-week>`	Records changed at the source	Medium	REC	no	RefetchCheck changed in last 7 days (Module 14)
`vendor-drop-<key>-<sub>-<YYYY-MM>`	Vendor score drop	Medium	LPV	no	vendor fell ≥10 points vs last month (Module 12)

`RECENT = 7 days`: older events raise nothing (e.g. on first sync). No Low alerts exist in code.

Flow

1. Every 300 s `raise_alerts` loops active clients in `tenant(client)` → `refresh`.
2. New key → create (`first_at = at` = event time or now). Existing key with any changed field, or previously resolved → update fields, `at = now`, `resolved_at = None` (reopens).

3. Open alerts no longer found: only lasting prefixes (`unmarked-`, `certs-expiring`, `stale-`, `incomplete-`) get `resolved_at = now`; events stay until reviewed.
4. Returns count of unreviewed, unresolved alerts.

API endpoints

- `GET /api/alerts` — any client user. Up to 500 rows: id `al-<pk>`, severity, kind, title, detail (+ "Resolved <d Mon>."), at, domain, reviewed, resolved, reviewedBy.
- **Alert emails (2026-10-23, `apps/alerts/emails.py`):** when an alert first appears, or comes back after being resolved, the company's active **Admins only** get one email (title, detail, severity, "Open Alerts" button) if the Admin chose email for that type in Settings → Notifications (`alert_email_choice`; defaults all on except Vendor score drop; `PATCH /api/settings/email/notifications {kind, email}`, Admin only). `alert.emailed_at` records it; a mail problem never stops alerts.
- `POST /api/alerts/al-<id>/review` — anyone who can see it; sets reviewed/by/at once; logged `alert_reviewed`.

Scheduled jobs

`alerts` → `apps.alerts.tasks.raise_alerts`, every 300 seconds.

Admin panel

None registered.

Frontend pages fed

Alerts page (`/alerts`, AlertsScreen, AlertRow "Mark as reviewed"), Overview alerts list, Sidebar badge. Also listed in the monthly report PDF.

Reports (apps/reports) — Module 11 step 5

Purpose

Make, keep and serve client reports: the monthly **Period audit report** PDF (reportlab), the **Rulebook snapshot** PDF, and the **Metric export** CSV. Other report types are recorded as "not available yet". Every report text passes a banned-word check before release.

Data it uses

`scoring.portal.score_page`, latest `SnapshotRun` for the month, `findings.build.ranked`, `alerts.Alert` (first raised in the month), `rulebook.snapshot.snapshot_pdf`, `evidence.holds.Holds` (legal holds).

Tables

`client_report` (`Report`), locked: `client, name, period` (first day of month), `status` (`ready | blocked | unavailable`), `file` (private, `client-reports/<client>/<uuid>.pdf`), `sha256` (of the file), `content_sha256` (of the figures, printed in the footer), `banned` (words that blocked it), `made_by` (user name or "CiV (automatic)"), `made_at, purged_at`.

Report types

- `Period audit report` (PDF) — made here.
- `Rulebook snapshot` (PDF) — made here (Module 2 snapshot, kept in history).
- `Metric export` (CSV) — columns `metric, value, shown, state, score_0_100, note` from the month's saved metrics; no snapshot → only the header row.
- Any other name (Evidence file export, Certification pack, Litigation evidence package) → a row with status `unavailable`. (The evidence file is built elsewhere in Module 14; `evidence/api.py` reuses `banned_words`.)
- The list of types itself stays from the portal sample (`frontend/src/mock/data/disclosure.ts: reportTypes`).

Period report content (period.py)

`figures` (also the fingerprinted data): client name, month, Audit Score + grade, partial, family scores, caps, F1–F6 values ("Pending"), D, exposure (value, grade, D × venue × targeting × volume), every finding (severity, title, why, domain, status Open/Resolved, decision label — **never the note**, "PDFs get forwarded"), alerts first raised in the month (date, severity, title, reviewed), codes not yet measured / rule pending, revised flag. Footer on every page: "Figures fingerprint (SHA-256): ...". Ends with the disclaimer: "Comply iV measures and records. It operates no controls, sits in no call path and makes no compliance decisions. Findings are measurements, not legal advice. Comply iV is not a law firm." Letter size, teal header tables.

Banned-word check (words.py)

Regex, whole words, case-insensitive: violation(s), breach(es), illegal, compromised consent, fake lead(s), fraud*, genuine, non-compliant / noncompliant, ensure(s) compliance, protects you, guarantee(d), scrub/scrubbed/scrubbing, screen/screening, court-ready, you must, fix step(s), leads to claim back, stored write-once. Checked against the JSON of all figures + disclaimer. A hit → report saved as `blocked` with the words, logged `report_blocked`, no file.

Flow

1. On the 6th at 06:00 UTC `monthly_reports`: for every active client (in tenant) `latest_or_make(client, last month, 'CiV (automatic)')` — reuses a ready, unpurged period report or makes one. Then (all clients) `purge_old`.
2. `make_period`: figures → banned check → `build` (content SHA-256 + PDF) → save file + file SHA-256 → log `report_made`.
3. Generate from portal: `POST /api/reports {name, period}` (period as `YYYY-MM` or `Mon YYYY`) → make by type.
4. Download: `GET /api/reports/period?period=YYYY-MM` (makes if missing; 409 "held back for a wording check" when blocked) or `GET /api/reports/rep-<id>/file`. Each download logged `report_downloaded`; `Cache-Control: private, no-store`; file names `audit-report-YYYY-MM.pdf`, `rulebook-snapshot-YYYY-MM.pdf`, `metrics-YYYY-MM.csv`.

5. Keep 4 years (`KEEP = 4×365 days` from `made_at`), then file deleted, `purged_at` set, fingerprints kept; skipped while a legal hold (Module 14) covers the client and period/date.

Key functions

- `apps/reports/make.py:make_period, latest_or_make, last_month, purge_old, make_rulebook, make_metrics`.
- `apps/reports/period.py:figures, text_of, table, render, build`.
- `apps/reports/words.py:banned_words`.
- `apps/reports/api.py:row, month_from, send, ReportsView, PeriodReportView, ReportFileView`.

API endpoints

- `GET /api/reports` — any client user; `{runs: [...]}` (last 100) with id `rep-<pk>`, name, period, requestedBy, at, status (ready/blocked/unavailable), url, sha256.
- `POST /api/reports` — any client user (no Admin check in code).
- `GET /api/reports/period?period=` — any client user.
- `GET /api/reports/rep-<id>/file` — any client user, own client only, ready and unpurged.

Scheduled jobs

`monthly-reports` → `apps.reports.tasks.monthly_reports`, 06:00 UTC on day 6 (after the month locks on day 5+).

Admin panel

`ReportAdmin` (Admin → Records → "Client reports"): read-only list (made_at, client, name, period, status, banned, made_by), filters status/client; no add/change. A report's page has a **Download** button (`/civ-admin/reports/report/<id>/download/`, logged `report_downloaded` by staff; files are private, no public link) and shows "Banned words: ..." in plain words. Purpose: staff see blocked reports and fix the wording; the client then makes it again.

Frontend pages fed

Reports & Exports (`/reports`: ReportsScreen, ReportCard Generate, BulkExport), Evidence file screen (POST of an "Evidence file export" name → recorded unavailable). Rulebook card downloads directly from `/api/rulebook/snapshot`.

Litigation (apps/litigation) — Module 10 step 4b

Purpose

Industry-wide TCPA lawsuit numbers for the insurer's Litigation Intelligence page, read from WebRecon's public monthly litigation-statistics blog posts (no key; robots.txt allows). Not about any client. Company-level lawsuits (who was sued) come later from CourtListener (owner decision 2026-10-21).

Data it uses

`https://webrecon.com/sitemap.xml` → post URLs matching `https://webrecon.com/litigation-statistics/webrecon-<month>-<year>…`. User-Agent "CiV litigation reader (Comply iV; reads the monthly posts)". Timeout 30 s, max 2 MB per page.

Tables

`litigation_month` (`LitigationMonth`, public, not locked): `month` (unique, first day), `source` ('webrecon'), `url`, `tcpa` (TCPA suits filed in month), `tcpa_class` (putative class actions), `tcpa_ytd`, `change_month` (%), `change_ytd` (%), `repeat_plaintiffs` (% of all plaintiffs, all statutes, who sued before), `read_ok` (False = post found but TCPA line unreadable), `fetched_at`, `page_sha256`.

Flow

1. Mondays 05:30 UTC `read_webrecon` → `refresh`.
2. Read sitemap; map month → URL (when a month has two posts keep the longer URL, e.g. year in review).
3. To read: months never read OK, plus the newest two again (corrections); at most 24 per run (`PER_RUN`), 1 s apart.
4. Each page → strip HTML → regexes: "N TCPA, M Class Action"; or "N TCPA" not preceded by "Year to Date"; "Year to Date … N TCPA"; "TCPA +x%" (month change); "TCPA (+x%)" (YTD change); "x% of all plaintiffs … sued/filed … before".
5. `update_or_create` by month; `read_ok = 'tcpa' found`; on any fetch error the row is saved with empty figures (`read_ok=False`) so a site redesign shows up.

API

`GET /api/litigation` — client users **or** insurer (buyer) users; buyer answers go through `buyers.portal.as_buyer`. Returns `{}` before any post is read OK; else `headline` cards (merged by id over the sample): `filings-year` ("Federal TCPA filings, YYYY (so far)", YTD, % vs last year), `class-month` ("Class actions filed, Mon YYYY", share of the month's TCPA filings, vs same month last year), `repeat-plaintiffs`; plus `classByYear` (same calendar month over up to 4 years, needs ≥2), `classMonth`, `classSource` ("Source: WebRecon monthly statistics, read …"). Each card: status Verified, source WebRecon, url. Costs, claim types and the study stay sample.

Key functions

- `apps/litigation/webrecon.py:fetch, posts, text_of, number, read, refresh`.
- `apps/litigation/api.py:page, LitigationView`.
- `apps/litigation/tasks.py:read_webrecon`.

Scheduled jobs

`webrecon-litigation`, Mondays 05:30 UTC.

Admin panel

None registered.

Frontend pages fed

Litigation Intelligence (`/litigation`, `screens/insurer/LitigationScreen.tsx`, `useLitigation`) — an insurer-only path (`INSURER_ONLY_PATHS`, Underwriter view).

Portal screens → endpoints

Portal page (route)	Endpoints	Notes
Overview (`/`)	`GET /api/score?period`, `/api/metrics?period`, `/api/actions`, `/api/alerts`	score, grade, caps, projected, key numbers (M01, M10, M08, M09…), things to fix, alerts
Audit Scorecard (`/scorecard`) = Score + Domains + Metrics	`GET /api/domains?period`, `/api/score?period`, `/api/metrics?period`	month picker from `/api/session periods`; DomainDrawer also `/api/actions`; MetricDrawer `/api/metrics` + `/api/actions`
Action Queue (`/actions`) = Findings	`GET /api/actions`, `GET /api/score`, `PATCH /api/actions/f-<id>` (ActionDrawer)	
Alerts (`/alerts`)	`GET /api/alerts`, `POST /api/alerts/al-<id>/review`	full backend (no sample merge)
Reports & Exports (`/reports`)	`GET /api/reports`, `POST /api/reports`, `GET /api/reports/period?period`, `GET /api/reports/rep-<id>/file`	types from sample; Rulebook card uses `/api/rulebook/snapshot`
Contact Conduct (`/conduct`)	`GET /api/conduct`, `GET /api/metrics?period`	
Who You Contact (`/who-contact`)	`GET /api/who-contact`, `PATCH /api/who-contact/<id>`	
Litigation Intelligence (`/litigation`, insurer view)	`GET /api/litigation`	
Sidebar badges	`GET /api/actions`, `GET /api/alerts`	

There is no separate "Domains" or "Metrics" page in the portal nav: domains and metrics live on the Audit Scorecard and in drawers.

Tests present

`tests/scoring` (coverage, exposure, factors, portal, scores, snapshots), `tests/findings` (alerts, decisions, findings, projected), `tests/reports/test_period_report.py`, `tests/litigation/test_webrecon.py`, `tests/metrics` (conduct, who).

8. Vendors, evidence vault, insurer views and document AI (Modules 12, 14, 15, 16)

Read from the code on 22 October 2026. Backend = `/home/shakib/Audit/backend`, frontend = `/home/shakib/Audit/frontend/src`. Module numbers follow the backend's own numbering (12 vendors, 14 evidence, 15 buyers, 16 docai).

Shared facts used by all four areas:

- **Row-level security (RLS).** `apps/tenancy/sql.py:lock_table(table)` turns on Postgres RLS with policy `tenant_only`: a row is visible only when `civ.scope = 'all'` or `client_id = civ.client_id`. `lock_buyer_table(table)` (policy `tenant_or_buyer`) also lets a row through when `civ.scope = 'buyer'` and `org_id = civ.buyer_id`.
- **Scopes.** `apps/tenancy/scope.py`: `tenant(client)`, `all_clients()`, `buyer(org)`. `apps/tenancy/middleware.py:TenantScopeMiddleware` sets them per request: `/civ-admin/` with a staff login → all; `/api/` with a client user → tenant; `/api/` with a buyer session → buyer scope; anyone else → no scope (locked tables show nothing).
- **Access log.** `apps/access_log/recorder.py:record(action, …)` writes an `AccessEntry`; labels in `apps/access_log/labels.py` (e.g. `attestation_viewed` → "Insurer opened your sheet").
- **Client roles.** `IsClientUser` (`apps/accounts/permissions.py`); inside views `user.is_admin` and `user.is_counsel` ("the company's lawyer") gate writes.
- **Frontend proxy.** `frontend/src/app/api/[...path]/route.ts` + `server/backend.ts`. `BACKEND_PATHS` (forwarded straight to Django) include `vendor-contracts, vendor-disputes, evidence, evidence-file, evidence-bundle, legal-holds, buyer-links, statements, rating-engine, position, hash-check`. `MERGE_PATHS` (GET only; Django's real fields laid over sample data with `overlay()`) include `vault, vendors, insureds, litigation, settings`. A reply carrying header `X-CiV-Viewer: buyer` is passed through untouched (no sample mixed in), except `litigation`, which is overlaid on the sample. `SAMPLE_WHEN_REFUSED = ['rating-engine']`: a client who gets 403 from Django on `/rating-engine` gets the sample instead (client "trying" the underwriter view).

Vendor scorecard (Module 12) — apps/vendors

Purpose (plain words)

Measures every lead vendor (the companies that sell leads to the client) on the same six "problem rates", turns them into a 0–100 score and A–F grade, saves the score monthly so trends and sudden drops show, lists the leads the client may send back to the vendor for credit ("dispute candidates"), and keeps the vendor's contract on file (private, logged, deleted after 4 years). Also counts M07 (vendor terms on file), M29 (grade mix) and M30 (disputable lead value).

Data it uses, and from where

- **Leads** (`rates.py:all_leads`): two sources, merged into one shape `{vendor, sub_id, created_at, phone_tokens, price, ref}`:

1. `imports.CrmCreation` rows with a non-empty `vendor` (CRM records, Module 8). `ref` = `"<source_system>:<record_id>"`. Price from `crm_creation.price` (CRM fields `lead_cost`, `price`, `cpl`… per README).
2. `vendors.VendorLead` rows from uploaded **vendor lead files** (Upload centre source `vendor_leads`, parsed in `apps/uploads/read.py`). `ref` = vendor's lead ID or `"upload <id> row <n>"`.

- **Contacts**: `contacts.ContactEvent` (outbound, `is_primary=True`, `is_test=False`) matched by `phone_token`.
- **Consent results**: `consent.ConsentResult` (variant `strict`, not frozen, status not `pending`/`not_needed`) — statuses and `codes`.
- **Consent time**: `consent.ConsentProof` with `captured_at` and `date_known=True`.
- **Stops**: `optouts.OptOutEvent` (`counts=True`) and `uploads.Complaint` (`received_on`).
- **Settings**: Rulebook (`rulebook/v3.0.yaml` via `RuleSet().get(key, on=date)`), fallback `DRAFT` dict in `rates.py`:
  - `vs_window_days` 90, `vs_max_delivery_hours` 24, `vs_stop_window_days` 30, `vs_min_leads_for_score` 100, `vs_alert_drop_points` 10,
  - `vs_rate_weights` [0.35, 0.30, 0.15, 0.10, 0.05, 0.05] (fallback `score.WEIGHTS`), `vs_grade_bands` A 90 / B 80 / C 70 / D 60 (fallback `score.BANDS`),
  - `vendor_active_days` 90, `default_dispute_window_days` 0, `vendor_required_clauses` (status **proposed**, counsel item A15b, `counsel_approved: false`; 12 clauses, 5 required: consent, certificate, proof_on_request, optouts_passed, indemnity).

Tables

- `vendor_month` (`VendorMonth`, RLS `lock_table`): `client`, `period` (1st of month), `key` (vendor name lower-cased, single spaces), `sub_id` ('' = vendor row), `name`, `leads`, `rates` (JSON rate key → 0–1 or null), `counts` (JSON), `score` (0–100 or null), `grade` (1 char), `measured` (how many of the 6 rates measured), `saved_at`. Unique (`client, period, key, sub_id`).
- `vendor_contract` (`VendorContract`, RLS): `client`, `key`, `name`, `file` (stored at `vendor-contracts/<client_id>/<uuid><ext>`), `original_name`, `sha256`, `ends_on`, `dispute_days`, `clauses` (JSON clause id → bool, filled by Document AI), `uploaded_by`, `uploaded_at`, `purged_at`. Newest upload per vendor counts.
- `vendor_lead` (`VendorLead`, RLS): `client`, `upload` (FK `uploads.UploadedFile`), `row`, `lead_ref`, `phone_token` (indexed; never the number), `vendor`, `sub_id`, `received_at`, `price`. Index (`client, received_at`). Kept apart from CRM records so it never forms a CRM import batch.

Flow step by step

1. **Leads arrive**: CRM sync fills `crm_creation.vendor/sub_id/price`; or an Admin uploads a vendor lead file (CSV columns phone, vendor, sub-ID, received date, price, lead ID; column names matched loosely in `uploads/read.py`) → one `VendorLead` per valid row.

2. **Consent links in lead files** (`lead_certificates.py:link_certificates`): if a row has a certificate column (`certificate`, `trustedform_url`, `xxtrustedformcerturl`, `leadid_token`, `jornaya`, ...), the link (or a bare Jornaya token) becomes a `CertificateClaim` (deadline = received_at + claim window), the phone is matched to the claim while in memory (`certificates.claim.match_now`), and a `ConsentProof` (kind CERTIFICATE, `proof_type='web_cert'`, `source_system='vendor_lead_file'`, `source_hint='VENDOR_LEAD_FILE'`) is saved.
3. **Rates** (`rates.py:vendor_rates`) over the last 90 days, per vendor and per vendor+sub-ID:
   - a lead is "called" if a primary outbound contact to its phone code happened at/after the lead's creation and has a strict consent result;
   - **consent_problems** = called leads with any result `no_proof` or `conflicting` ÷ called leads;
   - **consent_gaps** = called leads with any code in {`NP_NO_CONSENT_TEXT`, `NP_PRETICKED`, `WK_WORDING`} ÷ called leads;
   - **late_delivery** = leads whose CRM creation was > 24 h after the latest known consent time (consent before creation) ÷ leads with a known consent time;
   - **stops** = leads with an opt-out or complaint within 30 days of the first contact ÷ leads whose 30-day window has finished;
   - **high_signals** and **name_match** = always null ("not measured yet", need Lead Inspector signals);
   - a rate with zero denominator = null (left out). `measured` counts non-null of the 4 measurable rates.
4. **Score** (`score.py:score_of`): penalty = $\Sigma\, w_i \cdot r_i$ over measured rates ÷ $\Sigma\, w_i$ of those rates (weights re-normalised); score = clamp(100 × (1 − penalty), 0, 100), 1 decimal. Under 100 leads → score null ("not enough data"). **Grade** (`grade_of`): A ≥ 90, B ≥ 80, C ≥ 70, D ≥ 60, else F.
5. **Nightly save** (`score.py:save_month`): in `apps.scoring.tasks.snapshot_metrics` (04:00 UTC), for every active client: delete this month's `vendor_month` rows, recreate them. Earlier months stay untouched.
6. **Trend** (`history`): last 6 monthly scores (None where missing).
7. **Drop alert** (`drops`): this month's score vs last month's; a fall ≥ 10 points → in `apps/alerts/raise_.py:gather` raises alert key `vendor-drop-<key>-<subId>-<YYYY-MM>`, kind "Vendor score drop", severity MEDIUM, domain LPV, not lasting.
8. **Contracts**: Admin or lawyer uploads on the Vendor Ledger → `VendorContract` with sha256; logged `vendor_contract_added`. Opening the file is logged `vendor_contract_opened`, served `as_attachment` with `Cache-Control: private, no-store`.
9. **Dispute candidates** (`disputes.py:candidates`): leads of the last 400 days; per vendor a window = the unexpired contract's `dispute_days` (basis "contract"), else Rulebook `default_dispute_window_days` (0, basis "no contract on file"); window 0 → none. A lead counts if `created_at + window ≥ today` and a contact to it has strict result `no_proof`/`conflicting`. Row = lead ref, vendor, sub-ID, date, price, result ("No proof"/"Conflicting"), reason text (from `consent.words.REASONS`), masked number, number ref. No-price leads counted in `noPrice`, left out of the total.
10. **Ledger page** (`portal.py:vendors_page`) assembles everything (see Frontend).

11. **Purge** (`contracts.py:purge_old`, task nightly 04:50): contract files older than 4×365 days deleted (fingerprint kept, `purged_at` set) unless a legal hold covers the upload date or the vendor key.

Logic and rules

- Vendor key = `' '.join(name.lower().split())`; sub-ID rows are extra rows, never mistaken for the vendor.
- Before the first nightly save of the month, the page works rates/scores out on the spot (not saved).
- **M07 Vendor terms on file** (`contracts.py:terms_on_file`): active vendors (lead in last 90 days) with an unexpired newest contract ÷ active vendors. "Not measured" if no vendors ("No vendor named in your CRM records") or no contract uploaded ("Contracts not supplied"). Note adds "clauses not checked yet (needs the AI reader)" or, when Document AI filled `clauses`, "n of m checked have every required clause". Saved monthly via `scoring/snapshots.py`.
- **M29 grade mix** (`score.py:grade_mix`): vendor rows only (not sub-IDs), count per grade, lowest named, count with not enough data.
- **M30 Disputable lead value** (`disputes.py:disputable_value`): sum of prices of all dispute candidates; note gives counts, no-price leads, vendors with no contract.
- Contract upload rules: Admin or counsel only; vendor + file required; extensions `.pdf .doc .docx .png .jpg .jpeg .txt`; ≤ 20 MB; `endsOn` YYYY-MM-DD; `disputeDays` integer.
- CiV never contacts vendors; dispute CSV is for the client to send.
- Real clients never see sample vendors: with no vendors the page says "Not measured" / "No vendors yet".

Key functions/classes

- `apps/vendors/rates.py:setting` — Rulebook value or DRAFT fallback.
- `apps/vendors/rates.py:vendor_key` — normalise vendor name.
- `apps/vendors/rates.py:all_leads` — CRM + lead-file leads in one shape.
- `apps/vendors/rates.py:vendor_rates` — the six rates per vendor and sub-ID.
- `apps/vendors/score.py:score_of / grade_of` — score and grade.
- `apps/vendors/score.py:save_month` — nightly monthly snapshot.
- `apps/vendors/score.py:history / drops / grade_mix` — trend, drop alert, M29.
- `apps/vendors/contracts.py:latest / state / active_vendors / terms_on_file / purge_old` — contracts, M07, 4-year purge.
- `apps/vendors/disputes.py:window_days / candidates / disputable_value` — dispute list and M30.
- `apps/vendors/lead_certificates.py:claims_for / link_certificates` — certificate links in lead files.
- `apps/vendors/portal.py:current_rows / vendors_page` — Vendor Ledger payload.
- `apps/vendors/api.py:ContractsView / ContractFileView / DisputesView` — client API.

- `apps/provenance/api.py:VendorsView` — `GET /api/vendors` wrapper; `vendor_id(name)` = `'v-' + sha256[:10]`.

API endpoints

- `GET /api/vendors` — Vendor Ledger payload (`provenance.api.VendorsView`), any client user.
- `GET /api/vendor-contracts` — up to 200 contracts, newest first, any client user.
- `POST /api/vendor-contracts` — multipart `vendor, file, endsOn, disputeDays`; Admin or lawyer; 201 with row; logged.
- `GET /api/vendor-contracts/vc-<id>/file` — download (404 if purged); any client user of that company; logged.
- `GET /api/vendor-disputes?vendor=<name>` — dispute candidates of one vendor (404 if no leads); `&csv=1` → CSV `dispute-candidates-<key>.csv` (columns lead, vendor, sub_id, date, price, result, reasons, number), logged `vendor_disputes_downloaded`. Any client user.
- Contract row shape: `{id 'vc-<pk>', vendor, file, endsOn, disputeDays, state none|on_file|expired, uploadedBy, uploadedAt, url|null}`.

Scheduled jobs

- `apps.scoring.tasks.snapshot_metrics` — daily 04:00 UTC: includes `save_month(client)` per active client.
- `apps.alerts.tasks.raise_alerts` — every 300 s: includes vendor drop alerts.
- `apps.vendors.tasks.purge_old_contracts` — daily 04:50 UTC.
- `apps.uploads.tasks.purge_old_uploads` — daily 04:55 UTC: deletes vendor lead files after 4 years, skipping held numbers/vendors/dates.

Admin panel parts

- None registered for `vendors` (no `admin.py`). The `vendor_required_clauses` draft is a Rulebook rule update approved in the Rulebook/Reg Changes admin (data migration `regulatory/0008`, per README).

Frontend pages fed

- **Vendor Ledger** `/vendors` (`screens/vendors/VendorsScreen.tsx`, `useVendors()` → `/vendors`, a MERGE path): KPIs Grades (gradeMix/gradeMixNote), Dispute candidates, Score-drop alerts (alertNote); "No vendors yet" note; "Vendor scores" bar list; `VendorTable` (columns Vendor/sub-ID, Leads, High-signal ["Not measured yet"], Score ["Not enough data"], Grade, Month on month + "Alert" pill, trend sparkline (header says "4 months", data is 6), Signed terms "On file/Not on file" + Contract/Add contract button, Dispute candidates); `AddContractCard` (list + upload via `apiUpload('/vendor-contracts')`, Admin/lawyer).
- **Vendor drawer** (`components/drawers/VendorDrawer.tsx`): `ContractForm` (upload/see contract), `DisputeList` (`useDisputes` → `/vendor-disputes?vendor=`, only when `vendor.contract !== undefined`, i.e. a real backend vendor; CSV link; each number links to `/numbers/<ref>`), the rates bar list with "Partial: n of 6 rates measured. Not measured yet: …", score trend line.

- Vendor row payload: `{id, name, subId, leads, highSignals:null, score, grade, notEnoughData, monthOnMonth, alert, history[6], termsOnFile, contract, disputeCandidates, rates[{label,value%}], notMeasuredRates, ratesMeasured, ratesTotal:6}`.
- Also: Overview/metrics pages get M07, M29 (via grade mix), M30 through Module 10 snapshots; alerts page gets "Vendor score drop".

Evidence vault (Module 14) — apps/evidence (+ vault stats in apps/anchoring)

Purpose (plain words)

Lets a client's Admin or lawyer produce court-ready evidence for one phone number (or up to 25 at once): everything CiV knows about that number, plus the fingerprint (SHA-256) of every record CiV saw, the nightly seal it belongs to, and whether the record at the source still matches. Also lets the company place **legal holds** so nothing under a dispute is deleted at its 4-year date.

Data it uses, and from where

- The number page data: `contacts.api.number_file(client, ref)` and `contacts_of(client, ref)` (`ref` = `'p-' + first 24 hex chars of the phone code`).
- `consent.ConsentProof` (+ `certificate`), `optouts.OptOutEvent` — to list records behind the file.
- `ledger.Fingerprint` (sha256, source_grade, fetched_at, leaf_index, merkle_path, anchor) and its `anchoring.MerkleAnchor` (day, root, tsa_time, ots_proof). `anchoring.seal.verify(fp)` → matches / changed / not_sealed.
- The client's tool connections (`connections.Connection` with status CONNECTED or STALE) and `sync.connector.connector_for(connection).fetch_one(record_id)`.
- `reports.words.banned_words(text)` — the wording check.
- Phone codes: `contacts.normalize.codes_for(client).token(number)`, `keyring.phones.normalize`, `contacts.normalize.mask`.

Tables

- `refetch_check` (`RefetchCheck`, RLS): `client`, `source_system`, `source_record_id`, `checked_at`, `result` (`matches` | `changed` | `gone` | `cannot` | `failed`), `sha256` (of the record as fetched now), `matched` (FK Fingerprint, nullable), `why` (`on_demand` | `weekly`). Index (`client, source_system, source_record_id, checked_at`). Only fingerprints kept, never the record.
- `legal_hold` (`LegalHold`, RLS): `client`, `scope` (`number` | `vendor` | `date range`), `target` (what is shown: masked number / vendor name / dates text), `phone_token`, `vendor_key`, `starts`, `ends`, `reason` (≤300), `set_by`, `set_at`, `released_by`, `released_at`.

Flow step by step

Evidence file (step 1)

1. Admin/lawyer opens the number page and clicks "Export this file" (PDF) or "JSON".

2. `GET /api/evidence-file/<ref>?format=pdf|json` → `file.build(client, ref)`:
   - page = `number_file`; contacts = `contacts_of`; token = last contact's phone code;
   - `records_of` lists (what, tool, record id) for every contact, consent proof, certificate and opt-out (deduped);
   - `fingerprints` lists each Fingerprint of those records with seal (day, root, leaf, path, timestamp, bitcoin flag), seal check words, and `at_source` (latest RefetchCheck result + date);
   - output: company, masked number, ref, note "The full number is not held by CiV; add it from your own records.", owners (epochs), contacts, proofElements, optOuts, listChecks, cameFrom, consent, seal, fingerprints, disclaimer.
3. Banned-word check on the JSON text; a hit → 409 "held back for a wording check", logged `evidence_file_blocked`.
4. PDF made by `pdf.render` (reportlab, landscape letter; tables: Owners, Contacts, Proof elements, Fingerprints; footer prints SHA-256 of the data on every page).
5. Download logged `evidence_file_downloaded` with format and SHA-256 of the bytes; `Cache-Control: private, no-store`.

Re-fetch check (step 2)

1. On demand: `POST /api/evidence-file/<ref>/refetch` → all records behind the number → `refetch.check_records`.
2. For each (tool, record id): find a usable connection; none → `cannot`; `fetch_one` raises `NotImplementedError` → `cannot`; other exception → `failed`; returns None → `gone`; else sha256 of `record.raw`, record deleted from memory, compared with any stored Fingerprint of that record → `matches` (with `matched` FK) or `changed`.
3. Weekly: `weekly_sample(client)` picks 50 distinct sealed call/text records (from the first 5,000 distinct), deterministic random seed `"<client pk>|<ISO year-week>"`.
4. Changed results in the last window raise alert `refetch-changed-<G-V>`, kind "Records changed at the source", MEDIUM, domain REC (`alerts/raise_.py`).

- `fetch_one` exists in Twilio, Vicidial, HubSpot, Salesforce (prefix `name:key` for CRM objects / tables), Convoso ("TO CONFIRM on a real account"); Five9 has none → `cannot`.

Bulk export + certification pack (step 3)

1. Reports page "Bulk evidence export" box: `POST /api/evidence-bundle {numbers:[…], pack:bool}`.
2. Max 25 numbers (400 if more or none). Each typed number → normalised → phone code in memory → latest ContactEvent; not seen → "not found" list (last 4 digits).
3. Build every file, banned-word check over all (409 on hit).
4. ZIP: `evidence-files/NN-<ref8>.pdf` and `.json`; `not-found.txt`; with pack: `certification/fingerprint-list.csv` (number, record, tool, record_id, read_at_utc, sha256, seal_day, seal_root, leaf, path, seal_check, at_source), `how-civ-captures-records.txt`, `how-to-check-a-fingerprint.txt`, `custodian-declaration-DRAFT.txt` (marked draft for counsel; company name filled), `made.txt`.

5. Logged `evidence_bundle_downloaded` with counts and SHA-256; header `X-Not-Found`; filename `certification-pack-YYYYMMDD.zip` or `evidence-files-YYYYMMDD.zip`.

Legal holds (step 4)

1. Admin/lawyer on the Evidence Vault page: `POST /api/legal-holds {scope, target, reason}`.
2. `holds.place`: number → normalised (else "Enter the full phone number.") → stored as phone code + masked target; vendor → vendor key + cleaned name; date range → two dates (ISO or US m/d/yyyy; exactly two, start ≤ end) → `starts/ends`, target "1 Jan 2026 – 31 Mar 2026".
3. Logged `legal_hold_placed`; release `POST /api/legal-holds/lh-<id>/release` sets `released_by/at`, logged `legal_hold_released`.
4. Every 4-year deletion job builds `Holds()` (all active holds, read in `all_clients()` scope) and skips anything `covers(client_id, when, tokens, vendors)` returns True for: uploads (`uploads/effects.py`), proof request files (`provenance/requests.py`), page captures (`captures/capture.py`), reports (`reports/make.py`), vendor contracts (`vendors/contracts.py`).
5. Number page shows "Under legal hold" (`contacts.api.held` → `number_held`).

Logic and rules

- Evidence file download, re-fetch, bulk export, placing/releasing holds: **Admins and the lawyer only**; viewing holds: everyone in the company.
- Numbers are never stored as typed; only phone codes and masked labels.
- Fingerprints and seals are never deleted; holds protect files only.
- Seal words: "Matches its seal" / "Does not match its seal" / "Not sealed yet (sealed tonight)".
- Disclaimer on every file: "Comply iV measures and records. It operates no controls, sits in no call path and makes no compliance decisions. Findings are measurements, not legal advice. Comply iV is not a law firm."
- M36 evidence coverage + coverage cap (README step 5) lives in `scoring/coverage.py` (out of this slice): CiV/independent/API = 1, client uploads 0.7, statements/missing 0; below `coverage_cap_floor` 0.60 the grade is held at C.

Key functions/classes

- `apps/evidence/file.py:records_of / fingerprints / at_source / build` — evidence file data.
- `apps/evidence/pdf.py:data_sha / render` — PDF with data-fingerprint footer.
- `apps/evidence/refetch.py:check / check_records / weekly_sample / latest` — re-fetch check.
- `apps/evidence/bundle.py:resolve / files_for / zip_bundle` (+ texts `HOW_CIV_CAPTURES`, `HOW_TO_CHECK`, `CUSTODIAN_TEMPLATE`) — bulk export.
- `apps/evidence/holds.py:Holds.covers / parse_dates / place / number_held` — legal holds.
- `apps/sync/connector.py:Connector.fetch_one` — base (raises NotImplementedError).
- `apps/anchoring/api.py:VaultView / PositionView / HashCheckView` — vault stats, position lookup, hash check.

API endpoints

- `GET /api/evidence-file/<ref>?format=pdf|json` — Admin/lawyer (403 otherwise); 404 no file; 409 banned word.
- `POST /api/evidence-file/<ref>/refetch` — Admin/lawyer; returns `{checked, results: {label:count}}`.
- `POST /api/evidence-bundle` — Admin/lawyer; ZIP.
- `GET /api/legal-holds` — any client user; up to 200 rows `{id 'lh-<pk>', scope, target, reason, setBy, setOn, releasedOn}`.
- `POST /api/legal-holds` — Admin/lawyer; 201.
- `POST /api/legal-holds/lh-<id>/release` — Admin/lawyer.
- `GET /api/evidence/<ref>` — number page / evidence file screen data (`contacts.api.NumberView`, Module 7/8 area; feeds the screen).
- `GET /api/vault` — `{artifacts (fingerprint count), anchors, lastAnchor, engagedSince, coverage[12 months: before|covered|partial]}`; `GET /api/position?phone=&date=`; `GET /api/hash-check?hash=` (anchoring app).

Scheduled jobs

- `apps.evidence.tasks.weekly_refetch` — Sundays 03:40 UTC: 50 records per active client.
- Holds are read by: `purge_old_uploads` 04:55, `purge_old_proof_files` 04:50, `purge_old_captures` 04:45, `purge_old_contracts` 04:50, report purge (in `reports/make.py`; schedule unclear from this slice).
- `apps.alerts.tasks.raise_alerts` — every 300 s (changed-at-source alert).

Admin panel parts

- None (no `admin.py` in `apps/evidence`).

Frontend pages fed

- **Evidence Vault** `/vault` (`screens/vault/VaultScreen.tsx`, `useVault` → `/vault`, MERGE path): KPIs Records held, Daily anchors (last anchor), "Deleted under retention" (not sent by Django → stays sample value), Coverage strip; `PositionLookup` (`/position`); `LegalHolds` (`useLegalHolds` → `/legal-holds`; place form and Release button for Admin/lawyer).
- **Evidence file / number page** `/numbers/<ref>` (`screens/evidence/EvidenceFileScreen.tsx`, `useEvidenceFile` → `/evidence/<ref>`): "Under legal hold" pill; for real `p-…` refs and Admin/lawyer: "Export this file" (PDF link to `/api/evidence-file/<ref>?format=pdf`), "JSON", "Check again at the source" (`POST /evidence-file/<ref>/refetch`). Sample numbers instead POST `/reports` (mock).
- **Reports → Bulk export** (`screens/reports/BulkExport.tsx`): `fetch('/api/evidence-bundle')` with CSRF token, downloads ZIP; pack checkbox.

Insurer views (Module 15) — apps/buyers

Purpose (plain words)

Lets a client share its monthly **risk attestation** (a standard underwriting data sheet) with its own insurer or a company buying it (acquirer). The client invites the underwriter by email from Settings; the underwriter signs in to an insurer-only view (Portfolio, Exposure, Integrity, Attestation, Litigation, Export), sees only companies that share with it right now, never contact-level data, and can push the sheet to its own pricing software ("rating engine"). The client can sign statements about itself; CiV shows any conflict with what it measured.

Data it uses, and from where

- Client: `clients.Client` (name, industry).
- Attestation build (`attestation.py:build`): latest Module 10 score snapshot (`scoring.snapshots.latest(client, period)` → `run.score` with `exposure`, `coverage`, `factors`, `defect`, `partial`; `run.metrics` for M11/M12 worst-case), `reconcile.api.attestation(client)` (last-30-day frequency drivers + `monitoring`, `controls.fingerprinted`), `reconcile.models.PeriodIntegrity` (completeness, reconciliation rows), `vendors.VendorMonth` (vendor count), Document AI trusted answers (`docai.read.latest_answers` for `policy_file` and `contract_clauses`), statements.
- Conflicts (`attestation.py:conflict`, 90 days): `consent.ConsentResult`, `uploads.DncCheck`, `uploads.InternalListEntry`, `scoring.extra.internal_list`.

Tables

- `buyer_org` (`BuyerOrg`): `name`, `domain` (unique; company domain, or full address for free mail), `kind` (`insurer` | `acquirer`), `is_active` (CiV block switch), `created_at`.
- `buyer_user` (`BuyerUser`, AbstractBaseUser): `org`, `email` (unique), `name`, `is_active`, `created_at`, password, last_login.
- `buyer_link` (`BuyerLink`, `lock_buyer_table`): `org`, `client`, `granted_by`, `granted_at`, `ends_on`, `revoked_by`, `revoked_at`. `active(today)` = not revoked and (no end or today ≤ end).
- `buyer_invite` (`BuyerInvite`, `lock_table`): `org`, `client`, `email`, `name`, `invited_by`, `token_hash` (unique), `sent_at`, `expires_at`, `used_at`.
- `insured_statement` (`Statement`, `lock_table` — client only, buyers never read it): `client`, `topic` (`dnc_checked`, `internal_list`, `optouts_honored`, `consent_all`, `other`), `text` (≤500), `signed_by`, `signed_at`, `withdrawn_at`.
- `attestation_snapshot` (`AttestationSnapshot`, policy `shared_with_buyer`: all / own client / buyer scope AND client_id in that buyer's active `buyer_link` rows (not revoked, `ends_on` null or ≥ today)): `client`, `period`, `version`, `payload` (JSON), `sha256`, `made_at`, `locked`. Unique (`client, period, version`).
- `rating_engine` (`RatingEngine`, policy `org_only`: all or buyer's own org): `org` (one-to-one), `adapter` (`json_post`), `url`, `key` (encrypted via `connections.crypto.lock`, not editable), `key_hint` (last 4), `monthly`, `updated_by`, `updated_at`.
- `rating_send` (`RatingSend`, `lock_buyer_table`): `org`, `client`, `snapshot`, `company`, `sent_at`, `by` (person or "Monthly send"), `ok`, `status`, `detail`.

Flow step by step

Sharing and sign-up (step 1)

1. Client Admin, Settings → Insurer sharing: `POST /api/buyer-links {name, email, person, kind, endsOn}`.
2. `invite.share`: email must contain "@"; may not belong to a client user; existing buyer user → its org; else org by `domain_of(email)` (free-mail domains like gmail.com → the full address); else create org (needs company name). Blocked org → refused. Acquirer needs an end date; end date in the past refused (view).
3. If an active link already exists, reuse it; else create `BuyerLink` and immediately `attestation.save(client, this month)` so the insurer sees data at once.
4. If the person has no account: new invite token (only hash stored), old unused invites for that email deleted, 7-day lifetime (`INVITE_LIFETIME`), email "<Company> shares its CiV risk attestation with you" with link `<PORTAL_URL>/sign-in?invite=<token>` (console backend; not delivered yet).
5. Logged `buyer_link_granted` (+ "invite sent").
6. Person opens link → `accounts/views/invites.py` checks client invites then `buyers.invite.open_invite` (looked up in `all_clients()` since nobody is signed in) → `accept` creates `BuyerUser`, marks invite used, signs in.
7. Sign-in: `accounts/views/sign_in.py` tries ClientUser first, then `buyer_sign_in` (password, user + org active) → `buyers.session.sign_in` (own session keys `civ_buyer_user_id`, `civ_buyer_auth_hash`). Session data always `view: 'underwriter'`, `role: 'member'`, `orgKind`, `orgName`, `clientName` = org name, `sampleData: false`.
8. Every API request from a buyer runs in `buyer(org)` scope; DRF auth `BuyerSessionAuthentication` (CSRF checked); permission `IsBuyerUser`. Client endpoints use `IsClientUser`, so they refuse buyers.
9. Client Admin stops: `POST /api/buyer-links/bl-<id>/revoke` → logged `buyer_link_revoked`; the buyer loses access at its next request (RLS).

Statements and snapshot (step 2)

1. Admin or lawyer signs on the client's Risk Attestation page ("C. Insured states"): `POST /api/statements {topic, text}`; empty text on a fixed topic → the topic label is the text; `other` needs text. Logged `statement_signed`. Withdraw `POST /api/statements/st-<id>/withdraw`, logged.
2. `statements(client)` checks each topic for a conflict: DNC → conflict "No DNC check records supplied..." if none uploaded; internal list → "Internal do-not-call list not supplied" or count of contacts after a number joined the list (90 d); opt-outs → count of results with `NP_AFTER_OPT_OUT`; consent → count of legal-basis results `no_proof`/`conflicting`; other → "Info only: CiV cannot check this". Conflicts listed with treatment "Measured value used; the statement is shown only".
3. Nightly (`scoring.tasks.snapshot_metrics`, after findings refresh) and on first share: `attestation.save(client, month)`:
   - build payload; sha256 of sorted JSON;
   - locked if today ≥ first of next month + 5 days (`LOCK_DAYS`);

- same sha as newest → no new row (mark locked if due);
- newest not locked → overwrite in place;
- newest locked and content changed → new version (old kept).

4. Payload `civ.attestation.v1`: `schema, company, industry, reading` (factor values, venue/targeting/volume multipliers, source text "Measured from records · <Mon YYYY> [· partial] · draft bands"), `period` YYYY-MM, `monitoring`, `frequencyDrivers` (each field `{value, grade, worstCase, note?}`; grades C from records; M12/M11 worst case → grade N; `vendors` C; `vendorTermsShare` D = share of AI-read contracts with indemnity found), `recordsIntegrity` (basisMix, reconciliation, evidenceCoverage A, coverageMix, completeness B, fingerprinted A, exposure value/grade/defect/partial/factors), `controls.policy` (True when a trusted policy_file reading says `tcpa_policy`, else null), `insuredStates` (statements, conflicts, signedBy, asOf). No contact-level rows.

Insurer pages (step 3)

1. `GET /api/insureds` → newest snapshot per client (Postgres `DISTINCT ON client_id`); RLS hides non-sharing companies → mapped by `portal.insured` into the portal's `Insured` shape; sorted by name; header `X-CiV-Viewer: buyer`.
2. `GET /api/insureds/c-<id>` (in `reconcile.api.OwnInsuredView`): buyer → `buyers.portal.one` (404 "This company does not share with you." otherwise) and `seen()` writes `attestation_viewed` ("Insurer opened your sheet") once per day per person and company. A client user calling the same route gets its own live page (`reconcile.attestation`, Module 10 exposure reading, statements incl. `statementRows`, completeness, reconciliation).
3. `GET /api/litigation` is open to buyers (public court figures), marked buyer.
4. Fields CiV doesn't measure are null: `marketingShare, smsShare, optOutMedianHours, optOutFailShare, rndExposure, litigatorContacts, unknownCallerIds`, plus `praStates, priorStated, priorDocket, openMatters`, `controls.trainingCurrency`, `controls.optOutTests90d`, `exposure.outside`. `sheetHash` = first 12 chars of sha256; `asOf` = made_at date; `selfReported` false.

Rating engine

1. Insurer on Underwriting Export: `POST /api/rating-engine {url, key, monthly}` → `check_address(url)`: https only, host resolves, every resolved IP must be global and not multicast (blocks private/local). Key required on first save; stored locked, hint last 4. `DELETE` removes. `GET` returns url, keyHint, monthly, adapter label, last 10 sends — never the key.
2. "Send to rating engine": `POST /api/rating-engine/send {insured:'c-<id>'}` → newest snapshot (only if sharing, else 404) → `engines.send`: unlock key, `JsonPost.send` re-checks address, body = `sheet_of(snapshot)` = payload + `companyId`, `version`, `sha256`, `madeAt`, JSON sorted keys; headers `Authorization: Bearer <key>`, `X-CiV-Signature: sha256=<HMAC-SHA256(key, body)>`, `X-CiV-Sheet-Sha256`, UA "CiV attestation sender"; no redirects (`_NoRedirect`); 15 s timeout; 2xx = accepted. Row in `rating_send`; access log `attestation_sent` / `attestation_send_failed` on the company. Reply 200 if ok else 502.
3. Monthly (`send_monthly`, 6th 06:30): for each active org with `monthly=True`, each active link, last month's newest **locked** snapshot, sent once (skips if an ok send of that snapshot exists).

Security checks (step 4) — `tests/buyers/test_security.py`: tries every `/api/` route as a signed-in insurer; only `session, insureds, insureds/<ref>, litigation, rating-engine, health, invites/check` (reads) and `session/sign-in, session/sign-out, invites/accept, rating-engine, rating-engine/send, session/password` (writes) may answer 2xx; nothing read or sent contains phone codes, record ids or `statementRows`; guessing another company's id → 404; access ends when sharing is revoked, an acquirer's end date passes, the org is blocked or the person is turned off; one insurer never sees another's engine; the key never comes back.

Logic and rules

- One buyer org per email domain; free-mail addresses are their own org; a client can never add a person to another company's insurer.
- An email used by a client user cannot become a buyer (and vice versa, `accounts/views/users.py`).
- Only Admins share/stop; Admins or lawyer sign/withdraw statements.
- Statements never raise a value; conflicts show the measured value.
- Insurers see the **newest** snapshot only; snapshots of locked months are versioned, never overwritten.
- CiV staff never create buyers; they only view and block.
- Loss model left out of the backend (owner, 2026-10-08) — but the frontend still computes one (see limits).

Key functions/classes

- `apps/buyers/models.py` — BuyerOrg, BuyerUser, BuyerLink, BuyerInvite, Statement, AttestationSnapshot, RatingEngine, RatingSend.
- `apps/buyers/invite.py:domain_of / send / share / open_invite / accept` — sharing and invites.
- `apps/buyers/session.py:sign_in / sign_out / current_buyer / session_data` — buyer session.
- `apps/buyers/authentication.py:BuyerSessionAuthentication / IsBuyerUser` — DRF auth/permission.
- `apps/buyers/api.py:LinksView / RevokeView / StatementsView / WithdrawStatementView` — client API.
- `apps/buyers/attestation.py:field / conflict / statements / reading / build / sha_of / save` — attestation snapshot.
- `apps/buyers/portal.py:newest / insured / seen / as_buyer / InsuredsView / one` — insurer pages.
- `apps/buyers/rating.py:view / RatingEngineView / SendView / send_monthly` — rating engine.
- `apps/buyers/engines.py:check_address / _NoRedirect / _post / JsonPost / sheet_of / send` — adapter.
- `apps/reconcile/api.py:OwnInsuredView` — `/api/insureds/<ref>` for clients and buyers.
- `apps/litigation/api.py:LitigationView` — court figures for both.
- `apps/tenancy/scope.py:buyer` + `middleware.py` — buyer scope.

API endpoints

- `GET /api/buyer-links` — client user; `{links:[{id 'bl-<pk>', org, kind, grantedBy, grantedOn, endsOn, stoppedOn, active}]}`.
- `POST /api/buyer-links` — client Admin; 201 + `invited`.
- `POST /api/buyer-links/bl-<id>/revoke` — client Admin.
- `GET /api/statements` — client user (statementRows). `POST /api/statements` — Admin/lawyer. `POST /api/statements/st-<id>/withdraw` — Admin/lawyer.
- `GET /api/insureds` — buyer only.
- `GET /api/insureds/<ref>` — client user (own page, ref ignored) or buyer (`c-<client id>`).
- `GET /api/litigation` — client or buyer.
- `GET|POST|DELETE /api/rating-engine` — buyer only. `POST /api/rating-engine/send` — buyer only.
- Shared account routes: `/api/session`, `/api/session/sign-in`, `/api/session/sign-out`, `/api/invites/check`, `/api/invites/accept`.

Scheduled jobs

- `apps.scoring.tasks.snapshot_metrics` — daily 04:00 UTC: `save_attestation(client, this month)`.
- `apps.buyers.tasks.monthly_rating_sends` — 6th of each month 06:30 UTC (runs in `all_clients()`).

Admin panel parts

- `apps/buyers/admin.py:BuyerOrgAdmin` (sidebar group "Insurers" → "Insurers and acquirers", `admin:buyers_buyerorg_changelist`): list name, domain, kind, active, People count, Companies sharing; filters kind/active; only `is_active` editable (blocks the whole insurer); no add. Inlines (2026-10-23): **People (joined)** (name, email, last login, active toggle, **Send password link**, Delete tick box), **Waiting invitations** (Sent / Expired with **Resend** and **Cancel**), **Companies sharing** (client, granted by/at, until, status, **Stop sharing**, which the company sees in its log). **Delete** an insurer from its page after an "Are you sure?" page: its people, invitations, sharing and rating engine go; each company that shared gets a log line "Insurer deleted by CiV".
- No admin for statements, snapshots, rating engines or sends.

Frontend pages fed

- **Settings** → **Insurer sharing** (`screens/settings/SharingTab.tsx`): form (Company, Underwriter's email, Their name, Insurer/Acquirer, Until) for Admins → `POST /buyer-links`; table Shared with / Since / Until / Status + "Stop sharing" → `/buyer-links/<id>/revoke`.
- Client nav group "Shared with your insurer": `/integrity`, `/attestation`, `/uw-export` (client sees its own company; `useViewer` sets insuredId `'sunpath'` → `/insureds/sunpath` → own live page).
- Underwriter nav (`UNDERWRITER_NAV`): `/portfolio`, `/exposure`, `/integrity`, `/attestation`, `/litigation`, `/uw-export`. `useViewer` picks the first real insured until another is chosen in `InsuredPicker` (`useInsureds`).

- **Portfolio** (`PortfolioScreen`, `/insureds`): KPIs Insureds, Portfolio expected annual loss ("Loss model v0, uncalibrated"; "Not measured" if no contacts), Below 60% evidence coverage, Statement conflicts; insureds table.
- **Exposure Indicator** (`ExposureScreen`, `/insureds/<id>`): exposure, defect term D, multipliers, evidence coverage (computed in `lib/exposure`).
- **Data Integrity** (`IntegrityScreen`): coverage, completeness, conflicts, unknown caller IDs; `BasisMixCard`, `ReconciliationCard`, `ConflictsCard`.
- **Risk Attestation** (`AttestationScreen`): KPIs Evidence coverage, Fields at worst case, Sheet fingerprint, Monitoring; A. Frequency drivers table (each with source grade chip); B. Records integrity (underwriter also sees "Theoretical statutory exposure" $500–$1,500 per unconsented contact); C. Insured states (+ `StatementForm` for the client: topics, Sign and add, Withdraw → `/statements`).
- **Litigation Intelligence** (`LitigationScreen`, `/litigation`).
- **Underwriting Export** (`ExportScreen`): `PlainPayload` shows the payload in plain words; payload built **in the browser** (`payloadFor`) from the Insured object (+ `exposure_indicator` and `model_v0` loss model for the underwriter only); "Send to rating engine" → `/rating-engine/send`; "Copy JSON"; `RatingEngineCard` (address, key [password field, hint], monthly checkbox, Save/Remove, last sends) → `/rating-engine`.
- Session/user menu: `mergeSession.ts`, `UserMenu.tsx`, `Sidebar.tsx` handle the buyer session (underwriter view only).

Document AI (Module 16) — apps/docai (+ spoken consent flag in apps/conversations)

Purpose (plain words)

One shared "AI slot" that will read documents: vendor contracts (which required clauses are in them, end date, dispute days), captured consent pages (do the consent words really give this company consent), and policy/training uploads (is this a written TCPA policy, a training record, ...). The slot is **off** until the owner picks the AI model; nothing is sent anywhere and documents wait. An AI's answers are only written into records after it passes an exam (≥50 hand-checked examples, ≥95% right) and is ≥90% sure.

Data it uses, and from where

- `vendors.VendorContract` files (not purged) → feature `contract_clauses`; questions from Rulebook `vendor_required_clauses`.
- `captures.PageCapture` (ok, not purged; newest per page; `consent_block` or `text`, sha `consent_block_sha256` or `text_sha256`) → `consent_page`; seller names = client name + `brand_names` lines.
- `uploads.UploadedFile` with source `policies` (not CSV, not purged) → `policy_file`.
- Exam cases: `apps/docai/exam/<feature>.json` (currently 2 cases each).

Tables

- `doc_reading` (`DocReading`, RLS `lock_table`): `client`, `feature` (`contract_clauses` | `consent_page` | `policy_file`), `ref` (e.g. `vendor_contract:12`, `page_capture:5`, `uploaded_file:9`), `input_sha256`, `status` (`waiting` | `done` | `unclear` | `failed`), `answers` (JSON), `confidence`, `model`, `prompt_version`, `trusted`, `applied`, `note`, `queued_at`, `read_at`. Unique (`client, feature, ref, input_sha256, prompt_version`) → the same file with the same instructions is never read twice.
- `doc_exam_run` (`DocExamRun`, no client, no RLS): `feature`, `model`, `prompt_version`, `cases`, `right`, `passed`, `mistakes` (≤50), `note`, `ran_at`.
- `conversation_review` (in `apps/conversations`, Module 9) gained `consent_at`, `consent_check` (`waiting` | `confirmed` | `rejected`), `consent_checked_by` (migration `conversations/0006_spoken_consent`).

Flow step by step

1. Every 15 min `read_documents`: for each active client (tenant scope) `read.queue(client)` creates a `DocReading` (status waiting) for every source document of every feature with the current prompt version (`ai.PROMPT_VERSIONS`, all `v1`).
2. Then in `all_clients()` scope `read_waiting(limit=50)`: if `ai.active()` is None → returns 0 (slot off, nothing sent).
3. With a reader plugged in (`ai.use(reader)`): `read_one` loads the document in memory (`{'file', 'mime', 'name'}` or `{'text'}`), calls `reader(feature, document, questions)` → `{answers, confidence}`; deletes the document from memory. Exception → `failed` ("Could not be read (<Error>)"). Confidence ≥ 0.90 → `done`, else `unclear` (kept, never applied).
4. `trusted` = the reader's model passed the latest exam for that feature and prompt version. `done` + trusted → `feature.apply(ref, answers)`, `applied=True`; `done` but not trusted → note "Read, not applied: the AI has not passed its exam for this yet".
5. Apply per feature:
   - `contract_clauses`: `VendorContract.clauses` = {clause id: found bool}; fills `ends_on` and `dispute_days` (0–365) only when the client left them empty ("what the client typed always wins").
   - `consent_page`: nothing written; kept on the reading ("the consent page card reads it").
   - `policy_file`: nothing written; the attestation reads trusted readings (`controls.policy`).
6. Daily 02:20 `run_doc_exams`: if a reader is active, for each feature whose exam is due (none yet for this model+prompt, or older than 30 days) → `exam.run`; on pass → `apply_trusted` applies earlier `done` answers of that model/prompt.
7. Exam grading (`exam.right`): contracts → each expected clause's found flag (quotes ignored) and `ends_on`/`dispute_days` if given; other features → every expected key equal. Pass = cases ≥ 50 AND right ≥ 95%. A stand-in client "Example Solar Co" is used for questions.
8. Consumers: `vendors.contracts.terms_on_file` (M07 note "n of m checked have every required clause"); `buyers.attestation.build` (`vendorTermsShare` = share of applied contract readings with `indemnity` found, grade D; `controls.policy` True if any applied policy reading has kind `tcpa_policy`).
9. **Spoken consent** (calls/texts, Module 9 review): the conversation reviewer also returns `{"consent": turn index}`; `conversations.review.consent_flag(at)` stores `consent_at` and

`consent_check='waiting'`. It is only a flag for the client's lawyer or an Admin to confirm — never counts as consent on its own.

Logic and rules

- Slot off = nothing leaves CiV; documents queue as `waiting`.
- `MIN_CONFIDENCE` 0.90; `MIN_CASES` 50; `PASS_MARK` 0.95; exam re-run every 30 days or when model/prompt version changes.
- Readings are cached by (client, feature, ref, input sha256, prompt version).
- Only applied (trusted, sure) readings are used downstream (`latest_answers(..., trusted_only=True)`).
- Questions: contracts → clause list + `ends_on`, `dispute_days`; consent page → `seller_names` + 5 checks (`names_seller`, `written_consent`, `autodialer_or_prerecorded`, `not_a_condition`, `unchecked_box`); policy → `kind` (tcpa_policy, training_record, vendor_contract, other), `dated`, `people_trained`.

Key functions/classes

- `apps/docai/ai.py:use / active` (+ `PROMPT_VERSIONS`, `MIN_CONFIDENCE`) — the slot.
- `apps/docai/features.py:ContractClauses / ConsentPage / PolicyFile` (`sources`, `load`, `questions`, `apply`), `FEATURES` — the readers.
- `apps/docai/read.py:queue / read_one / read_waiting / apply_trusted / latest_answers` — queue and read.
- `apps/docai/exam.py:cases / right / run / latest / trusted / due` — accuracy gate.
- `apps/docai/tasks.py:read_documents / run_doc_exams` — Celery tasks.
- `apps/conversations/review.py:consent_flag` — spoken consent flag.

API endpoints

- None in `apps/docai` (no urls.py). No endpoint found to confirm/reject a spoken-consent flag (`consent_checked_by` is not written anywhere outside the model/migration).

Scheduled jobs

- `apps.docai.tasks.read_documents` — every 15 min (`crontab(minute='*/15')`).
- `apps.docai.tasks.run_doc_exams` — daily 02:20 UTC.
- (`apps.conversations.tasks.review_conversations` every 60 s and `run_exams` 02:10 belong to Module 9; off until an AI is chosen.)

Admin panel parts

- None for docai (no `admin.py`); exam runs and readings are not visible in the admin panel.

Frontend pages fed

- Indirectly only: Vendor Ledger M07 note; Risk Attestation "Lead vendors · with signed indemnification" (`vendorTermsShare`) and "Written policy" (`controls.policy`).

- No screen shows `consent_page` readings yet, and no screen for confirming spoken consent was found.

9. The portal (frontend)

Source: `/home/shakib/Audit/frontend` as of 2026-10-22, with uncommitted changes in the working tree. Everything here comes from reading the code.

1. Tech stack

From `frontend/package.json`:

Piece	Version / note
Next.js	`16.3.8` (App Router). `AGENTS.md` warns: "This is NOT the Next.js you know". Route `params` and `searchParams` are Promises that get `await`ed (see `numbers/[phone]/page.tsx`, `sign-in/page.tsx`).
React / React DOM	`19.2.8`
Data fetching	`@tanstack/react-query ^5.104.1`. One `QueryClient` lives in `src/app/providers.tsx` with `retry: false`, `refetchOnWindowFocus: false`, `staleTime: 30_000`.
Styling	Tailwind CSS `^3.4.19` + PostCSS + autoprefixer. Design tokens are CSS variables in `src/app/globals.css` (136 lines): `--brand #0e8c7f`, `--ok`, `--warn`, `--serious`, `--bad`, `--info`, `*-ink` versions, `--series-*`. Dark mode is `html[data-theme='dark']`. Shared classes: `.card`, `.btn`, `.btn-primary`, `.btn-ghost`, `.pill-*`, `.tbl`, `.field`, `.seg`, `.note-box`, `.kpi-tag`, `.tiny`.
Fonts	`next/font/google`: Sora (display), Plus Jakarta Sans (body), JetBrains Mono (mono).
Testing	Vitest `^2.1.9` (jsdom `^30`), Testing Library (react 16, dom 10, user-event 14, jest-dom 7), MSW `^2.15.0`, `@vitejs/plugin-react`.
Lint	ESLint 9 with `eslint-config-next` 16.3.8 (core-web-vitals + typescript).
TypeScript	`^5`; `typecheck` script = `next typegen && tsc --noEmit`.
Build output	`next.config.ts` → `output: 'standalone'` (for Docker).

Scripts: `npm run dev` (next dev, port 3000), `npm run build`, `npm start`, `npm run lint` (`eslint`), `npm test` (`vitest run`), `npm run typecheck`.

Deployment files:

- `Dockerfile`: two stages on `node:22-alpine`. It runs `npm ci && npm run build`, copies `.next/standalone`, `.next/static` and `public`, runs as user `node` with `PORT=3000`, and starts with `node server.js`. The README says that in Docker the portal reaches Django at `BACKEND_URL=http://backend:8000`.
- `vercel.json`: Next.js framework, region `iad1`, and these headers: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: strict-origin-when-cross-origin`, `X-Robots-Tag: noindex, nofollow`.
- The only environment variable the frontend code reads is **`BACKEND_URL`**. It is read on the server only, in `src/server/backend.ts` and `src/app/api/[...path]/route.ts`.

Page title metadata: `Comply iV — Client Portal`. The theme is applied before the first paint by an inline script that reads `localStorage['civ-theme']`.

2. Folder layout

```
frontend/
  src/app/                      Next App Router
    layout.tsx                  <html data-theme>, fonts, <Providers>
    providers.tsx               QueryClientProvider
    globals.css
    sign-in/page.tsx            sign-in / reset / new password / accept invite
    (portal)/layout.tsx         wraps every portal page in <PortalShell>
    (portal)/page.tsx           "/" Overview
    (portal)/<route>/page.tsx   each page.tsx only renders one Screen component
    (portal)/numbers/[phone]/page.tsx
    api/[...path]/route.ts      the single /api/* handler (mock / Django / merge)
  src/screens/<area>/           one folder per page; <Name>Screen.tsx + its cards
  src/components/shell/         PortalShell, Sidebar, Topbar, UserMenu, ViewAsSwitch, Banners,
Locked,
                                ViewBlocked, PeriodSelect, Freshness, Disclaimer, SearchBox,
DrawerHost,
                                ThemeToggle, nav.ts
  src/components/ui/            Card, Kpi, Loader, PageHead, SectionTitle, Seg (segmented
tabs/filters),
                                Drawer, Toggle, Empty, Info, Icon, Delta, GradeBadge, StatusChip,
TierChip,
                                SeverityPill, VerdictPill, DomainCode, MetricCode, NotMeasured
  src/components/charts/        hand-made SVG charts: BarList, Columns, StackedBar, TrendLine,
Sparkline,
                                AreaSparkline, Donut, Gauge, Meter, CoverageStrip, Legend,
ChartTip, useWidth
  src/components/drawers/       slide-out panels: Action, Contact, Domain, Metric, Vendor, WhoArea
(+ActionRows)
  src/api/client.ts             apiGet / apiSend / apiUpload (fetch wrapper, CSRF, error text)
  src/api/queries.ts            one React Query hook per endpoint + useSend + useCanCall
  src/api/types.ts              every data shape (473 lines); also DISCLOSURE text, BASIS_LABEL,
FAMILIES
  src/state/                    PortalContext (month + insured), DrawerContext (open panel),
useTheme
  src/lib/                      views.ts, exposure.ts, lossModel.ts, format.ts, months.ts,
notMeasured.ts,
                                status.ts, whoContact.ts, wording.ts
  src/mock/api.ts               in-memory stand-in for Django (pure function mockApi)
  src/mock/data/*.ts            sample data: session, account, disclosure, domains, engagement,
evidence,
                                insurer, intelligence, metrics, whoContact, work
  src/server/backend.ts         BACKEND_PATHS / MERGE_PATHS / WRITE_PATHS / overlay() / forward()
  src/server/mergeSession.ts    joins Django's session with the mock's sample-only fields
  tests/                        vitest + MSW tests (see §9)
```

House rules visible in the code: one component per file, a short doc comment on each, and screens never call `fetch` themselves. They go through hooks in `queries.ts`. There are three exceptions that call `fetch` or an `<a href="/api/...">` link directly to download files: `BulkExport` (evidence bundle ZIP), `PlanTab` (billed-numbers CSV), and `ReportCard` (rulebook snapshot PDF). `EvidenceFileScreen` (evidence PDF/JSON) and `DisputeList` (CSV) download through links.

src/lib

- `views.ts`: `UNDERWRITER_PATHS`, `INSURER_ONLY_PATHS`, `viewBlock()` (see §5).
- `exposure.ts`: Exposure Indicator v2.2. It has the six factor weights (they add up to 1), the names, the law for each, `exposureOf()` (defect term D × the venue, targeting and volume multipliers), the grades (A <25, B 25–39, C 40–48, D 49–66, F 67+, where higher means more exposed), and the domain→factor and factor→metric maps.
- `lossModel.ts`: "Loss model v0". It has the parameters, `lossModel(insured)` (chance of a suit, cost if sued, expected loss, number of worst-case fields) and the `money` formatter. It is marked as a hypothesis until it is calibrated.
- `format.ts`: score, percent, count and date formatting, `gradeOf`, `TIER_LABEL`, and `truncateHash`.
- `notMeasured.ts`: `shareOr` and `countOr` print "Not measured", never a sample value or zero, for a real insurer.
- `status.ts`: consent status colours, the meaning of each status, and `CHECK_NAMES` (the five checks).
- `whoContact.ts`: the area state order, source chip meta, and `CHECK_COLUMNS` (the four lists).
- `wording.ts`: `BANNED_WORDS` plus `findBannedWords` and `assertNoBannedWords(Deep)`. This is a guard that no text a client can see uses banned words (engineering spec §9). It is used in tests.
- `months.ts`: short month labels for history charts.

3. How data reaches a screen

3.1 Client side

- `src/api/client.ts`
  - `apiGet(path, query)` → `GET /api{path}?{query}` (empty query values are dropped).
  - `apiSend(method, path, body)` → JSON body with an `X-CSRFToken` header read from the `csrftoken` cookie.
  - `apiUpload(path, FormData)` → multipart POST with the CSRF header (used for proof files, uploads and vendor contracts).
  - Errors become `ApiError(status, message)`. The message is DRF's `detail`, or the first message of the first field.
  - Under the test runner the base URL is `http://localhost`. In the browser it is relative.
- `src/api/queries.ts`: the hooks are `useGet` (query key `[path, query]`) and the ones built on it. `useSend()` runs a mutation and, on success, calls `invalidateQueries()` with no filter, so **every query refetches after any change**. `useCanCall()` is a mutation (`POST /can-call`) that refreshes nothing.

Hooks and their endpoints (every `GET` unless noted):

Hook	Endpoint
useSession	/session
useScore(period)	/score?period=
useDomains(period)	/domains?period=
useActions	/actions
useAlerts	/alerts
useContacts(status,q)	/contacts?status=&q=
useContact(id)	/contacts/{id}
useConsent	/consent
useConsentPages	/consent-pages
useCertificates	/certificates
useRevocation	/revocation
useOptOuts	/optouts
useWhoContact	/who-contact
useVault	/vault
usePosition(phone,date)	/position?phone=&date= (only when both are set)
useHashCheck(hash)	/hash-check?hash= — defined but not used by any screen
useLegalHolds	/legal-holds
useEvidenceFile(phone)	/evidence/{phone}
useMetrics(period)	/metrics?period=
useConduct	/conduct
useLeads	/leads
useProvenance	/provenance
useProofRequests	/proof-requests
useDisputes(vendor, enabled)	/vendor-disputes?vendor=
useVendorContracts	/vendor-contracts
useVendors	/vendors
useRatingEngine(enabled)	/rating-engine
useBuyerLinks	/buyer-links
useSources	/sources
useUploads	/uploads
useReports	/reports
useRulebook	/rulebook

Hook	Endpoint
useRulebookCalendar(year)	/rulebook/calendar?year=
useRegChanges	/reg-changes
useReadiness	/readiness
useSettings	/settings
useUsers	/users
useAccessLog	/access-log
useHealth	/data-health (Django's own `/api/health` is a server check and is not forwarded)
useInsureds(enabled)	/insureds
useInsured(id)	/insureds/{id}
useLitigation	/litigation
useSearch(q)	/search?q= (only when q has 2 or more characters)
useCanCall (mutation)	POST /can-call {number}

3.2 Server side: src/app/api/[...path]/route.ts

A single handler serves `GET`, `POST`, `PATCH` and `DELETE` for every `/api/*` request (`dynamic = 'force-dynamic'`). It decides where each request goes, in this order:

1. **Search** (`GET /search`, only with `BACKEND_URL`): the request is forwarded to Django. The real hits (numbers, vendors) are joined with the mock's hits of kind domain, metric and page, and the list is cut to 10.
2. **MERGE pages** (`mergesBackend`: `BACKEND_URL` is set, method is GET, and the first path segment is in `MERGE_PATHS`):
   - The request is forwarded to Django.
   - If Django answers with the header **`x-civ-viewer: buyer`** (a real insurer or acquirer, Module 15), **no sample data is used**. Django's reply is passed through as it is. The one exception is `litigation`, which is `overlay(sample litigation, real)` because general court figures stay sample.
   - Otherwise the route also computes the mock's answer:
     - `/insureds` (the list) or `/insureds/{id}` where id ≠ `sunpath` → the sample is returned. This is a client trying the sample insurer view; only its own company `sunpath` gets real fields.
     - mock 403 → 403 (a page this view may not open).
     - Django not ok, or mock not 200 → Django's reply.
     - Not JSON (for example a report PDF) → Django's reply passes through.
     - Otherwise → **`overlay(sample, real)`**.

3. **BACKEND pages** (`usesBackend`): `BACKEND_URL` is set and either the first segment is in `BACKEND_PATHS` and segment 2 ≠ `view-as`, or the first segment is in `WRITE_PATHS` (for any method that was not caught by step 2).
   - If Django answers 403 and the path is in `SAMPLE_WHEN_REFUSED` (`rating-engine`), the mock's answer is returned instead. This happens when a client tries the sample insurer view.
   - `GET /session` that succeeds → `mergeSession(django, mock)`, and Django's `set-cookie` headers are copied onto the reply.
   - Everything else → Django's reply as it is.
4. **Otherwise → mock** (`mockApi`).

`forward()` builds `${BACKEND_URL}/api/{path}{search}`. It passes these headers: `cookie`, `content-type`, `x-csrftoken`, `origin`, `user-agent`. The body is sent as raw bytes (`arrayBuffer`) so file uploads work, with `redirect: 'manual'` and `cache: 'no-store'`. On the way back it copies `content-type`, `x-civ-viewer` and every `set-cookie`, and returns the body as bytes so photos and PDFs pass through. A 204 reply gets a null body.

3.3 The lists (src/server/backend.ts)

BACKEND_PATHS (fully real from Django when `BACKEND_URL` is set): `session, users, invites, rulebook, reg-changes, rule-updates, sources, data-health, contacts, evidence, can-call, optouts, consent, certificates, consent-pages, provenance, proof-requests, access-log, uploads, conduct, billed-numbers, position, hash-check, alerts, vendor-contracts, vendor-disputes, evidence-file, evidence-bundle, legal-holds, buyer-links, statements, rating-engine`

SAMPLE_WHEN_REFUSED: `rating-engine`.

MERGE_PATHS (reads only; Django's fields laid over the sample): `vault, settings, revocation, insureds, metrics, who-contact, readiness, leads, vendors, litigation, score, domains, actions, reports`

WRITE_PATHS (writes on partly real pages that Django already saves): `who-contact, actions, reports, settings, setup` (alert email choices and Setup checklist ticks are saved per company since 2026-10-23).

Always mock (even with `BACKEND_URL`): any path in no list. `POST /session/view-as` goes to Django (each person's own view). Real insurers' replies carry `x-civ-viewer: buyer` and get no sample. File downloads keep their name and privacy headers (`content-disposition`, `cache-control`) through the proxy.

`overlay(sample, real)` (tested in `tests/lib/overlay.test.ts`):

- Objects are merged key by key. Real values replace sample values, and keys the real object lacks keep their sample value.
- For a list, if every item on both sides shares a key field (`code`, then `id`, then `name`) and **at least one** real item matches a sample item, the lists are merged item by item. Unmatched sample items stay, and new real items are appended.
- If no real item matches any sample item (for example the company's own vendors), the **real list replaces** the sample list.
- An empty real list does **not** replace the sample (`real.length` must be at least 1). Scalars are replaced.

3.4 mergeSession(backend, mock)

- If Django says the user is not signed in → `{ signedIn: false }`.
- If Django says `view === 'underwriter'` (a real insurer or acquirer) → `{...mock, ...backend, sampleData: false}`. The insurer's own name is used and the View-as switch is hidden.
- Else if the **mock's** session view is `underwriter` (the client chose the sample "View as Underwriter") → `{...mock, signedIn: true}`, which shows the sample insurer identity.
- Else → `{...mock, ...backend, sampleData: true}`. **Real client users therefore still have `sampleData: true`**, which means the View-as switch appears for them. The month list (`periods`), `runs`, `updatedAt` and so on come from the mock unless Django sends them.

3.5 When the sample shows

- **With no `BACKEND_URL`:** every page is mock. The mock session starts signed in as Dana Reyes, Admin of "SunPath Residential Solar" (`clientId sunpath`, Solar, Growth plan, direct engagement). The months are Sep, Aug, Jul and Jun 2026, and `updatedAt` is 2026-09-30T04:10Z. Sign-in accepts any email and password (the form says "Sample data: any email and password will sign you in.").
- **With `BACKEND_URL`:**
  - BACKEND pages are real.
  - MERGE pages show real fields over sample fields, so a sample card stays until its module fills it.
  - A real insurer gets no sample data (except general litigation figures).
- **The "View as" switch** (account menu, shown only when `session.sampleData` is true): Owner ↔ Underwriter. It sends `POST /session/view-as {view}`, which is always mock, and then goes to `/portfolio` or `/`.
  - In the underwriter view the mock session becomes "J. Merchant, Falcon Risk Services, Underwriting pilot · 5 insureds" with `orgKind: insurer`.
  - The mock then refuses (403) every read outside `UNDERWRITER_READS = session, insureds, litigation, rating-engine, rulebook, reg-changes, search`, and every write except `rating-engine` and `session` ("The underwriter view is read-only.").
  - The mock's `view-as` (demo only) also accepts `role` and `isCounsel`, used by tests. With the backend, `view-as` goes to Django and changes only the signed-in person's own view.

3.6 The mock (src/mock/api.ts)

- `mockApi({method, path, query, body})` is a pure function shared by the dev route handler and by MSW in the tests. A mutable `db` is cloned from the fixtures, and `resetMock()` restores it.
- Every path except `/session*` returns 401 when signed out.
- Mock-only behaviours:
  - `score`, `domains` and `metrics` follow `?period=` through a four-point history.
  - `rulebook` always comes back with `impact: []`; the impact table is for CiV staff only.
  - `runs` are filtered to `outputVisibility === 'client'`.

- `vendor-contracts`, `buyer-links` and `billed-numbers` are empty, empty, and 404 respectively.
- `insureds` (the list) is underwriter-only. A client may read only `insureds/sunpath`.
- `litigation` is underwriter-only.
- `can-call` returns a fixed answer: area code 850 → Florida, two time zones, not safe; anything else → California, ok.
- `rule-updates/{id}/review` and `reg-changes/suggest` require `isCounsel`.
- `users` changes require Admin and keep at least one active Admin ("A company needs at least one active Admin."). You cannot remove yourself.
- The rating-engine mock requires an `https://` URL and a key, and records sends as "(sample)".
- Most writes add an access-log entry.

4. Shell, layout and global UI

`(portal)/layout.tsx` → `PortalShell` does the following:

- Loads `useSession`. While pending it shows "Loading…". On error it shows "The portal could not be loaded. {message}". When the user is signed out it does `router.push('/sign-in')`.
- **No lawyer-only lock** (removed 2026-10-22): every client user sees every page. **Admin-only pages:** Settings and Users & Access (`AdminOnly` wrapper: "Only an Admin can open this page"; hidden from others' account menu). **My account** (`/account`) is open to everyone, insurers too.
- **View block:** `viewBlock(view, pathname)` (see §5) shows `ViewBlocked` instead of the page.
- The layout is a `Sidebar` (fixed, 248px, a slide-in drawer on small screens), a `Topbar`, then `Banners`, the page, the `Disclaimer` and the `DrawerHost`.

Topbar:

- `SearchBox`. Client placeholder: "Search domains, metrics, numbers, vendors…". Underwriter placeholder: "Search insureds and pages…".
- For a client: `Freshness` ("Updated HH:MM UTC" with a green or amber dot and a "· N source stale" note; it links to `/sources` and uses `useSources`) and `PeriodSelect` (the month picker).
- For an underwriter: the static text "Attestations refresh monthly · 5 insureds" and no month picker.
- `UserMenu`.

SearchBox: hits are grouped by kind (Domain, Metric, Number, Vendor, Page, Insured).

- A number goes to `/numbers/{e164}`.
- A page goes to its route.
- An insured sets `insuredId` and goes to `/attestation`.
- A domain, metric or vendor opens that drawer.

Month picker (`PeriodSelect` + `PortalContext.period`): the options are `session.periods`, and undefined means the latest. It drives `useScore`, `useDomains` and `useMetrics` (Overview, Scorecard, Conduct, the drawers) and the month label on Reports. `Banners` adds:

- "Showing **{month}** for scores, domains and metrics. Lists of contacts, actions and alerts always show the current state." when the chosen month is not the latest.
- "This month was revised on **{date}** after late records or a rule change. The earlier version is kept." when `period.revisedOn` is set (Module 10).

Disclaimer at the foot of every page: "Comply iV measures and records. It operates no controls, sits in no call path and makes no compliance decisions. Findings are measurements, not legal advice. Comply iV is not a law firm."

UserMenu (account menu, top right):

- The name and email.
- For a client only, the `ACCOUNT_LINKS`: **Setup & Readiness** (`/setup`), **Settings** (`/settings`), **Users & Access** (`/users`).
- A Role line (Admin/Member). The button subtitle reads "Admin · lawyer" when `isCounsel` is true; for an underwriter it reads "Underwriter · Falcon Risk/{org}".
- The `ViewAsSwitch` (only when `sampleData` is true).
- `ThemeToggle` (light/dark, saved in `localStorage 'civ-theme'`).
- "Sign out" → `POST /session/sign-out`.

Sidebar:

- Shows "Client portal" or "Insurer portal" under the "Comply iV" logo.
- Badges: on Action Queue, the number of actions not resolved; on Alerts, the number not reviewed. These are always 0 in the underwriter view.
- Footer: the client name and "{plan} plan · {vertical}" (for an underwriter, `vertical`).

Drawers (`DrawerContext`, one at a time; `Drawer` closes on Esc, on the overlay, or on the close button):

Drawer	Opened from	Contents
`domain`	Scorecard rows, `DomainCode` chips, search	Tabs Result / Evidence / History / Actions / Definition: grade badge, checkpoints bar, top finding, sources, the metrics that feed it, trend line, open actions for the domain.
`metric`	KPI tiles, `MetricCode`, factor tiles, search	Same tabs: value with month-on-month delta or a `NotMeasured` reason, evidence, "Measured by" (CiV / From your records / AI-assisted, sampled by a person), domains it feeds, history, related open actions.
`contact`	Ledger rows	Tabs Result / Evidence / Definition: status chip and code, reason, the five checks (Pass / Fails / Gap / Not reached / Not yet measured), legal bases, proof, evidence tiers, record fingerprint, flags. Link: "Open the evidence file for this number".
`action`	Action Queue rows, Overview top actions	Severity, domain, impact, Overdue; "What was measured" with first-found / found-again / resolved dates; example numbers linking to `/numbers/...`; fix step (with a "Draft · for your lawyers to review" pill when `fixStepDraft`). Editable:

Drawer	Opened from	Contents
		Assigned to (active users; disabled when CiV owns it), Due date, Status (Open / In progress / Resolved). Your decision (Accepted / Declined / Other plan + note → "Save decision"; shown only when the backend sends `decision`; "Your note stays in your workspace and is never sent to insurers."). Contest this finding (note → "Attach note"). Every change is `PATCH /actions/{id}`. The drawer is read-only unless the user is Admin or lawyer.
`vendor`	Vendor rows, bar list, search	Grade, score, month-on-month change, leads, high-signal leads, dispute candidates, signed terms, `ContractForm` (upload), `DisputeList`, rates behind the score (with a "Partial: x of y rates measured" note), score trend.
`who-area`	Who You Contact area tiles and check rows	Each check with its result, reason, source chip and metric; a masked contacts table (Number / Known / From / Flagged / Contacted); "Routed to counsel first."

5. Views and roles

Roles (`types.ts`): `Role = 'admin' | 'member'`.

- Admin: "Everything, including people, settings, evidence files and legal holds".
- Member: "Sees the portal and works on findings; no people, settings or evidence files".
- `isCounsel` is a separate yes/no **lawyer** mark on a user.
- CiV staff never use the portal. They use the CiV admin panel.

Views: `View = 'owner' | 'underwriter'`. A view is not a role. `Session.orgKind` is `client` or `insurer`.

`src/lib/views.ts`:

- `UNDERWRITER_PATHS = ['/portfolio','/exposure','/integrity','/attestation','/litigation','/uw-export']`
- `INSURER_ONLY_PATHS = ['/portfolio','/exposure','/litigation']`
- `viewBlock`:
  - An underwriter opening anything outside `UNDERWRITER_PATHS` gets `client-page`: "This page holds the client's own records … An underwriter sees scores and the attestation sheet, never contact-level records, findings or alerts" with a button to Portfolio.
  - A client opening an `INSURER_ONLY_PATHS` page gets `insurer-page`: "Only your insurer sees this page … You see your own attestation, data integrity and export" with buttons to Risk Attestation and Data Integrity.
  - The note in the code says the server refuses the data as well.

Navigation (`components/shell/nav.ts`):

Client sidebar = `NAV` + `SHARED_WITH_INSURER`:

- **Your position:** Overview `/`, Audit Scorecard `/scorecard`, Action Queue `/actions` (badge), Alerts `/alerts` (badge)
- **Evidence:** Consent Integrity `/consent`, Contact Ledger `/ledger`, Revocation Integrity `/revocation`, Who You Contact `/who-contact`, Evidence Vault `/vault`
- **Intelligence:** Contact Conduct `/conduct`, Lead Provenance `/leads`, Vendor Ledger `/vendors`
- **Disclosure:** Source Registry `/sources`, Reports & Exports `/reports`
- **Engagement:** Rulebook `/rulebook`, Regulatory Changes `/regulatory`
- **Shared with your insurer:** Data Integrity `/integrity`, Risk Attestation `/attestation`, Underwriting Export `/uw-export`
- Account menu (not in the sidebar): `/setup`, `/settings`, `/users`

Underwriter sidebar = `UNDERWRITER_NAV`. It is one group, titled "Falcon Risk · underwriting" in the sample, or "{clientName} · underwriting" for a real insurer. It holds Portfolio, Exposure Indicator, Data Integrity, Risk Attestation, Litigation Intelligence and Underwriting Export. A code comment records the owner's decision of 2026-10-22: no Engagement pages for the insurer. `/rulebook` and `/regulatory` are therefore blocked for an underwriter. Note that the mock's underwriter search still lists Rulebook and Regulatory Changes as pages, so a search hit can lead to a blocked page (a small inconsistency).

`useViewer()` (insurer screens):

- The underwriter sees the insured picked in `PortalContext.insuredId`. A real insurer whose list does not contain the current id gets the first insured in its list.
- A client always sees its own company, `'sunpath'`.
- Eyebrow text: "{org} · underwriting" or "Shared with your insurer".
- `InsuredPicker` (a select box) appears only for an underwriter.

Where each permission is checked in the UI. The comments say the server checks too.

Thing	Who may change it
Action: assign, date, status, decision, contest	Admin or lawyer (Members read only)
Invite, change role or lawyer mark, remove access	Admin. Cannot remove yourself; the last Admin is protected (mock and server).
Insurer sharing (create or stop buyer links)	Admin
Legal holds: set and release	Admin or lawyer (everyone sees the list)
Evidence file PDF/JSON export and "Check again at the source" for real numbers (`p-…`)	Admin or lawyer
Vendor contract upload	Admin or lawyer
Insured statements: sign and withdraw	Admin or lawyer
Proof request answer	the server says so through the `canAnswer` flag
Rule update review (agree/disagree) and "Suggest a change"	`isCounsel` only, and not in the underwriter view
Who You Contact lead-source switches	anyone except in the underwriter view

Thing	Who may change it
Settings, Users & Access	Admin only (others: "Only an Admin can open this page"; the server refuses them too)

Sign-in screen (/sign-in, outside the portal shell)

`SignInScreen` shows the brand panel (`SignInBrand`) and one of four forms. The form is chosen by `modeFromLink(reset, invite)`: `?reset=<uid>.<token>` gives the new-password form and `?invite=<token>` gives the invite form.

Form	Request(s)	Behaviour
SignInForm	`POST /session/sign-in {email,password}`	Goes to `/` on success. No "Forgot your password?": CiV staff send password links.
ResetRequestForm	`POST /session/reset {email}`	Always answers "If that email has an account, a reset link is on its way. It works for 1 hour."
NewPasswordForm	`POST /session/reset/confirm {link,password}`	Requires 12 or more characters and that the two entries match. Success: "Password changed … Other devices were signed out."
AcceptInviteForm	`GET /invites/check?token=` → `{email,name,clientName}`, then `POST /invites/accept {token,password}`	12 or more characters; signs the user in and goes to `/`.

All four requests are BACKEND with `BACKEND_URL` set (the `session` and `invites` paths). Without the backend, the mock answers `sign-in`, `sign-out` and `reset`. `reset/confirm` and `invites/*` have no mock (they return 404).

6. Page table

Legend for data sources (with `BACKEND_URL` set):

- **B** = fully from Django
- **M** = Django fields merged over the sample
- **S** = sample/mock only
- Without `BACKEND_URL`, everything is S.

`period` = follows the month picker.

Your position

Route	Screen	Shows	Hooks → source	User can
`/`	`overview/OverviewScreen`	AttentionBanner (red "Needs attention · N"): High alerts not reviewed; any cap holding the grade; High open actions; stale	useScore(period) M, useMetrics(period) M, useActions M, useAlerts B, useConsent B, useHealth B,	Click KPI cards (opens the metric drawer or navigates), click alerts and actions, follow

Route	Screen	Shows	Hooks → source	User can
		sources; records completeness below the floor; proof requests waiting. Each item is a link. Where you stand: `ScoreCard` (Audit Score, grade, caps, evidence coverage, confirmed-rules score, projected score, checkpoints run) + `ScoreTrendCard` ("Score history" with dated events). Data health: `DataHealthCard` (sources Live/Stale/Not supplied bar, data access level "Level N of 4", last change received, "read every 2 minutes, hourly catch-up"). Consent and domains: `StatusMixCard` + `CompactFactors` (six factors). Needs attention: `UrgentAlertsCard` ("Alerts to review") + `TopActions` ("Things to fix", the top 3). Key numbers: six cards — Consent coverage (M01), Contacts after opt-out (M10), Median time to stop (M08), Opt-outs your dialer missed (from `conversationReview`; otherwise "Not yet measured"), Records completeness (otherwise "Not available" / "Not yet measured"), Not measured (count of metrics).	useInsured(sunpath) M, useSession	banner links
`/scorecard`	`scorecard/ScorecardScreen`	KPIs: Audit Score, Domains measured (of 25), Not measured (domain codes). `FactorMap` ("Six factors": a tile per factor with its 0–100 value, weight, points added to D, and metrics; measured vs modeled). Checkpoint legend. `DomainRegister` ("Factor register" table: Code / Domain / Factor / Score / Grade / 4 months; unscored domains sink to the bottom).	useDomains(period) M, useScore(period) M, useMetrics(period) M, useInsured M	Open the domain drawer from a row; open the metric drawer from a factor tile
`/`	`actions/`	`ProjectedScore` (Today →	useActions M (writes B through	Filter: Open / Assigned to me /

Route	Screen	Shows	Hooks → source	User can
`actions`	`ActionsScreen`	Projected "if the top N actions are completed"). KPIs: Open actions, Overdue, Unassigned, Contested. "Queue" table: # / Action (pills "Finding contested", "Found again") / Severity / Domain / Who (You/CiV) / Score impact / Assigned to / Due / Status.	WRITE_PATHS), useScore M, useSession	Unassigned / Overdue / All. Row → action drawer (assign, due date, status, decision, contest) → `PATCH /actions/{id}`
`/alerts`	`alerts/AlertsScreen`	KPIs: Not reviewed, Reviewed, Email delivery. "Inbox" sorted by severity then time. `AlertRow`: severity, title, detail, kind, time, domain chip, "Resolved" (it went away by itself), "Reviewed by X". Admins are emailed the types chosen in Settings → Notifications.	useAlerts B	Filter: Not reviewed / All. "Mark as reviewed" → `POST /alerts/{id}/review` B

Evidence

Route	Screen	Shows	Hooks → source	User can
`/consent`	`consent/ConsentScreen`	Tabs. Summary: StatusMixCard ("Consent status mix"), StatusTrendCard ("Mix by month"), ReasonCodes ("Most common reasons"), LegalBasisCard ("Legal basis and certificates"), CertificatesCard ("Your certificates" table: Certificate / Source / State / Phone match / Found (claim-by date) / Made / Expires / Page / Used by), FiveChecks (fixed text: Proof exists → NO_PROOF; Proof can be opened → CONFLICTING; Proof matches the contact → NO_PROOF; Sources agree → CONFLICTING; No gaps in the record → WEAK). Certificate expiry: a KPI per horizon + "Certificates becoming unavailable" bars. Wording catalogue: a `ConsentPageCard` per	useConsent B, useCertificates B, useConsentPages B	Switch tabs. Per page, the "Your mark" segment (Approved / Not marked / Rejected) → `PATCH /consent-pages/{id} {mark}` B

Route	Screen	Shows	Hooks → source	User can
		consent form page (address, owner, first/last seen, leads, source label, CiV photo time and photo links, the consent wording quote, checklist x of y, changed date, missing items, item list with ✅/❌/⚠️/i/⏳, draft note).		
`/ledger`	`ledger/LedgerScreen`	`StatusTiles` (one card per consent status for this month; a click filters the table). "Contacts" table: When (UTC) / Number / Channel / Where / Category / Status / Reason / Flags. There is no Evidence column (a test checks this).	useContacts(status,q) B, useConsent B	Search a number (full or last 4 digits). Status filter: All / No proof / Conflicting / Weak / Verified / Pending / Not checked yet. Row → contact drawer
`/revocation`	`revocation/RevocationScreen`	`OptOutsCard` → `OptOutsTable` ("Opt-outs": Number / How / When / Deadline / Dialer marked / Contacted after; filters All / Open / Late / Contacted after STOP). KPIs: Median time to stop (→ M08 drawer), Slowest 10%, Not suppressed by deadline (deadline N business days), Contacted after deadline. `RevocationMatrix` ("Opt-out matrix": channel by system, hours / NEVER / not checked). "Time to stop" column chart (the "Never" column is flagged). `TestAuthorisation` ("Opt-out measurement": measured from the client's real opt-outs, no CiV test lines). `WordingCoverage` ("Opt-out wording coverage": Wording sent / Kind / Dialer / Messaging platform).	useRevocation M, useOptOuts B	Filter opt-outs; open the M08 drawer
`/who-contact`	`who-contact/WhoContactScreen`	Four `AreaTiles` (DNC, internal list, reassigned numbers, litigation; each coloured by its worst check). `WhoCheckedWhat` ("Who checked what" table). Rows are grouped as Check services,	useWhoContact M (writes B)	Tile or check → area drawer. Per lead vendor or list and per column, a toggle → `PATCH /who-contact/{sourceId} {column, checked}` (hidden in the underwriter view)

Route	Screen	Shows	Hooks → source	User can
		Lead vendors and Uploaded lists; the columns are the four lists; the legend chips are provider / list / stated / none. Note: "Checked by you" is shown to your insurer and counts as not done until the provider's results are attached. `ChecksList` ("Checks", Failing → Passing).		
`/vault`	`vault/VaultScreen`	KPIs: Records held (SHA-256 each), Daily anchors (last anchor time, or "No daily seal yet"), Deleted under retention (real count of files deleted after 4 years). "Coverage" `CoverageStrip` (12 months; engagement start date). `PositionLookup` ("Position lookup": phone + date → table Item / On record / Source; "Inside the covered period" or "Before the engagement · limited record"; a link to that number's evidence file when CiV has contacts with it, else "CiV has no contacts with this number..."; the sample note only in demo mode). `VerifyFile` ("Verify a file": the browser makes the file's SHA-256 and sends only that to `/api/hash-check` → Matches (a record read from a tool, a report or an evidence file/bundle CiV made, with its date) or No match; the file never leaves the browser). `LegalHolds` ("Legal holds" form + table: Hold on / Reason / Set by / Set / Status).	useVault M, usePosition B, useLegalHolds B, hash-check B, useSession	Look up a position. Verify a file. Set a hold (scope: a number / a vendor / a date range; target; reason) → `POST /legal-holds` B. "Release" → `POST /legal-holds/{id}/release` B (Admin or lawyer)
`/numbers/[phone]`	`evidence/EvidenceFileScreen`	The evidence file for one number (title = phone; subtitle = epoch and subscriber-period certainty). Pill "Under legal hold". "Came from: ...". Cards: Standing (worst status chip + a bar by status), List checks (do-	useEvidenceFile B (`/evidence/{phone}`), useSession	For real numbers (`p-...`) and Admin or lawyer: "Export this file" (downloads `/api/evidence-file/{phone}?format=pdf`), a "JSON" link (`?format=json`), and "Check again at the source" →

Route	Screen	Shows	Hooks → source	User can
		not-contact lists consulted: Listed / Not listed / Not measured), Anchor (Sealed / "Waiting for tonight's seal", records sealed x of y, time-stamped, sealed on, root fingerprint), `CallingWindowCard` (local calling window now; advice only), `OwnersCard` ("Owners of this number": Owner / From / To / Why CiV knows), `ConsentCard` (proofs on file for the current owner, needs, history, not yet measured), `ContactTimeline` (every contact oldest first, with opt-outs in sequence), `ProofElements` (eight elements: # / Element / On record / Source / Who holds it (consent page: "CiV (page photo, kept privately)"; third-party proof: claimed in the client's account / being claimed / not claimed in time / provider does not have it) / Captured by / Captured (UTC) / Fingerprint). An unknown number shows the error with a link "Back to the Contact Ledger".		`POST /evidence-file/{phone}/refetch` B (shows "Checked N records at the source: ..."). Members see "Only an Admin or your lawyer can export this file." For sample numbers: "Export this file" → `POST /reports {name:'Evidence file export · …'}`. Observed: the code renders two "Check again at the source" buttons for real numbers (one bound to `send`, one to `refetch`). This looks like a leftover duplicate.

Intelligence

Route	Screen	Shows	Hooks → source	User can
`/conduct`	`conduct/ConductScreen`	A `MetricTile` for each code in `conduct.metrics` (value, four-month trend, change since last month, note). "Contacts by hour of day" (recipient local time; hours outside 8am–9pm flagged). "Outside permitted hours, by state" (bar list with each state's window, plus "possible, location uncertain"	useConduct B, useMetrics(period) M, useSession	Tile → metric drawer

Route	Screen	Shows	Hooks → source	User can
		count). "Abandoned-call rate, by campaign" (30-day rate against a limit `abandonLimit`%; over the limit drawn in the second colour).		
`/leads`	`leads/LeadsScreen`	KPIs: Leads inspected, Consent completeness (→ M28), Page match (→ M27). "Leads by signal level" stacked bar (few 0–1 / some 2–3 / high 4 or more). "Most frequent signals", "Most often missing". Section "Where your numbers came from": `LadderCard` (provenance ladder, strongest first), `TopSourcesCard` (vendors in the CRM and websites in certificates, by numbers called), `UntracedCard` ("Untraced this month · N": Number / First call / Tool / CRM record / Why untraced), `UploadsCard` ("Uploaded lists": List / Numbers / With evidence / STOP numbers brought back / Proof rate). `ProofRequestsCard` ("Proof requests": Number / List / State / Your answer; files kept privately, opens logged, deleted after 4 years).	useLeads M, useProvenance B, useProofRequests B	Per proof request (when `canAnswer`): choose a file and a type (Signed form or contract / Call recording / Email or message / Other) → "Attach proof" (multipart `POST /proof-requests/{id}/answer` B), or "We have none" (`{none:true}`). States: Due date / Overdue / You have none / "Proof attached · CiV checking" / Checked: valid / Checked: not valid
`/vendors`	`vendors/VendorsScreen`	KPIs: Grades (mix), Dispute candidates (leads), Score-drop alerts. An empty-state note when there are no vendors ("CiV lists the vendors named in your CRM records or in vendor lead files you upload … scores each one once it has 100 leads"). "Vendor scores" ranked bars 0–100. "Vendors" table: Vendor / Leads / High-signal / Score / Grade / Month on month / 4 months / Signed terms / Dispute candidates. `AddContractCard` ("Vendor contracts" form + list: Vendor / File / Valid until / Return window / Status On file	useVendors M, useVendorContracts B, useDisputes B (in the drawer; only for vendors with a `contract` field, which means backend vendors), useSession	Row or bar → vendor drawer. Add a contract (vendor typed or picked, file, valid until, days to return a bad lead) → multipart `POST /vendor-contracts` B (Admin or lawyer). In the drawer: "Save contract", dispute list with "Download CSV" (`/api/vendor-disputes?vendor=…&csv=1`)

Route	Screen	Shows	Hooks → source	User can
		or Expired / Added).		

Disclosure

Route	Screen	Shows	Hooks → source	User can
`/sources`	`sources/SourcesScreen`	KPIs: Current (of N), Stale (names), Not supplied (what each blocks). "Sources by evidence tier" bar (Tier 1 = captured by CiV). "Sources" table: Source / Tier / Access / Last sync / Status / Feeds domains. `UploadCentre` ("Upload centre": each file fingerprinted on arrival and stored write-once; table File / Source / Received / Fingerprint / Rows read / Rows not read).	useSources B, useUploads B	Upload: choose a source that accepts uploads (CSV; "Policies, training, contracts" also takes pdf/doc/docx) → multipart `POST /uploads {source, file}` B
`/reports`	`reports/ReportsScreen`	Subtitle "Downloads for {month}". A "Download the {month} audit report (PDF)" button (`QuickReportDownload` → `GET /api/reports/period?period=YYYY-MM` B: makes it if needed or sends the latest). A `ReportCard` per report type (name, description, format pill, add-on → "Request a quote"). `BulkExport` ("Bulk evidence export"). `ReportHistory` ("History": Report / Period / Requested by / When / Status Ready / Held for review (blocked by the wording check) / Not available yet / Queued + Download link).	useReports M (writes B), useSession	"Generate" → `POST /reports {name, period}` B. If the reply has a `url`, the file downloads. The `rulebook` report downloads at once from `/api/rulebook/snapshot?period=YYYY-MM` (B). Bulk export (Admin or lawyer; greyed out for Members): paste up to 25 numbers → "Export N files" or "With certification pack" (raw `POST /api/evidence-bundle {numbers, pack}` → ZIP; the `X-Not-Found` header gives the count not found). On a 404 (no backend) it falls back to `POST /reports` (queued).

Engagement

Route	Screen	Shows	Hooks → source	User can
`/rulebook`	`rulebook/RulebookScreen`	`note` paragraph (from Django). Tab Settings: KPIs Set / Proposed ("Runs; results marked provisional") / TBD ("dependent checks do not	useRulebook B, useRulebookCalendar(year) B, useCanCall B	Search rules, filter All / Set / Proposed / TBD, switch tabs, change year. "Can I call now?": type a number → `POST /can-call` (the verdict pill, state and

Route	Screen	Shows	Hooks → source	User can
		run"); a version card with a bar of rules by status; "Settings in force" table (Rule + meaning / Place / Value in force / Owner (+ counsel item) / Status); `ImpactTable` only when `impact` is not empty (the mock always sends empty; the comment says CiV staff only). Tab Calendar & hours: `CanCallBox` ("Can I call now?"), `HolidaysCard` ("Business-day calendar" with a year picker), `PlaceRulesCard` "Calling hours" and "Contact limits", "Opt-out timing" (deadline business days, confirmation text minutes, stop keywords, texts follow calling hours, state holiday restrictions, each with a status pill), "Which time zone we use" (ordered fallback steps).		confidence, allowed hours (draft), each zone's local time inside/outside, note; the number is not stored)
`/regulatory`	`regulatory/RegulatoryScreen`	A `ChangeCard` per change, sorted with unconfirmed first, then newest: status pill (adopted / in force / proposed / court ruling), jurisdiction and date, title, summary, takes effect, Rulebook settings touched, metrics and domains touched, source link. "New rule values": a `RuleUpdateRow` each (rule, "New rule" tag, place, new value from a date, meaning, Draft / In the Rulebook, "Your lawyer X agrees/disagrees: comment"). `SuggestChange` card (lawyer only). Footer text about CiV approval.	useRegChanges B, useSession	"Mark as reviewed" → `POST /reg-changes/{id}/confirm` B. Lawyer only: "Review as our lawyer" / "Change review" → comment + Agree / Disagree → `POST /rule-updates/{id}/review {verdict, comment}` B. "Suggest a change" form (What changed, Where, Summary, Source, Link) → `POST /reg-changes/suggest` B ("Thank you. CiV received it...")

Shared with your insurer (client) / insurer pages (underwriter)

Route	Screen	Who	Shows	Hooks → source	User can
`/integrity`	`insurer/IntegrityScreen`	both	Note "Unverified counts as worst case". KPIs: Evidence coverage (below 60% → grade capped at C), Records completeness (logs ÷ invoices; below 0.9 flags the period), Statement conflicts, Unknown caller IDs. `BasisMixCard` ("What the score rests on": Source grade / Share / Weight / Counts toward coverage). `ConflictsCard` ("Statements that conflict with observation"). `ReconciliationCard` ("Reconciliation against independent totals": What CiV was given / Checked against / Records / Independent total / Ratio / Result / Source of total).	useInsured(viewer id) M (for a real buyer, Django's data as it is)	Underwriter: `InsuredPicker`
`/attestation`	`insurer/AttestationScreen`	both	KPIs: Evidence coverage, Fields at worst case, Sheet fingerprint (as-of date), Monitoring. A. Frequency drivers table (Field / Value / Source grade / Source): marketing contacts/month, prerecorded or AI voice share, purchased-lead share, manual-import share · evidence found, lead vendors · with signed indemnification, consent proof rate, opt-out median · not honored, contacts after opt-out · re-added, internal-list contacts, reassigned exposure, reassigned check before calling, unknown caller IDs, prior matters 3y (stated vs dockets). Worst-case notes: 20% prerecorded, 60% purchased, 60% consent. B. Records integrity: completeness, *theoretical statutory exposure 12 months*	useInsured M, useSession	Client Admin or lawyer: add a statement (topics: dnc_checked, internal_list, optouts_honored, consent_all, other + text) → `POST /statements {topic,text}` B ("Sign and add"); "Withdraw" → `POST /statements/{id}/withdraw` B. Underwriter: picker

Route	Screen	Who	Shows	Hooks → source	User can
			($500–$1,500 per unconsented contact) — underwriter only, records fingerprinted, policy · training currency, opt-out tests in 90 days. A client sees "Dollar figures are not shown in your portal" (decision D36). C. Insured states: conflicts + statements, "Signed by ...". For a client (when `statementRows` is present): `StatementForm`.		
`/uw-export`	`insurer/ExportScreen`	both	"Attestation payload" (`civ.attestation.v1`), shown in plain words (`PlainPayload`). The payload holds insured, as_of, sheet_sha256, frequency{...}, evidence{...}. The underwriter only also gets `exposure_indicator` (model v2.2, inside_out/outside_in, factors, defect term, multipliers, grade, capped) and `model_v0` (p_suit_12m, expected_cost, expected_annual_loss). The subtitle quotes the API shape `GET /v1/insureds/{id}/attestation?as_of=2026-09` (descriptive text; this frontend does not call it). `CoverageConditions` ("Conditions of coverage", draft for the carrier's counsel). "Monthly feed" (first business day; webhook on a 5-point change or a new open matter; history kept with fingerprints). Underwriter: `RatingEngineCard`.	useInsured M, useRatingEngine B (SAMPLE_WHEN_REFUSED), useSend	"Copy JSON" (clipboard). Underwriter: "Send to rating engine" → `POST /rating-engine/send {insured}` (a refusal shows a plain message such as "That address does not accept the sheet...", never the other server's page; the send history refreshes after every send; a client previewing its insurer's view sees neither the button nor the card). Rating engine card: address (https), key (password field; "Saved, ends in XXXX"), "Send every month on its own (the 6th, after the month is locked)" → `POST /rating-engine {url,key,monthly}` ("Saved."); "Remove" → `DELETE /rating-engine` ("Removed."). A list of past sends. Text: "Each sheet is signed (HMAC-SHA256 with your key)".
`/portfoli`	`insurer/PortfolioScreen`	underwriter only	KPIs: Insureds (monitored / outside-in only), Portfolio expected annual loss ("Loss	useInsureds (always asked afresh) — a real insurer gets only companies that share	Row → set the insured and go to `/attestation`

Route	Screen	Who	Shows	Hooks → source	User can
o			model v0, uncalibrated"), Below 60% evidence coverage, Statement conflicts. "Insureds" table: Insured (industry, contacts/mo, monitoring) / Exposure Indicator / Evidence (coverage chip, records completeness) / Flags (conflicts, unknown caller IDs) / Consent proof (worst case 60%) / Chance of suit per year / Expected annual loss ("Not measured" when there are no contacts).	with it right now, a client in its own Underwriter view gets only its own company with real data; "No company shares with you yet" when empty; the top bar shows the real count	
/exposure	insurer/ExposureScreen	underwriter only	Note "Proposed headline, pending sign-off. Shown in the insurer view only; clients keep the Audit Score (decision D1)". KPIs: Exposure Indicator (grade; measured or modeled; capped/raw), Defect term D, Pressure multipliers (×; venue / targeting / volume), Evidence coverage. FactorTable (Factor / Weight / Outside-in modeled / Inside-out measured). "No reading yet." when there is no reading.	useInsured M	Picker
/litigation	insurer/LitigationScreen	underwriter only	Headline cards (label, value, note, status Verified / Estimate / Statute, source link). "Class actions filed in {month}, by year" columns (latest year flagged; source TCPAWorld by default). "What a suit costs". "What drives the filings" (Claim type / Legal hook / Direction since 2024 / Share and Median cost shown as "Study output" placeholders). "The litigation study: how it gets built" (Step / How / Cost and time; decision D29; federal only first).	useLitigation M (for a real buyer: overlay over the sample litigation)	Read only

Account (account menu)

Route	Screen	Shows	Hooks → source	User can
`/setup`	`setup/SetupScreen`	Tabs. Readiness report: KPIs "Can the audit start?" (Yes / Not yet; test date, sample numbers), Checks ready (x of N; partial / missing), Recommended plan (numbers/month), Would be not measured (metrics). "Prerequisite checks" bar (Ready / Partial / Missing). "Preview on the sample" (status mix). "What each check found" (Check + how / Result / What was found / Fix). Setup checklist: a meter and the items, outstanding first (toggle, label, detail, count, Complete / Outstanding).	useReadiness M	Toggle an item (Admin only; others see it greyed out) → `PATCH /setup/{id} {done}` B, saved per company; rows CiV counts itself fill in automatically, the rest say "Not marked yet" until ticked
`/settings`	`settings/SettingsScreen`	Tabs. Company: name, industry, brands, states contacted, litigation forum, engaged since ("To change any of these, write to your CiV contact"); a Contacts list. Notifications: "Email notifications" toggles per alert kind (Emailed / Inbox only), emailed to the company's Admins; "Slack and webhook delivery are planned for a later release." Plan & usage: "{plan} plan" ({price}/month), used of limit with a meter, % used, months over the limit, the move-up rule and next plan, "Download the list of billed numbers" (`/api/billed-numbers` CSV, B); "Numbers contacted by month" columns with a limit line. Agreements: Document / Status Signed or Pending / Date / Covers. Insurer sharing: `SharingTab`.	useSettings M, useSession, useBuyerLinks B	The page is Admin only. Notification toggle → `PATCH /settings/email/notifications {kind,email}` B (saved per company). Sharing (Admin): Company, underwriter's email, their name, Insurer/Acquirer, Until (needed for an acquirer per the label; optional for an insurer) → `POST /buyer-links` B ("Shared. An invite to set a password was sent to them." when `invited`); a table Shared with / Since / Until / Status with "Stop sharing" → `POST /buyer-links/{id}/revoke` B (message "Stopped sharing with ..."); an insurer who already has an account gets a "now shares with you" email instead of an invite. Text: the insurer sees attestation, data integrity and exposure, never contacts, numbers, findings or notes
`/users`	`users/UsersScreen`	Tabs. People: `InviteForm` (Admin only: Name, Email, Role Admin/Member, "Is our lawyer" →	useUsers B, useAccessLog B, useSession	`POST /users`, `PATCH /users/{id} {role`

Route	Screen	Shows	Hooks → source	User can
		"Send invitation"; explains the role; a lawyer can also review rule drafts, suggest law changes and sign statements) + `UserTable` ("People with access": Person / Role (select if Admin and active) / Lawyer checkbox / Status Active, Invited or Invite expired (with sent date) / Last seen / "Resend" or "Resend invite", "Remove access" or "Cancel invite"). Access log: When / Who / Role / Did / What ("Every view of an evidence file, every export and every change").		

Not in the portal shell

- `/sign-in` (see §5).
- `/api/[...path]` (the route handler).

7. Other cross-cutting UI behaviour

- `Loader` wraps every query: it shows a loading state, an error message, or renders its children with the data.
- `NotMeasured` / `NOT_MEASURED`: "Not measured" is its own state and is never shown as a worst-case number. This shows up in the Key numbers, drawers, data health and insurer pages, and is tested in `datahealth.test.tsx` and `realInsurer.test.tsx`.
- Wording guard (`lib/wording.ts`): banned words in text a client can see. Report runs can be "Held for review" by the backend's wording check.
- Masked numbers and fingerprints: `truncateHash`; who-area flags show masked numbers.
- All times are labelled UTC.

8. Sample identities in the mock (for reference)

- Client: SunPath Residential Solar (`sunpath`), user Dana Reyes (Admin), plan Growth, months Jun–Sep 2026.
- The two runs: "Nightly run 30 Sep 2026" (Rulebook v1.3, approved, visible to the client) and "Provisional run · strict" (v1.4-p, internal, hidden).
- Underwriter: J. Merchant, Falcon Risk Services, 5 insureds.
- `OWN_INSURED = 'sunpath'`.

9. Tests (frontend/tests, run with npm test = vitest run)

Setup (`vitest.config.ts` + `vitest.setup.ts`):

- jsdom environment, globals on, alias `@ → src`, CSS off, includes `tests/**/*.test.{ts,tsx}`.
- MSW (`tests/msw.ts`) sends every `*/api/*` call through the same `mockApi` the dev server uses. `onUnhandledRequest: 'error'`, and `resetMock()` runs after each test.
- `next/link` and `next/navigation` are mocked (`tests/nav.ts` keeps a mutable pathname, params and a `push` spy).
- There is an in-memory `localStorage` shim.
- `tests/utils.tsx` `renderScreen()` wraps a screen in QueryClient + PortalProvider + DrawerProvider + DrawerHost.

Result of `npx vitest run` on 2026-10-22: **26 files, 150 tests, all passed** (about 10 s).

Kinds of tests:

- **lib** (pure functions):
  - `overlay.test.ts`: merge rules, including metrics by code, areas by id and checks by name, and litigation cards by id.
  - `buyerSession.test.ts`: `mergeSession` for a real insurer.
  - `exposurePending.test.ts`: pending factors have their weight spread over the measured ones.
  - `wording.test.ts`: the banned-word guard.
- **mock** (`mock/api.test.ts`):
  - 25 domains and 33 metrics.
  - The score averages only measured, non-excluded domains.
  - Periods.
  - Internal runs and rulebook impact are never shown.
  - Admin-only people changes and the last-Admin rule.
  - Sign-out refusal (401).
  - Action overdue is recomputed.
  - Search.
  - Hash check.
  - Underwriter vs client data refusals.
- **screens** (Testing Library + MSW; the backend is simulated by overriding MSW handlers in some tests):
  - `portal.test.tsx` (42 tests): attention bar; a menu entry for every screen; shell (menu, disclaimer, badge count, account menu, redirect to sign-in when signed out, Admin-only Settings and Users, search → domain drawer); interactions (domain drawer tabs, six factors → metric drawer, who-contact area and switch, assign action and contest, ledger filter, no Evidence column, mark alert reviewed, rulebook note and calendar tab, lawyer-only rule review and suggest, opt-outs with

deadlines and the late filter, legal hold set and release, evidence file and unknown number, invite and remove user, rulebook impact never shown, sign in → overview).

- The module-specific files: insurer (view switch, menus, blocks, dollar figures only for the underwriter, the export leaves out estimates), realInsurer, ratingEngine, statementForm (+ plain payload), sharingTab, consent (status names, number-page consent card, Wording tab, ledger status cards), leads (proof requests), numbers (number page "Not checked yet", calling window, owners, "Can I call now?" stricter zone, opt-outs card filters), datahealth (not-yet-measured states, completeness box), decision and findings (Module 11 action decisions and findings), alertRow, reportHistory, banners (revised month), scoreLabels (partial score / draft bands), vendorPartial (partial rates, no vendors), contractForm, addContract, bulkExport, evidenceExport.

Other commands: `npm run lint`, `npm run typecheck`, `npm run build`. I did not run these (they would only report; they were not needed for these notes).